

Introduction to Deep Learning

Sorbonne Université - Master 2 - Probabilités et Finance

- P. Gallinari, patrick.gallinari@sorbonne-universite.fr, <https://pages.isir.upmc.fr/gallinari/>
- Olivier Schwander, olivier.schwandler@sorbonne-universite.fr

Année 2025-2026

Course Outline and Organization

- ▶ Introductory ML course with a focus on Neural Networks and Deep Learning
- ▶ Organization
 - ▶ Part I: Introduction and classical machine learning by B. Wilbertz
 - ▶ Part 2: Neural Networks and Deep Learning – P. Gallinari – O. Schwander
 - ▶ Courses 15 h
 - ▶ Practice and exercises 12 h
- ▶ Outline
 - ▶ Neural Networks and Deep Learning
 - ▶ Introductory Concepts - Perceptron-Adaline
 - ▶ Linear Regression and Logistic Regression - Optimization Basics
 - ▶ Multilayer Perceptrons – Generalization Properties
 - ▶ Convolutional Neural Networks – Vision applications
 - ▶ Stochastic Generative Models

Ressources

► Books

- The following two books cover the course (more or less)
 - Understanding Deep Learning by S.J.D. Prince 2023 is a recent book covering many topics from the course?
 - Does not delve into the details but provides a good overview of the domain bases
 - Available at <http://udlbook.com>
 - Deep Learning, Foundations and concepts, by C. Bishop
 - <https://www.bishopbook.com/>
- This one is older (no transformers etc, but is a valuable ressource)
 - Pattern recognition and Machine Learning, C. Bishop, Springer, 2006
 - Chapters 3, 4, 5, 6, 7, 9,
- First deep learning book
 - Easy to read, somewhat outdated now but still a good reference
 - Deep Learning, An MIT Press book, I. Goodfellow, Y. Bengio and A. Courville, 2017
 - <http://www.deeplearningbook.org/>



Neural Networks and Deep Learning



Context

Context

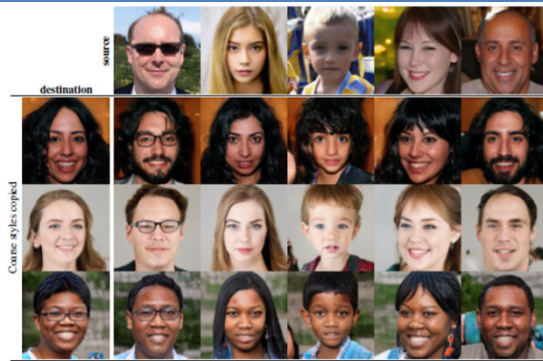
Deep Learning today

- ▶ Deep Learning is today the most popular paradigm in data science
- ▶ Popularized since 2006, first by some academic actors and then by big players (GAFAs, BATs, etc)
- ▶ It has initiated a « paradigm shift » in the field of data science / AI and definitely changed the way one will exploit data
 - ▶ e.g. key players have made available development platforms (initiated e.g. with TensorFlow, PyTorch, Jax)
 - ▶ Allowing the development in a « short time » of complex processing chains
 - ▶ Making complex DL methods available for a large community
- ▶ Today DL is developing at a much larger scale, including
 - ▶ Software development platforms and environments
 - ▶ Services in multiple domains: biotech, health, finance, client management, etc
 - ▶ AI4Science is an emerging discipline

Machine Learning successes

- ▶ Mainly concern the numerical world and GAFAs/BATs applications
 - ▶ Semantic data analysis: vision, speech, language, traces;
 - ▶ Virtual worlds, e.g. games

Generative models - (Karras et al. 2019) – Style GAN - NVIDIA



Generative models 2022 Stable-diffusion <https://stablediffusionweb.com/>

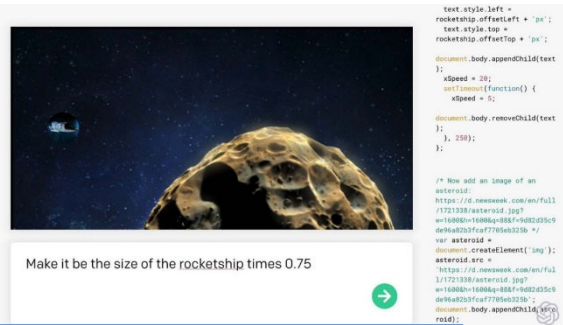


Alphastar, Vinyals et al. 2019 (Starcraft) - Deepmind

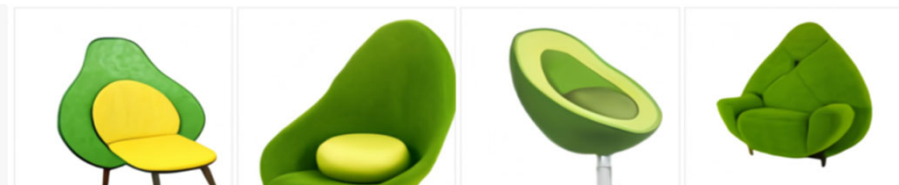


ChatGPT 2022

7



OpenAI-Codex 2021
natural language to code



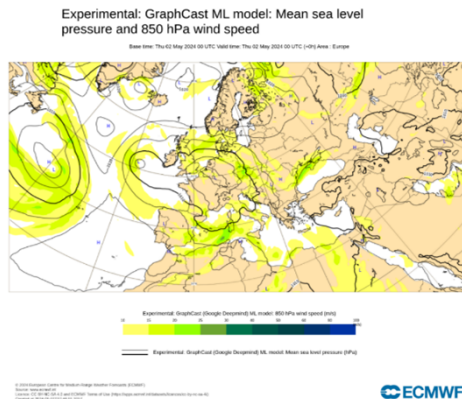
DALL.E - 2021 <https://openai.com/blog/dall-e/>
Text: an armchair in the shape of an avocado. . . .

Machine learning successes

- ▶ Progressively targets other domains
- ▶ Examples AI4Science

Weather forecast
GraphCast – Google
& DeepMind 2022

[ECMWF website](https://www.ecmwf.int/en/forecasts/graphcast)



Foundation models
Spatio-temporal

dynamics –

Hao et al. (ICML 2024)

<http://arxiv.org/abs/2403.03542>

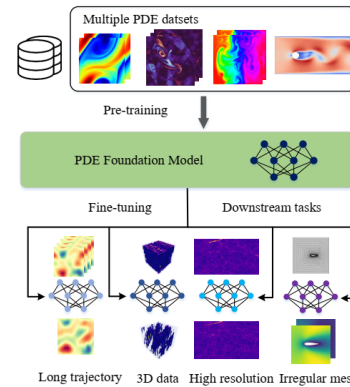
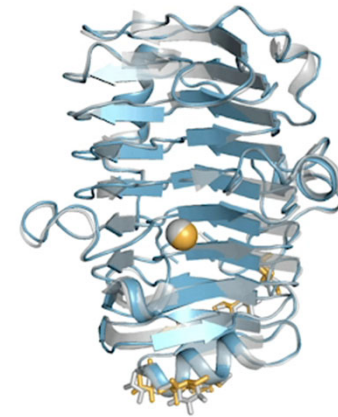


Figure 1. An illustration of pre-training a PDE foundation model using massive data from multiple PDE datasets. The pre-trained model is then used for fine-tuning different downstream operator learning tasks, which can be complex. (Best viewed in color)

Google DeepMind - Alphafold 3
3D protein structure prediction

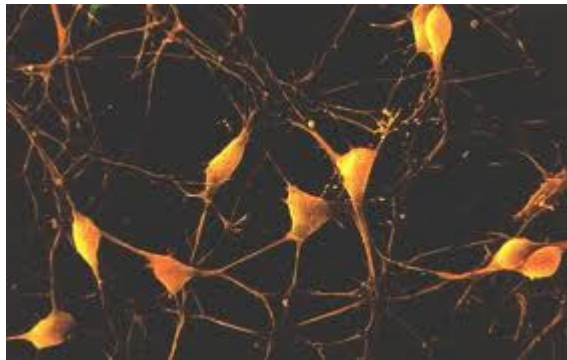


Introductory NN concepts

Intuitive introduction via 2 simple –historical- models
Perceptrons and Adalines

Neural Networks inspired Machine Learning

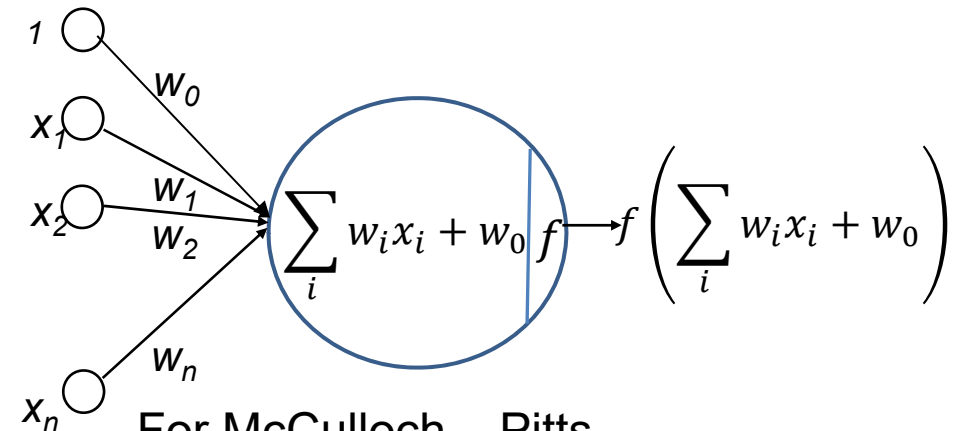
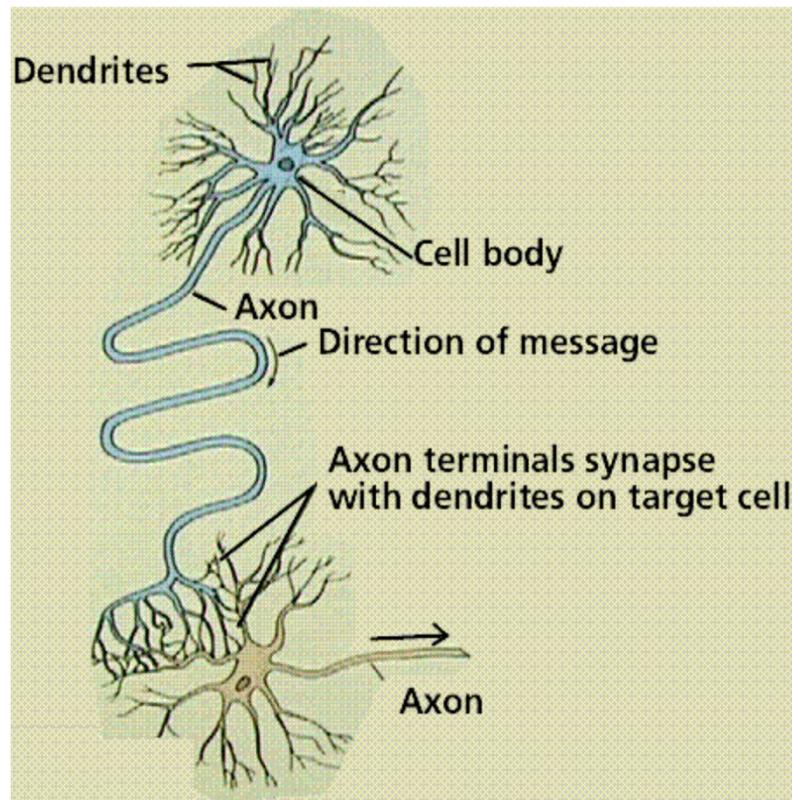
Brain metaphor



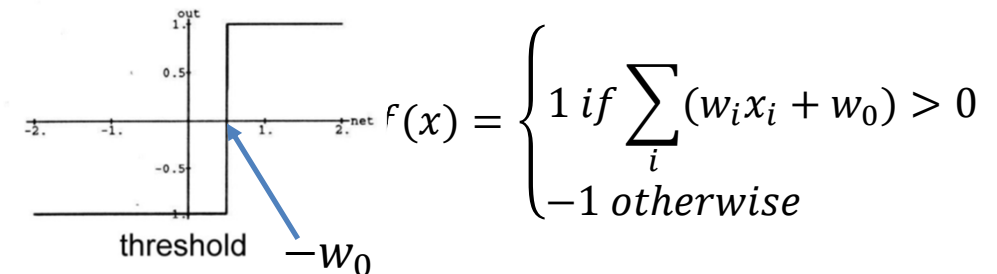
- ▶ Artificial Neural Networks are an important paradigm in Statistical Machine learning and Artificial Intelligence
- ▶ Human brain is used as a source of inspiration and as a **metaphor** for developing Artificial NN
 - ▶ Human brain is a dense network 10^{11} of simple computing units, the neurons. Each neuron is connected – in mean- to 10^4 neurons.
 - ▶ Brain as a computation model
 - ▶ Distributed computations by simple processing units
 - ▶ Information and control are distributed
 - ▶ Learning is performed by observing/ analyzing huge quantities of data and also by trials and errors

Formal Model of the Neuron

McCulloch – Pitts 1943

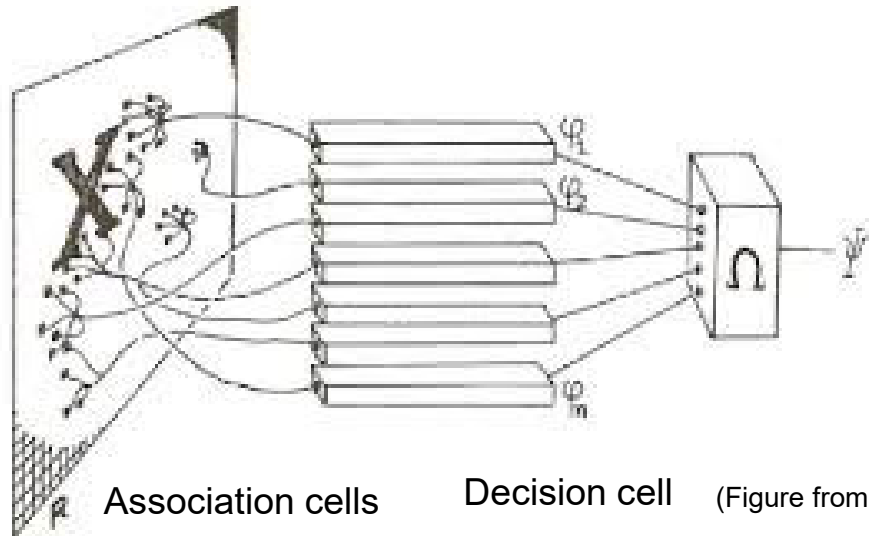


For McCulloch – Pitts neuron, f is a threshold (sign) function

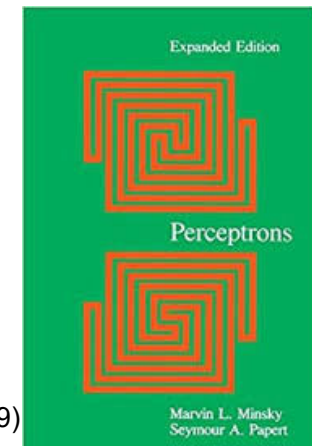


A synchronous assembly of neurons is capable of universal computations (aka equivalent to a Turing machine)

Perceptron (1958 Rosenblatt)



(Figure from Perceptrons, Minsky and Papert 1969)



- ▶ The decision cell is a threshold function (McCulloch – Pitts neuron)
 - ▶ $F(x) = \text{sgn}(\sum_{i=1}^n w_i x_i + w_0)$
- ▶ This simple perceptron can perform 2 classes classification

Perceptron Algorithm (2 classes)

Data

Labeled Dataset $\{(x^i, y^i), i = 1..N, x \in R^n, y \in \{-1, 1\}\}$

Output

classifier $w \in R^n$, decision $F(x) = \text{sgn}(\sum_{i=0}^n w_i x_i)$

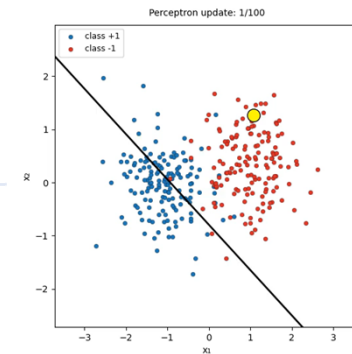
Initialize $w(0)$

Repeat (t)

Choose an example $(x(t), y(t))$

if $y(t)w(t) \cdot x(t) \leq 0$ then $w(t+1) = w(t) + \epsilon y(t)x(t)$

Until convergence



Training set
Classifier specification

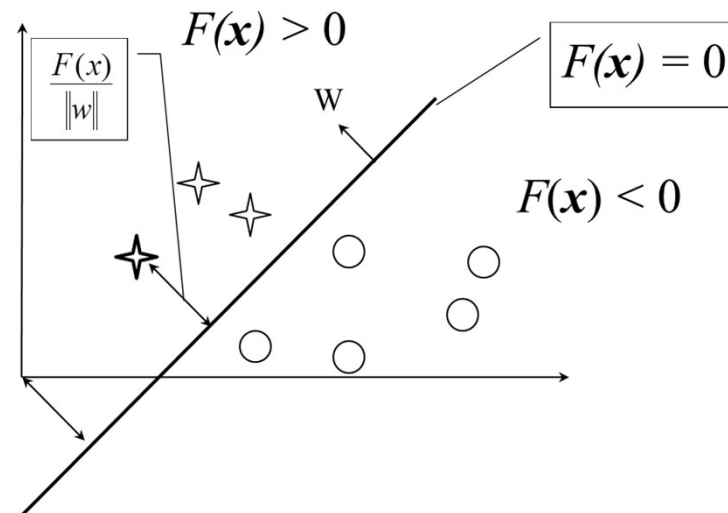
Stochastic
Algorithm

- ▶ The learning rule is a **stochastic gradient algorithm** for minimizing the number of wrongly predicted labels
- ▶ Multiple (p) classes: p perceptrons in parallel, 1 class versus all others!

Linear discriminant function

$$F(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + w_0 = \sum_{i=0}^n w_i x_i \text{ with } x_0 = 1$$

- ▶ Decision surface : hyperplane $F(\mathbf{x}) = 0$
- ▶ Properties
 - ▶ $\mathbf{w}$ is a normal vector to the hyperplane, it defines its orientation
 - ▶ distance from $\mathbf{x}$ to $H : r = F(\mathbf{x}) / \|\mathbf{w}\|$
 - ▶ if $w_0 = 0$ H goes through the origin



Perceptron algorithm performs a stochastic gradient descent

► Loss function

- $\mathcal{C} = -\sum_{(x,y)\text{missclassified}} \mathbf{w} \cdot \mathbf{x}y = -\sum_{(x,y)\text{miss-classified}} c(\mathbf{x}, y)$
- Objective : minimize $\mathcal{C}$

► gradient

- $grad_{\mathbf{w}}\mathcal{C} = \left(\frac{\partial \mathcal{C}}{\partial w_1}, \dots, \frac{\partial \mathcal{C}}{\partial w_n}\right)^T$ with $\frac{\partial \mathcal{C}}{\partial w_i} = -\sum_{(x,d)\text{missclassified}} x_i y$

► Learning rule

- Stochastic gradient descent for minimizing loss $\mathcal{C}$
- Repeat (t)
 - Choose an example $(\mathbf{x}(t), y(t))$
 - $\mathbf{w}(t) = \mathbf{w}(t-1) - \epsilon grad_{\mathbf{w}}c(\mathbf{x}, y)$

Multi-class generalization

▶ Usual approach: one vs all

- ▶ p classes = p " 2 class problems " : class C_i against the others
 - ▶ Learn p discriminant functions $F_i(x), i = 1 \dots p$
 - ▶ Decision rule: $x \in C_i$ if $F_i(x) > F_j(x)$ for $j \neq i$
 - ▶ This creates a partition of the input space
 - ▶ Each class is a polygon with at most $p - 1$ faces.
- ▶ Convex regions: limits the expressive power of linear classifiers

Perceptron properties (1958 Rosenblatt)

► **Convergence** theorem (Novikof, 1962)

- Let $D = \{(x^1, y^1), \dots, (x^N, y^N)\}$ a data sample. If
 - $R = \max_{1 \leq i \leq N} \|x^i\|$
 - $\sup_w \min_i y^i (w \cdot x^i) > \rho$ (ρ is called a margin)
 - The training sequence is presented a sufficient number of time
- The algorithm will converge after at most $\left\lceil \frac{R^2}{\rho^2} \right\rceil$ corrections

► **Generalization** bound (Aizerman, 1964)

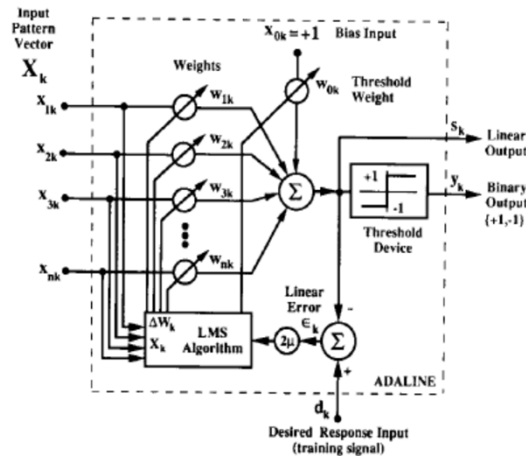
- If in addition we provide the following stopping rule:
 - Perceptron stops if after correction number k , the next $m_k = \frac{1+2 \ln k - \ln \eta}{-\ln(1-\epsilon)}$ data are correctly recognized
- Then
 - the perceptron will converge in at most $l \leq \frac{1+4 \ln R/\rho - \ln \eta}{-\ln(1-\epsilon)} \lceil R^2/\rho^2 \rceil$ steps
 - with probability $1 - \eta$, test error is less than ϵ

Link between training and generalization performance

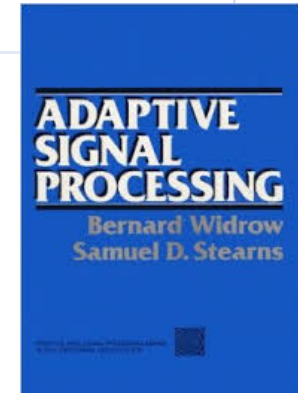
Convergence proof (Novikof)

- ▶ Hyp: lets take $w^* / \|w^*\| = 1$
 - ▶ $w_0 = 0, w_{t-1}$ is the weight vector before the t^{th} correction
 - ▶ $w_t = w_{t-1} + \epsilon y^t x^t$
 - ▶ $w_t \cdot w^* = w_{t-1} \cdot w^* + \epsilon y^t x^t \cdot w^* \geq w_{t-1} \cdot w^* + \epsilon \rho$
 - ▶ By induction $w_t \cdot w^* \geq t\epsilon\rho$
- ▶ $\|w_t\|^2 = \|w_{t-1}\|^2 + 2\epsilon y^t w_{t-1} \cdot x^t + \epsilon^2 \|x^t\|^2$
- ▶ $\|w_t\|^2 \leq \|w_{t-1}\|^2 + \epsilon^2 \|x_t\|^2$ since $y^t w_{t-1} \cdot x^t < 0$ (remember that x^t is incorrectly classified)
- ▶ $\|w_t\|^2 \leq \|w_{t-1}\|^2 + \epsilon^2 R^2$
- ▶ By induction $\|w_t\|^2 \leq t\epsilon^2 R^2$
- ▶ $t\epsilon\rho \leq w_t \cdot w^* \leq \|w_t\| \|w^*\| \leq \sqrt{t}\epsilon R \|w^*\|$
- ▶ $t \leq \frac{R^2}{\rho^2} \|w^*\|^2 = \frac{R^2}{\rho^2}$

Adaline – Adaptive Linear Element (Widrow - Hoff 1959)



$$\text{Linear unit: } F(x) = \sum_i w_i x_i + w_0$$

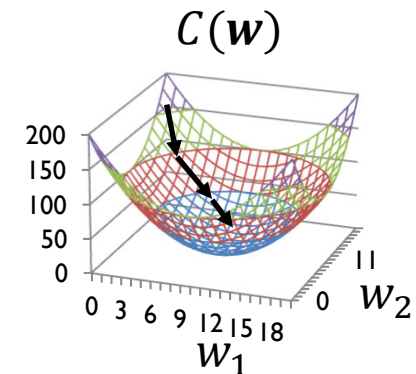


- ▶ « Least Mean Square » LMS algorithm
 - ▶ Loss: $c(x, y) = \|y - F(x)\|^2$
 - ▶ Algorithm: Stochastic Gradient Descent (Robbins – Monro (1951))

Iterate

Choose an example $(x(t), y(t))$
 $w(t + 1) = w(t) - \epsilon \nabla_w c(x, y)$

- ▶ Workhorse algorithm of adaptive signal processing: filtering, equalization, etc.



Adaline example motivating the need for adaptivity from an engineering perspective

► Adaptive noise cancelling

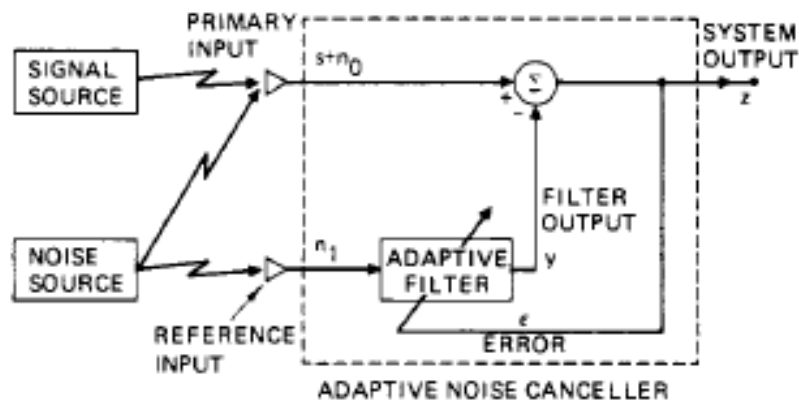


Fig. 1. The adaptive noise cancelling concept.



Widrow-Science in Action - YouTut

Fig. from Adaptive Signal Processing, Widrow, Stearn

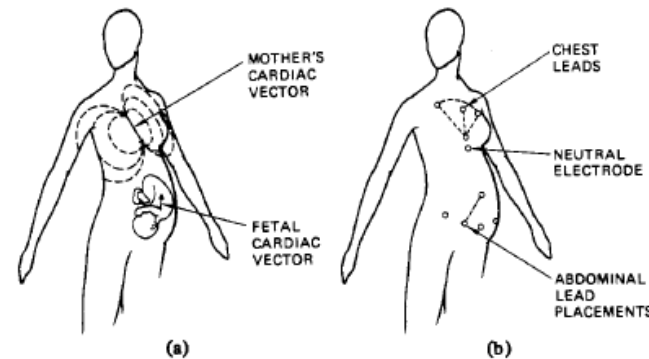


Fig. 14. Cancelling maternal heartbeat in fetal electrocardiography. (a) Cardiac electric field vectors of mother and fetus. (b) Placement of leads.

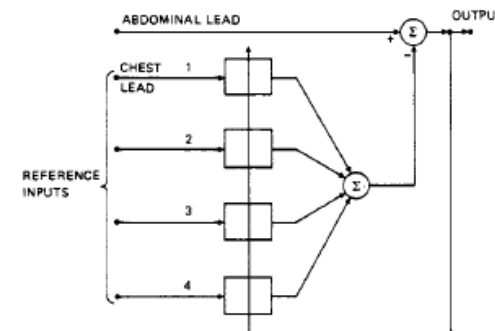


Fig. 15. Multiple-reference noise canceller used in fetal ECG experiment.

Heartbeat cancelling

Objective: get z as close as possible to the baby signal s

Adaline – heartbeat cancelling detailed

- ▶ With the notations of the Figure
- ▶ Hyp.:
 - ▶ s, n_0, n_1, y are stationary with zero means
 - ▶ s is uncorrelated with n_0, n_1 and then y
- ▶ Filtering scheme
 - ▶ output $z = s + n_0 - y$
 - ▶ Loss function to be minimized $E[z^2]$
- ▶ Then
 - ▶ $z^2 = s^2 + (n_0 - y)^2 + 2s(n_0 - y)$
 - ▶ $E[z^2] = E[s^2] + E[(n_0 - y)^2] + 2E[s(n_0 - y)]$
 - ▶ $E[z^2] = E[s^2] + E[(n_0 - y)^2]$ since s and $(n_0 - y)$ are not correlated
- ▶ So that
 - ▶ $\text{Min } E[z^2] = E[s^2] + \text{Min } E[(n_0 - y)^2]$
- ▶ When the filter is trained to minimize $E[z^2]$, it also minimizes $E[(n_0 - y)^2]$
- ▶ Then y is the best LMS estimate of n_0 , and z is the best LMS estimate of signal s (since $z - s = n_0 - y$)

Introductory concepts

Summary of key ideas

- ▶ Learning from examples
 - ▶ Perceptron and Adaline are supervised learning algorithm
 - ▶ Training and test set concepts
 - ▶ Parameters are learned from a training set, performance is evaluated on a test set
 - ▶ Supervised means each example is a couple (x, y)
- ▶ Stochastic optimization algorithms
 - ▶ Training requires exploring the parameter space of the model (the weights)
 - ▶ For NNs, most optimization methods are based on stochastic gradient descent
- ▶ Generalization properties
 - ▶ Learning $\neq$ Optimization
 - ▶ One wants to learn functions that generalize well



Optimisation : gradient methods – introduction



Optimization

Batch gradient algorithms

► Batch gradient general scheme

► Training Data Set

► $D = \{(\mathbf{x}^1, \mathbf{y}^1), \dots, (\mathbf{x}^N, \mathbf{y}^N)\}$

► Objective

- Optimize a loss function $C(\mathbf{w}) = \sum_{i=1}^N c_{\mathbf{w}}(\mathbf{x}^i, \mathbf{y}^i)$
- Sum of individual losses $c_{\mathbf{w}}(\cdot, \cdot)$ on each example $(\mathbf{x}^i, \mathbf{y}^i)$

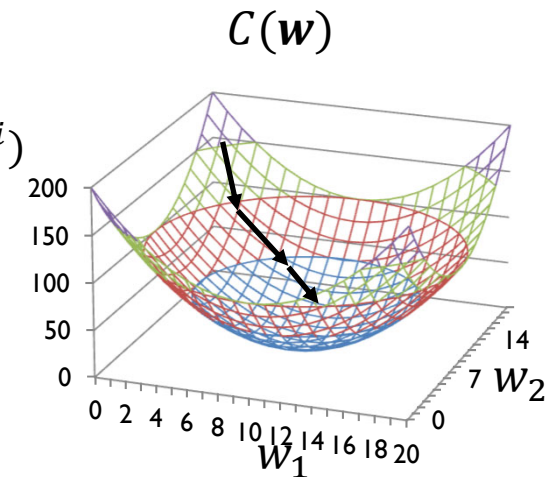
► Principle

- Initialize $\mathbf{w} = \mathbf{w}(0)$
- Iterate until convergence
 - $\mathbf{w}(t+1) = \mathbf{w}(t) + \epsilon(t)\Delta_{\mathbf{w}}(t)$

- $\Delta_{\mathbf{w}}(t)$ is the descent direction, $\epsilon(t)$ is the gradient step

► Both are determined via local information computed from $C(\mathbf{w})$, using approximations of the 1st or 2nd order of $C(\mathbf{w})$

- e.g. steepest descent, is a 1st order gradient with : $\Delta_{\mathbf{w}}(t) = -\nabla_{\mathbf{w}}C(\mathbf{t})$, $\epsilon(t) = \epsilon$



Optimization

Batch second order gradients

- ▶ Consider a quadratic approximation of the loss function

- ▶ C is approximated via a parabola

- $C(w) = C(w(t)) + (w - w(t))^T \nabla C(w(t)) + \frac{1}{2}(w - w(t))^T H(w - w(t))$

- where $w(t)$ is the parameter vector at time t

- H is the Hessian of $C(.) : H_{ij} = \frac{\partial^2 C}{\partial w_i \partial w_j}$

- ▶ Differentiating w.r.t. w

- $\nabla C(w) = \nabla C(w(t)) + H(w - w(t))$

- ▶ The minimum of C is obtained for

- $\nabla C(w) = 0$

- ▶ Several iterative methods could be used

- ▶ E.g. Newton

- $w(t + 1) = w(t) - H^{-1} \nabla C(w(t))$

- Complexity $O(n^3)$ for the inverse + partial derivatives

- In practice one makes use of quasi-Newton methods : H^{-1} is approximated iteratively

Optimization

Stochastic Gradient algorithms

► Objectives

- Training NNs involves finding the parameters w by optimizing a loss

► Difficulties

- Deep NN have a large number of parameters and meta-parameters, the loss is most often a non linear function of these parameters: the optimization problem is non convex
- Optimization for Deep NN is often difficult:
 - Multiple local minima with high loss, might not be a problem in high dimensional spaces
 - Flat regions: plateaus $\rightarrow$ 0 gradients, saddle points $\rightarrow$ pb for 2nd order methods
 - Sharp regions: gradients may explode
 - Deep architectures: large number of gradient multiplications may often cause gradient vanishing or gradient exploding

► Solutions

- There is no unique answer to all these challenges
- The most common family of optimization methods for Deep NN is based on **stochastic gradient algorithms**
 - **Exploit the redundancy in the data, at the cost of high variance in gradient estimates**
- Deep Learning has developed several heuristic training methods
- They are provided in the different toolboxes (Pytorch etc)
- Some examples follow

Optimization

Stochastic gradient algorithms (From Ruder 2016)

► Data + Loss

► Training Data Set

- $D = \{(\mathbf{x}^1, \mathbf{y}^1), \dots, (\mathbf{x}^N, \mathbf{y}^N)\}$

► Loss function

- $C(\mathbf{w}) = \sum_{i=1}^N c_{\mathbf{w}}(\mathbf{x}^i, \mathbf{y}^i)$

► All the algorithms are given in vector form

► Basic Stochastic Gradient Descent

► Initialise $\mathbf{w}(0)$

► Iterate until stop criterion

► sample un exemple $(\mathbf{x}(t), \mathbf{y}(t))$

- $\mathbf{w}(t + 1) = \mathbf{w}(t) - \epsilon \nabla_{\mathbf{w}} c(\mathbf{x}(t), \mathbf{y}(t))$

► Rq: might produce a lot of oscillations

► Momentum

► Dampens oscillations

- $\mathbf{m}(t) = \gamma \mathbf{m}(t - 1) + \epsilon \nabla_{\mathbf{w}} c(\mathbf{x}(t), \mathbf{y}(t))$

- $\mathbf{w}(t + 1) = \mathbf{w}(t) - \mathbf{m}(t)$



(a) SGD without momentum



(b) SGD with momentum

Figures from (Ruder 2016)

Optimization

SGD algorithms with Adaptive learning rate

► Adagrad

- One learning rate for each parameter w_i at each time step t

► Iteration t

- Compute gradient $\mathbf{g}(t) = \nabla_{\mathbf{w}} c(\mathbf{x}(t), \mathbf{y}(t))$ Vector
- Accumulate squared gradients for each component $r_i(t) = r_i(t-1) + (g_i(t))^2$ Scalar
 - kind of gradient variance
 - Sum of the squared gradients up to step t

► Componentwise:

- $w_i(t+1) = w_i(t) - \frac{\epsilon}{\sqrt{r_i(t) + \epsilon'}} \nabla_{w_i} c(\mathbf{x}(t), \mathbf{y}(t))$ Scalar

► In vector form

- $\mathbf{w}(t+1) = \mathbf{w}(t) - \frac{\epsilon}{\sqrt{\mathbf{r}(t) + \epsilon'}} \odot \nabla_{\mathbf{w}} c(\mathbf{x}(t), \mathbf{y}(t))$ Vector
- $\odot$ elementwise multiplication, $\epsilon' (\approx 10^{-8})$ avoids dividing by 0, $\frac{\epsilon}{\sqrt{\mathbf{r}(t) + \epsilon'}}$ is a vector with components $\frac{\epsilon}{\sqrt{r_i(t) + \epsilon'}}$

- Default : learning rate shrinks too fast

► RMS prop

- Replace $r(t)$ in Adagrad by an exponentially decaying average of past gradients

- $\mathbf{r}(t) = \gamma \mathbf{r}(t-1) + (1-\gamma) \mathbf{g}(t) \odot \mathbf{g}(t), \quad 0 < \gamma < 1$
- $\mathbf{w}(t+1) = \mathbf{w}(t) - \frac{\epsilon}{\sqrt{\mathbf{r}(t) + \epsilon'}} \odot \nabla_{\mathbf{w}} c(\mathbf{x}(t), \mathbf{y}(t))$ Vector

Optimization

SGD algorithm with momentum and Adaptive learning rate

▶ Adam (adaptive moment estimation)

▶ Computes

- ▶ Adaptive learning rates for each parameter
- ▶ An exponentially decaying average of past gradients (momentum)
- ▶ An exponentially decaying average of past squared gradients (like RMSprop)

▶ Iteration t

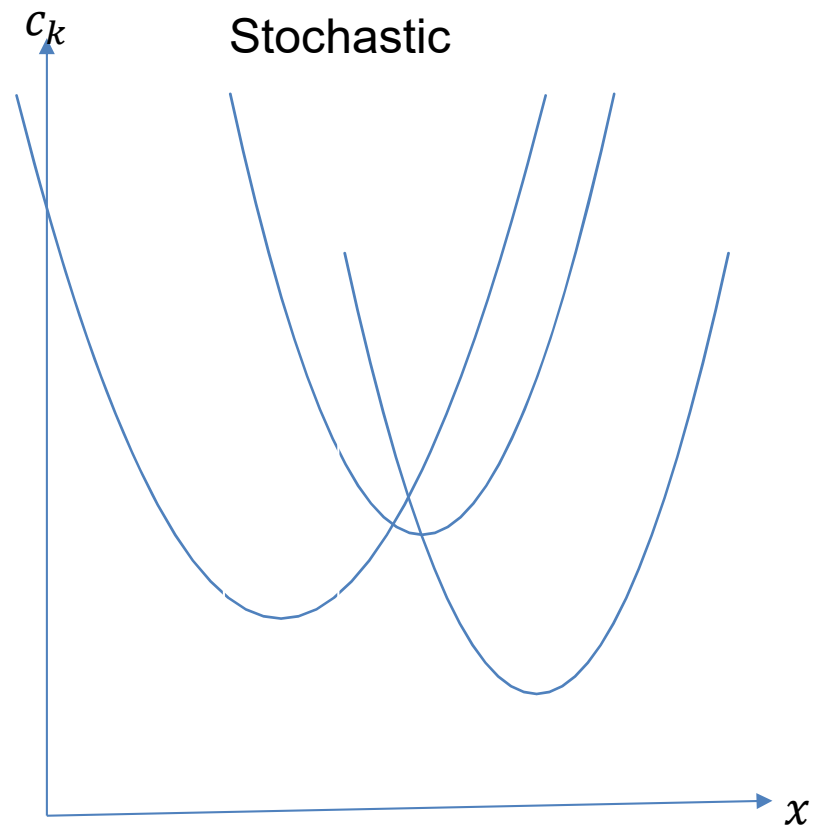
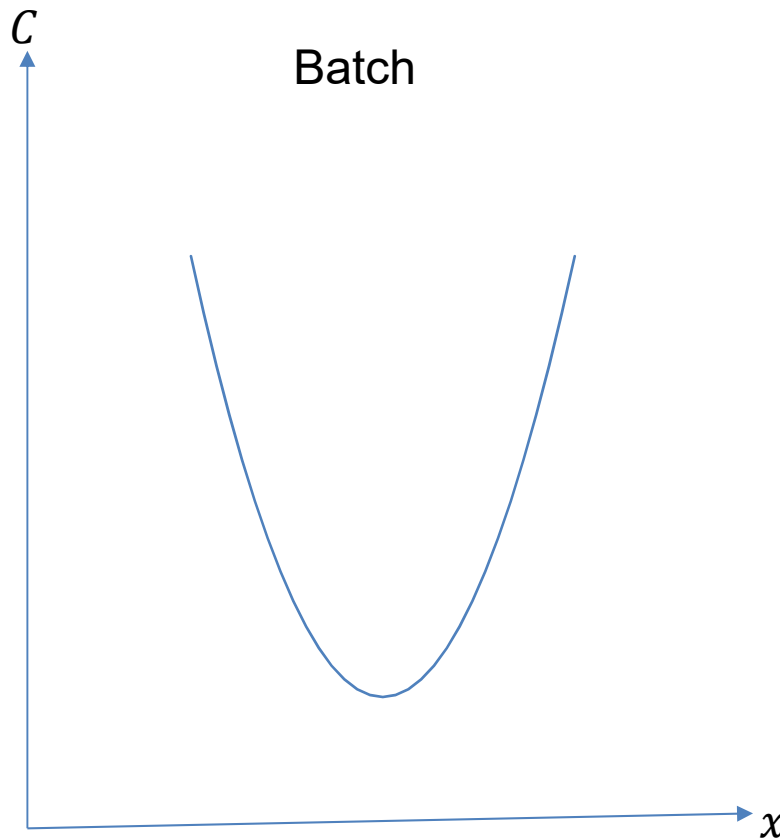
- ▶ Momentum term : $\mathbf{m}(t) = \gamma_1 \mathbf{m}(t-1) + \epsilon(1 - \gamma_1) \mathbf{g}(t)$
- ▶ Gradient variance term: $\mathbf{r}(t) = \gamma_2 \mathbf{r}(t-1) + \epsilon(1 - \gamma_2) \mathbf{g}(t) \odot \mathbf{g}(t)$
- ▶ $\mathbf{w}(t+1) = \mathbf{w}(t) - \frac{\epsilon}{\sqrt{\mathbf{r}(t) + \epsilon'}} \odot \mathbf{m}(t)$
- ▶ Bias correction
 - The 2 moments are initialized at 0, they tend to be biased towards 0, the following correction terms reduce this effect
 - Correct bias of $\mathbf{m}$: $\mathbf{m}(t) = \frac{\mathbf{m}(t)}{1 - \gamma_1^t}$
 - Correct bias of $\mathbf{r}$: $\mathbf{r}(t) = \frac{\mathbf{r}(t)}{1 - \gamma_2^t}$

Batch vs stochastic gradient



$$C = \frac{1}{N} \sum_k c_k$$

C : global loss
 c_k : individual (pattern k) loss



Gradient methods as numerical integration of ordinary differential equations (ODE)

- ▶ Let $l: R^d \rightarrow R$ a function we seek to minimize
 - ▶ We make the assumption that l is « well behaved »
- ▶ Consider the following gradient flow equation
 - $$\frac{dw(t)}{dt} = -\nabla l(w(t))$$
 - $$w(0) = w_0$$
- ▶ Taylor expansion around $w(t)$ is:
 - $$w(t+h) = w(t) + h \frac{dw(t)}{dt} + O(h^2)$$
- ▶ Lets take $t = kh$, by neglecting the second order terms, we get the explicit Euler method for integrating ODEs
 - $$w_{k+1} = w_k - h \nabla l(w(t))$$
 - ▶ Which is the steepest descent algorithm
- ▶ Message
 - ▶ This interpretation of Gradient Descent as a numerical integration method for the gradient flow equation allows us to use the results from numerical analysis to characterize useful properties e. g. stability / consistence of the method
 - ▶ This is used for analyzing more sophisticated GD algorithms

Optimization Summary

- ▶ Which method to use?
 - ▶ No « one solution for all problems »
 - ▶ For large scale applications, Adam is often used today as a default choice together with minibatches
 - ▶ But... simple SGD with heuristic learning rate decay can sometimes be competitive ...
- ▶ Batch, mini batch, pure SGD
 - ▶ Stochastic methods exploit data redundancy
 - ▶ Mini batch well suited for GPU
 - ▶

Regression and Logistic Regression

Regression

▶ Linear regression

▶ Objective : predict real values

▶ Training set

▶ $(\mathbf{x}^1, y^1), \dots, (\mathbf{x}^N, y^N)$

▶ $\mathbf{x} \in \mathbb{R}^n, y \in \mathbb{R}$: single output regression

▶ Linear model

▶ $F(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} = \sum_{i=0}^n w_i x_i$ with $x_0 = 1$

▶ Loss function

▶ Mean square error

□ $C = \frac{1}{2} \sum_{i=1}^N (y^i - \mathbf{w} \cdot \mathbf{x}^i)^2$

▶ Steepest descent gradient (batch)

▶ $\mathbf{w} = \mathbf{w}(t) - \epsilon \nabla_{\mathbf{w}} C, \nabla_{\mathbf{w}} C = \left(\frac{\partial C}{\partial w_1}, \dots, \frac{\partial C}{\partial w_n} \right)^T$

▶ $\frac{\partial C}{\partial w_k} = \frac{1}{2} \sum_{i=1}^N \frac{\partial}{\partial w_k} (y^i - \mathbf{w} \cdot \mathbf{x}^i)^2 = - \sum_{i=1}^N (y^i - \mathbf{w} \cdot \mathbf{x}^i) x_k^i$

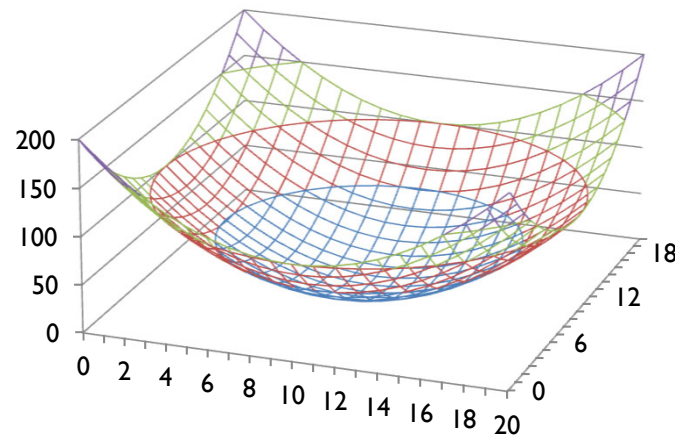
for component w_k

▶ $\mathbf{w} = \mathbf{w}(t) + \epsilon \sum_{i=1}^N (y^i - \mathbf{w} \cdot \mathbf{x}^i) \mathbf{x}^i$

in vector form

Regression

- ▶ Geometry of mean squares



- ▶ Regression with multiple outputs $\mathbf{y} \in \mathbb{R}^p$
 - ▶ Simple extension: p independent linear regressions

Probabilistic Interpretation

- ▶ Statistical model of linear regression
 - ▶ $y = \mathbf{w} \cdot \mathbf{x} + \epsilon$, where ϵ is a random variable (error term)
 - ▶ Hypothesis ϵ is i.i.d. Gaussian
 - ▶ $\epsilon \sim N(0, \sigma^2)$, $p(\epsilon) = \frac{1}{\sqrt{2\pi}\sigma} \exp(-\frac{\epsilon^2}{2\sigma^2})$
 - ▶ The posterior distribution of y is then
 - ▶ $p(y | \mathbf{x}; \mathbf{w}) = \frac{1}{\sqrt{2\pi}\sigma} \exp(-\frac{(y - \mathbf{w} \cdot \mathbf{x})^2}{2\sigma^2})$
 - ▶ Likelihood
 - ▶ $L(\mathbf{w}) = \prod_{i=1}^N p(y^i | \mathbf{x}^i; \mathbf{w})$
 - Likelihood is a function of $\mathbf{w}$, it is computed on the training set
 - ▶ Maximum likelihood principle
 - ▶ Choose the parameters $\mathbf{w}$ maximizing $L(\mathbf{w})$ or any increasing function of $L(\mathbf{w})$
 - ▶ In practice, one optimizes the log likelihood $l(\mathbf{w}) = \log L(\mathbf{w})$
 - ▶ $l(\mathbf{w}) = N \log \left(\frac{1}{\sqrt{2\pi}\sigma} \right) - \frac{1}{2\sigma^2} \sum_{i=1}^N (y^i - \mathbf{w} \cdot \mathbf{x}^i)^2$
 - ▶ This is the MSE criterion
- ▶ This provides a probabilistic interpretation of regression
 - ▶ Under a gaussian hypothesis max likelihood is equivalent to MSE minimization

Logistic regression – 2 classes

- ▶ Linear regression can be used (in practice) for regression or classification
- ▶ For classification a proper model is logistic regression

- ▶ $F_w(x) = \sigma(w \cdot x) = \frac{1}{1 + \exp(-w \cdot x)}$

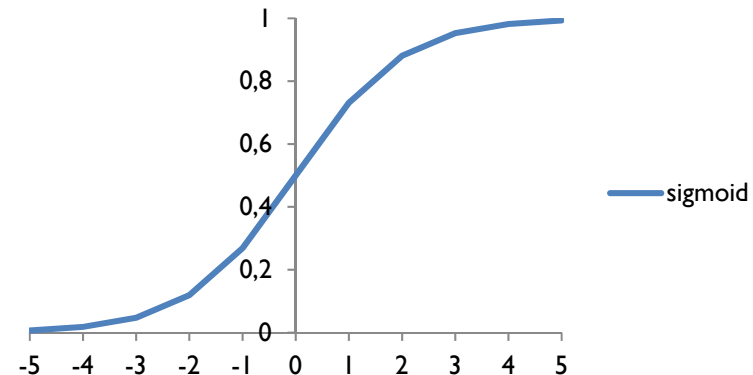
- ▶ Logistic (or sigmoid) function

- ▶ $\sigma(z) = \frac{1}{1 + \exp(-z)}$

- hint

- $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

- ▶ Hyp: $y \in \{0,1\}$



Optimization

Batch second order gradients

- ▶ Consider a quadratic approximation of the loss function
 - ▶ C is approximated via a parabola (i.e. a 2^{nd} order (quadratic) approximation of the loss function $C(w)$)
 - $C(w) = C(w(t)) + (w - w(t))^T \nabla C(w(t)) + \frac{1}{2} (w - w(t))^T H (w - w(t))$
 - where $w(t)$ is the parameter vector at time t
 - H is the Hessian of $C(.)$: $H_{ij} = \frac{\partial^2 C}{\partial w_i \partial w_j}$
 - ▶ Differentiating w.r.t. w
 - $\nabla C(w) = \nabla C(w(t)) + H(w - w(t))$
 - ▶ The minimum of C is obtained for
 - $\nabla C(w) = 0$
 - ▶ Several iterative methods could be used
 - ▶ E.g. Newton
 - $w(t + 1) = w(t) - H^{-1} \nabla C(w(t))$
 - Complexity $O(n^3)$ for the inverse of the Hessian + partial derivatives
 - In practice one makes use of quasi-Newton methods : H^{-1} is approximated iteratively

Optimization

Stochastic Gradient algorithms

► Objectives

- Training NNs involves finding the parameters w by optimizing a loss

► Difficulties

- Deep NN have a large number of parameters and meta-parameters, the loss is most often a non linear function of these parameters: the optimization problem is non convex
- Optimization for Deep NN is often difficult:
 - Multiple local minima with high loss, might not be a problem in high dimensional spaces
 - Flat regions: plateaus with 0 gradients, saddle points are a pb for 2nd order methods
 - Sharp regions: gradients may explode
 - Deep architectures: large number of gradient multiplications often cause gradient vanishing or gradient exploding

► Solutions

- There is no unique answer to all these challenges
- The most common family of optimization methods for Deep NN is based on **stochastic gradient algorithms** (actually mini-batch stochastic gradients).
 - **Exploit the redundancy in the data, at the cost of high variance in gradient estimates**
- Deep Learning has developed several heuristic training methods
- They are provided in the different toolboxes (Pytorch etc)
- Some examples follow

Optimization

Stochastic gradient algorithms (From Ruder 2016)

- ▶ Data + Loss
 - ▶ Training Data Set
 - ▶ $D = \{(\mathbf{x}^1, \mathbf{y}^1), \dots, (\mathbf{x}^N, \mathbf{y}^N)\}$
 - ▶ Loss function
 - ▶ $C(\mathbf{w}) = \sum_{i=1}^N c_{\mathbf{w}}(\mathbf{x}^i, \mathbf{y}^i)$
 - ▶ All the algorithms are given in **vector form**
- ▶ Basic Stochastic Gradient Descent
 - ▶ Initialise $\mathbf{w}(0)$
 - ▶ Iterate until stop criterion
 - ▶ sample un exemple $(\mathbf{x}(t), \mathbf{y}(t))$
 - ▶ $\mathbf{w}(t + 1) = \mathbf{w}(t) - \epsilon \nabla_{\mathbf{w}} c(\mathbf{x}(t), \mathbf{y}(t))$
 - ▶ Rq: might produce a lot of oscillations
- ▶ Momentum
 - ▶ Dampens oscillations
 - ▶ $\mathbf{m}(t) = \gamma \mathbf{m}(t - 1) + \epsilon \nabla_{\mathbf{w}} c(\mathbf{x}(t), \mathbf{y}(t))$
 - ▶ $\mathbf{w}(t + 1) = \mathbf{w}(t) - \mathbf{m}(t)$



(a) SGD without momentum



(b) SGD with momentum

Figures from (Ruder 2016)

Optimization

SGD algorithms with Adaptive learning rate

► Adagrad

- One learning rate for each parameter w_i at each time step t

► Iteration t

- Compute gradient $\mathbf{g}(t) = \nabla_{\mathbf{w}} c(\mathbf{x}(t), \mathbf{y}(t))$ Vector
- Accumulate squared gradients for each component $r_i(t) = r_i(t-1) + (g_i(t))^2$ Scalar
 - kind of gradient variance
 - Sum of the squared gradients up to step t
- Componentwise:
- $w_i(t+1) = w_i(t) - \frac{\epsilon}{\sqrt{r_i(t) + \epsilon'}} \nabla_{w_i} c(\mathbf{x}(t), \mathbf{y}(t))$ Scalar

► In vector form

- $\mathbf{w}(t+1) = \mathbf{w}(t) - \frac{\epsilon}{\sqrt{\mathbf{r}(t) + \epsilon'}} \odot \nabla_{\mathbf{w}} c(\mathbf{x}(t), \mathbf{y}(t))$ Vector
- $\odot$ elementwise multiplication, $\epsilon' (\approx 10^{-8})$ avoids dividing by 0, $\frac{\epsilon}{\sqrt{\mathbf{r}(t) + \epsilon'}}$ is a vector with components $\frac{\epsilon}{\sqrt{r_i(t) + \epsilon'}}$

- Default : learning rate shrinks too fast

► RMS prop

- Replace $r(t)$ in Adagrad by an exponentially decaying average of past gradients

- $\mathbf{r}(t) = \gamma \mathbf{r}(t-1) + (1-\gamma) \mathbf{g}(t) \odot \mathbf{g}(t), \quad 0 < \gamma < 1$
- $\mathbf{w}(t+1) = \mathbf{w}(t) - \frac{\epsilon}{\sqrt{\mathbf{r}(t) + \epsilon'}} \odot \nabla_{\mathbf{w}} c(\mathbf{x}(t), \mathbf{y}(t))$ Vector

Optimization

SGD algorithm with momentum and Adaptive learning rate

▶ Adam (adaptive moment estimation)

▶ Computes

- ▶ Adaptive learning rates for each parameter
- ▶ An exponentially decaying average of past gradients (momentum)
- ▶ An exponentially decaying average of past squared gradients (like RMSprop)

▶ Iteration t

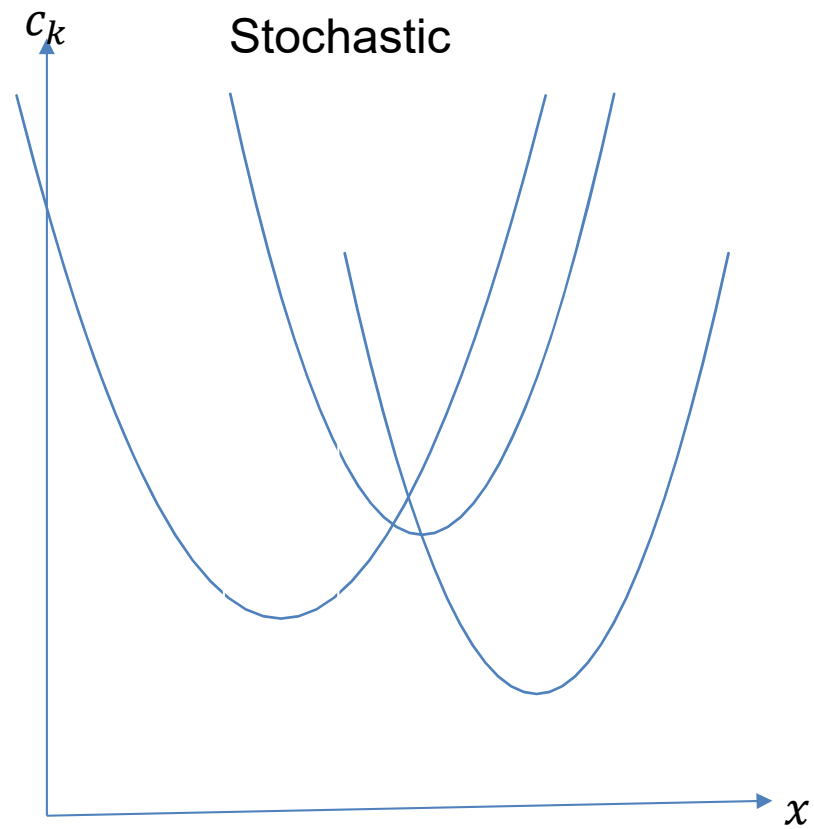
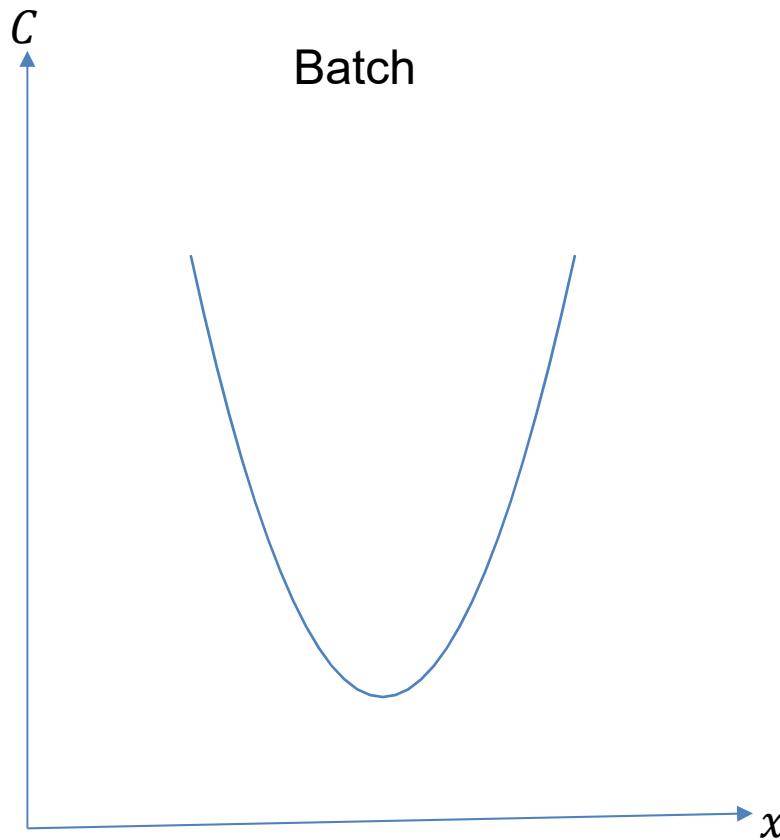
- ▶ Momentum term : $\mathbf{m}(t) = \gamma_1 \mathbf{m}(t-1) + \epsilon(1 - \gamma_1) \mathbf{g}(t)$
- ▶ Gradient variance term: $\mathbf{r}(t) = \gamma_2 \mathbf{r}(t-1) + \epsilon(1 - \gamma_2) \mathbf{g}(t) \odot \mathbf{g}(t)$
- ▶ $\mathbf{w}(t+1) = \mathbf{w}(t) - \frac{\epsilon}{\sqrt{\mathbf{r}(t) + \epsilon'}} \odot \mathbf{m}(t)$
- ▶ Bias correction
 - The 2 moments are initialized at 0, they tend to be biased towards 0, the following correction terms reduce this effect
 - Correct bias of $\mathbf{m}$: $\mathbf{m}(t) = \frac{\mathbf{m}(t)}{1 - \gamma_1^t}$
 - Correct bias of $\mathbf{r}$: $\mathbf{r}(t) = \frac{\mathbf{r}(t)}{1 - \gamma_2^t}$

Batch vs stochastic gradient



$$C = \frac{1}{N} \sum_k c_k$$

C : global loss
 c_k : individual (pattern k) loss



Gradient methods as numerical integration of ordinary differential equations (ODE)

- ▶ Let $l: R^d \rightarrow R$ a function we seek to minimize
 - ▶ We make the assumption that l is « well behaved »
- ▶ Consider the following gradient flow equation
 - $\frac{dw(t)}{dt} = -\nabla l(w(t))$
 $w(0) = w_0$ (t is a continuous variable here)
- ▶ Taylor expansion around $w(t)$ is:
 - ▶ $w(t+h) = w(t) + h \frac{dw(t)}{dt} + O(h^2)$
- ▶ Lets take $t = kh$, by neglecting the second order terms, we get the explicit Euler method for integrating ODEs
 - ▶ $w_{k+1} = w_k - h \nabla l(w_k)$
 - ▶ Which is the steepest descent algorithm
- ▶ **Message**
 - ▶ This interpretation of Gradient Descent as a discretized numerical approximation method for the gradient flow equation allows us to use the results from numerical analysis to characterize useful properties e.g. stability / consistence of the method
 - ▶ This is used for analyzing more sophisticated GD algorithms

Optimization Summary

- ▶ Which method to use?
 - ▶ No « one solution for all problems »
 - ▶ For large scale applications, Adam is often used today as a default choice together with minibatches
 - ▶ But... simple SGD with heuristic learning rate decay can sometimes be competitive ...
- ▶ Batch, mini batch, pure SGD
 - ▶ Stochastic methods exploit data redundancy
 - ▶ Mini batch well suited for GPU
 - ▶ Parallelize on the data, limited by the memory size

Regression and Logistic Regression

Regression

▶ Linear regression

▶ Objective : predict real values

▶ Training set

▶ $(\mathbf{x}^1, y^1), \dots, (\mathbf{x}^N, y^N)$

▶ $\mathbf{x} \in R^n, y \in R$: single output regression

▶ Linear model

▶ $F(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} = \sum_{i=0}^n w_i x_i$ with $x_0 = 1$

▶ Loss function

▶ Mean square error

$$\square C = \frac{1}{2} \sum_{i=1}^N (y^i - \mathbf{w} \cdot \mathbf{x}^i)^2$$

▶ Steepest descent gradient in batch version (just remove the sum for the SGD version)

▶ $\mathbf{w} = \mathbf{w}(t) - \epsilon \nabla_{\mathbf{w}} C, \nabla_{\mathbf{w}} C = (\frac{\partial C}{\partial w_1}, \dots, \frac{\partial C}{\partial w_n})^T$

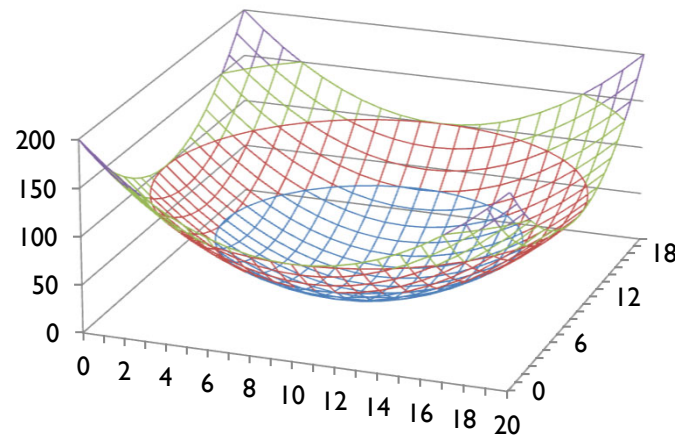
▶ $\frac{\partial C}{\partial w_k} = \frac{1}{2} \sum_{i=1}^N \frac{\partial}{\partial w_k} (y^i - \mathbf{w} \cdot \mathbf{x}^i)^2 = - \sum_{i=1}^N (y^i - \mathbf{w} \cdot \mathbf{x}^i) x_k^i$ for component w_k

▶ **Iterate**

$$\square \mathbf{w}(t+1) = \mathbf{w}(t) + \epsilon \sum_{i=1}^N (y^i - \mathbf{w}(t) \cdot \mathbf{x}^i) \mathbf{x}^i \quad \text{in vector form}$$

Regression

- ▶ Geometry of mean squares



- ▶ Regression with multiple outputs $\mathbf{y} \in \mathbb{R}^p$
 - ▶ Simple extension: p independent linear regressions

Probabilistic Interpretation

- ▶ Statistical model of linear regression
 - ▶ $y = \mathbf{w} \cdot \mathbf{x} + \epsilon$, where ϵ is a random variable (error term)
 - ▶ Hypothesis ϵ is i.i.d. Gaussian
 - ▶ $\epsilon \sim N(0, \sigma^2)$, $p(\epsilon) = \frac{1}{\sqrt{2\pi}\sigma} \exp(-\frac{\epsilon^2}{2\sigma^2})$
 - ▶ The posterior distribution of y is then
 - ▶ $p(y | \mathbf{x}; \mathbf{w}) = \frac{1}{\sqrt{2\pi}\sigma} \exp(-\frac{(y - \mathbf{w} \cdot \mathbf{x})^2}{2\sigma^2})$
 - ▶ Likelihood
 - ▶ $L(\mathbf{w}) = \prod_{i=1}^N p(y^i | \mathbf{x}^i; \mathbf{w})$
 - Likelihood is a function of $\mathbf{w}$, it is computed on the training set
 - ▶ Maximum likelihood principle
 - ▶ Choose the parameters $\mathbf{w}$ maximizing $L(\mathbf{w})$ or any increasing function of $L(\mathbf{w})$
 - ▶ In practice, one optimizes the log likelihood $l(\mathbf{w}) = \log L(\mathbf{w})$
 - ▶ $l(\mathbf{w}) = N \log \left(\frac{1}{\sqrt{2\pi}\sigma} \right) - \frac{1}{2\sigma^2} \sum_{i=1}^N (y^i - \mathbf{w} \cdot \mathbf{x}^i)^2$
 - ▶ This is the MSE criterion
- ▶ This provides a probabilistic interpretation of regression
 - ▶ Under a gaussian hypothesis max likelihood is equivalent to MSE minimization

Logistic regression – 2 classes

- ▶ Linear regression can be used (in practice) for regression or classification
- ▶ For classification a proper model is logistic regression

- ▶ $F_w(x) = \sigma(w \cdot x) = \frac{1}{1 + \exp(-w \cdot x)}$

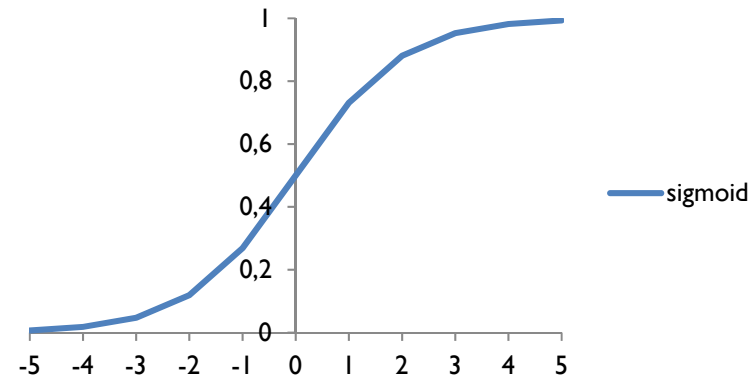
- ▶ Logistic (or sigmoid) function

- ▶ $\sigma(z) = \frac{1}{1 + \exp(-z)}$

- hint

- $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

- ▶ Hyp: $y \in \{0,1\}$



Logistic regression – 2 classes

Probabilistic interpretation

- ▶ Since $y \in \{0,1\}$, we make a Bernoulli hypothesis for the posterior distribution

- ▶ $p(y = 1|x; \mathbf{w}) = F_w(\mathbf{x})$ et $p(y = 0|x; \mathbf{w}) = 1 - F_w(\mathbf{x})$
- ▶ In compact format
 - $p(y|x; \mathbf{w}) = (F_w(\mathbf{x}))^y (1 - F_w(\mathbf{x}))^{1-y}$ with $y \in \{0,1\}$

- ▶ Likelihood

- ▶ $L(\mathbf{w}) = \prod_{i=1}^N (F_w(\mathbf{x}^i))^{y^i} (1 - F_w(\mathbf{x}^i))^{1-y^i}$

- ▶ Log-likelihood

- ▶ $l(\mathbf{w}) = \sum_{i=1}^N y^i \log F_w(\mathbf{x}^i) + (1 - y^i) (\log(1 - F_w(\mathbf{x}^i)))$
 - This is minus the cross-entropy between the target and the estimated posterior distribution

- ▶ Steepest descent algorithm (batch version) for minimizing cross entropy (again remove the sum sign for the SGD version)

- ▶ Componentwise: $\frac{\partial l(\mathbf{w})}{\partial w_k} = \sum_{i=1}^N (y^i - F_w(\mathbf{x}^i)) x_k^i$
- ▶ **Vector** form: $\nabla_{\mathbf{w}} l = \sum_{i=1}^N (y^i - F_w(\mathbf{x}^i)) \mathbf{x}^i$
- ▶ Algorithm: iterate (t)
 - $\mathbf{w}(t+1) = \mathbf{w}(t) - \epsilon \nabla_{\mathbf{w}} C = \mathbf{w}(t) + \epsilon \sum_{i=1}^N (y^i - F_w(\mathbf{x}^i)) \mathbf{x}^i$
 - This is similar to the regression form with $F_w(\mathbf{x}^i)$ in place of the linear function

Multivariate logistic regression: multiclass classification

- ▶ Consider a p class classification problem,
 - ▶ Classes are encoded by “one hot” indicator vectors. Each indicator vector is of dimension p
 - ▶ An item can belong to only one class
 - ▶ Class 1: $\mathbf{y} = (1, 0, \dots, 0)^T$
 - ▶ Class 2 : $\mathbf{y} = (0, 1, \dots, 0)^T$
 - ▶ ...
 - ▶ Class p : $\mathbf{y} = (0, 0, \dots, 1)^T$
- ▶ $F_{\mathbf{W}}(\mathbf{x})$ is a vector valued function with values in R^p
 - ▶ Its component i is a **softmax function** (generalizes the sigmoid)
 - ▶ $\hat{\mathbf{y}}_i = F_{\mathbf{W}}(\mathbf{x})_i = \frac{\exp(\mathbf{w}_i \cdot \mathbf{x})}{\sum_{j=1}^p \exp(\mathbf{w}_j \cdot \mathbf{x})}$
 - Note : here $\mathbf{w}_j \in R^n$ is a vector, $\hat{\mathbf{y}}_i \in R$ is the i^{th} component of $\hat{\mathbf{y}}$
- ▶ The probabilistic model for the posterior is a multinoulli (also known as categorical) distribution
 - ▶ $p(\text{Class} = i | \mathbf{x}; \mathbf{w}) = \frac{\exp(\mathbf{w}_i \cdot \mathbf{x})}{\sum_{j=1}^p \exp(\mathbf{w}_j \cdot \mathbf{x})} = \text{softmax}(\mathbf{w}_i \cdot \mathbf{x})$

Multivariate logistic regression

▶ Notations

- ▶ $\mathbf{s}^i = W\mathbf{x}^i$ is the **logit** for input $\mathbf{x}^i$
 - ▶ $W = (\mathbf{w}_1, \dots, \mathbf{w}_p)^T$ is a $p \times n$ matrix of weights
 - ▶ $\mathbf{s}^i = (s_1^i, \dots, s_p^i)^T \in \mathbb{R}^p$
- ▶ $\hat{\mathbf{y}}^i = \text{softmax}(\mathbf{s}^i)$ is the output for input $\mathbf{x}^i$ (here σ applies component-wise, i.e. $\hat{y}_j^i = \text{softmax}(s_j^i)$)
 - ▶ $\hat{\mathbf{y}}^i = (\hat{y}_1^i, \dots, \hat{y}_p^i)^T \in \mathbb{R}^p$

▶ Let $\hat{\mathbf{y}}$ be a computed output for input $\mathbf{x}$ (we drop the index i for simplicity), then

- ▶ $\frac{\partial \hat{y}_j}{\partial s_i} = \hat{y}_j(I_{ji} - \hat{y}_i)$ with I_{ji} elements of the identity matrix (1)

▶ Likelihood

- ▶ $L(W) = p(Y|X; W) = \prod_{i=1}^N \prod_{j=1}^p (\hat{y}_j^i)^{y_j^i}$, X and Y are the column wise matrices of input and output vector

Multivariate logistic regression

- ▶ Log likelihood

- ▶ $l(W) = \sum_{i=1}^N \sum_{j=1}^p y_j^i \ln \hat{y}_j^i$ again this is minus the cross entropy for the multiclass classification problem

- ▶ Gradient of the log likelihood

- ▶ $\nabla_{w_k} l(W) = - \sum_{i=1}^N (\hat{y}_k^i - y_k^i) x^i$ by using identity (1)

- ▶ Training algorithm

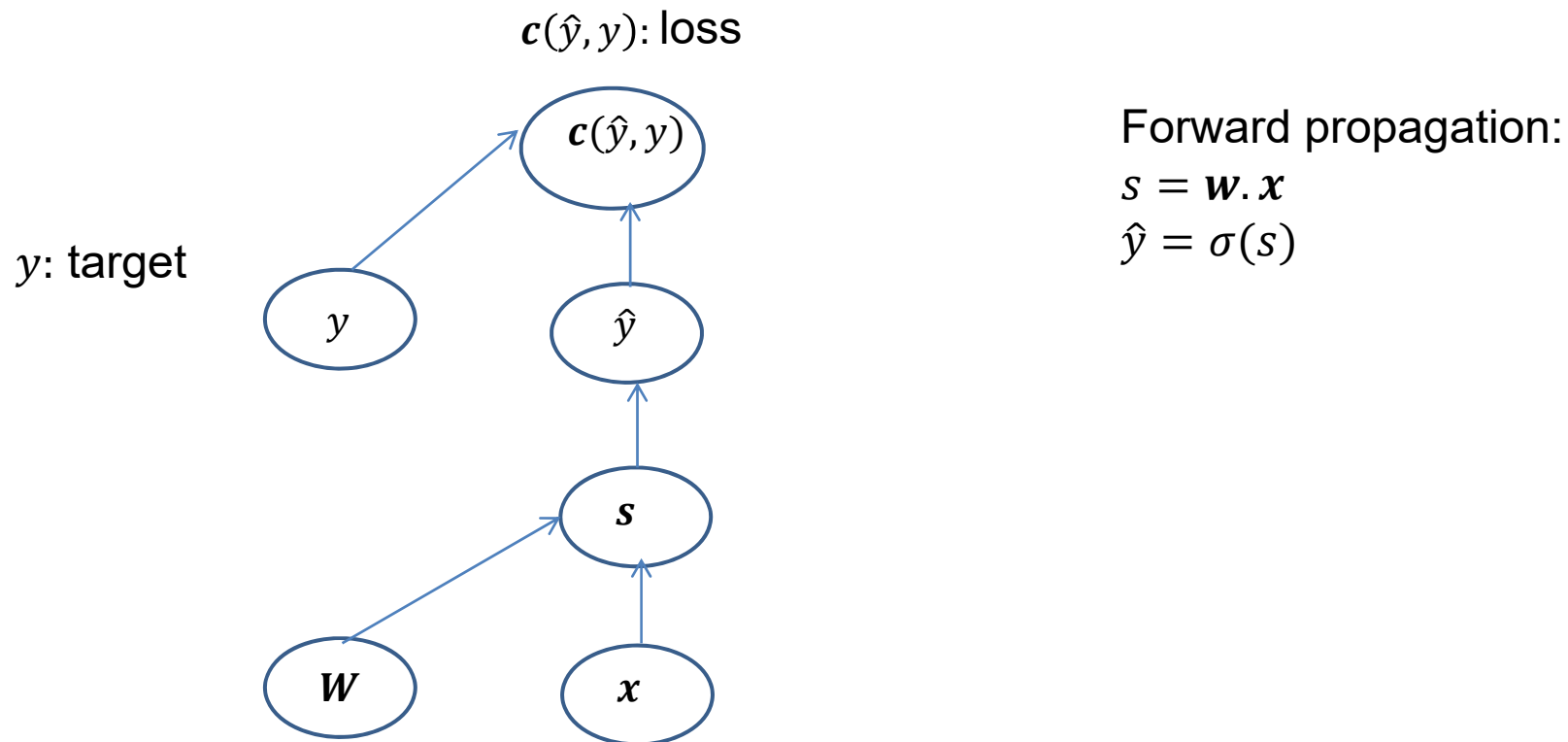
- ▶ As before, one may use a gradient method for maximizing the log likelihood.
- ▶ When the number of classes is large, computing the soft max is prohibitive, alternatives are required

Probabilistic interpretation for non linear models

- ▶ These results extend to non linear models, e.g. when $F_w(x)$ is a NN
- ▶ Non linear regression
 - ▶ Max likelihood is equivalent to MSE loss optimization under the Gaussian hyp.
 - ▶ $y = F_w(x) + \epsilon, \epsilon \sim N(0, \sigma^2)$
 - ▶ $p(y | x; w) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y - F(x))^2}{2\sigma^2}\right)$
 - ▶ log – likelihood $l(w)$
 - ▶ $l(w) = N \log\left(\frac{1}{\sqrt{2\pi}\sigma}\right) - \frac{1}{2\sigma^2} \sum_{i=1}^N (y^i - F(x^i))^2$
- ▶ Classification
 - ▶ Max likelihood is equivalent to cross entropy maximization under Bernoulli/multinomial distribution
 - 2 classes: if y is binary and we make the hypothesis that it is conditionally Bernoulli with probability $F(x) = p(y = 1|x)$ we get the cross entropy loss
 - More than 2 classes: same as logistic regression with multiple outputs

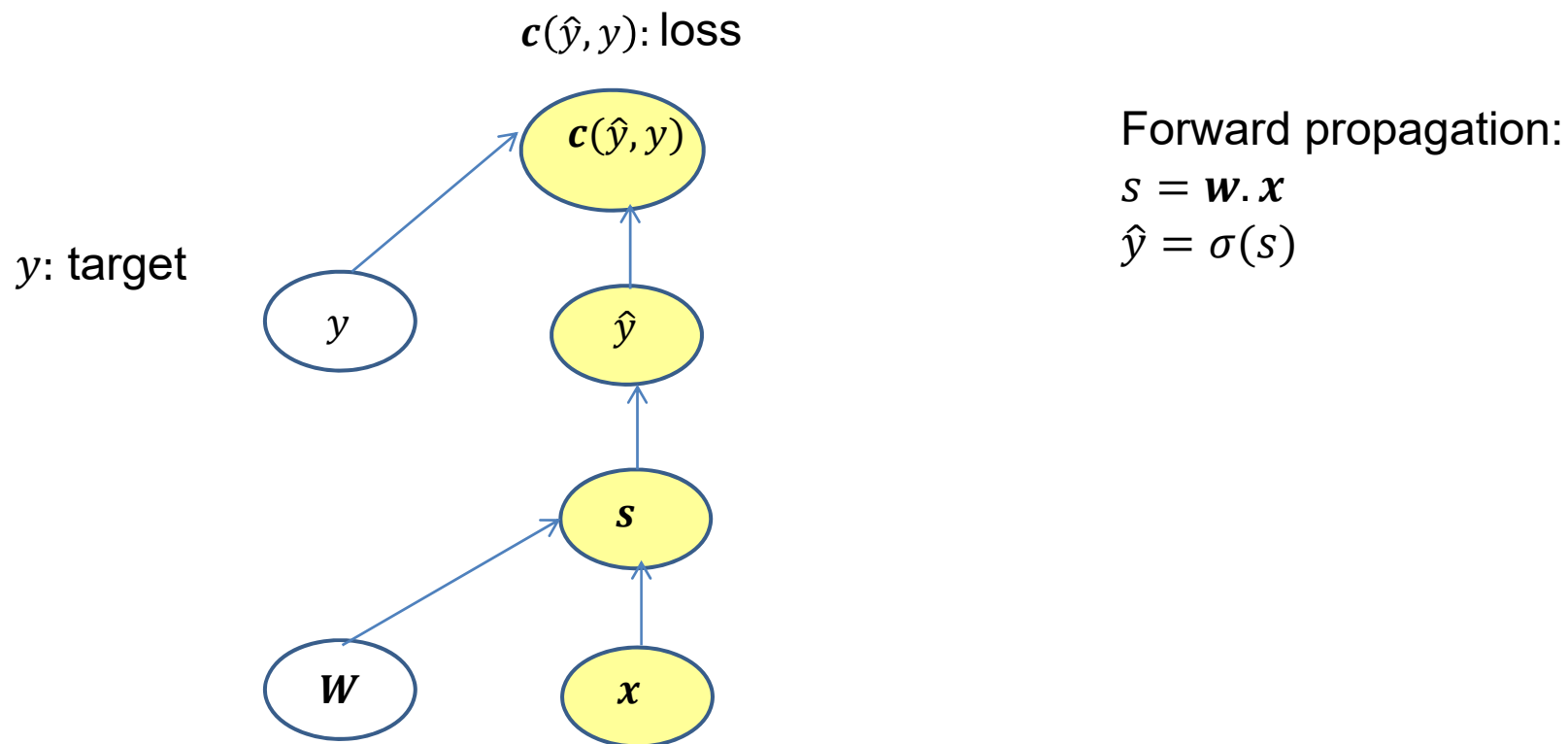
Logistic regression – Computational graph -SGD

► Forward pass



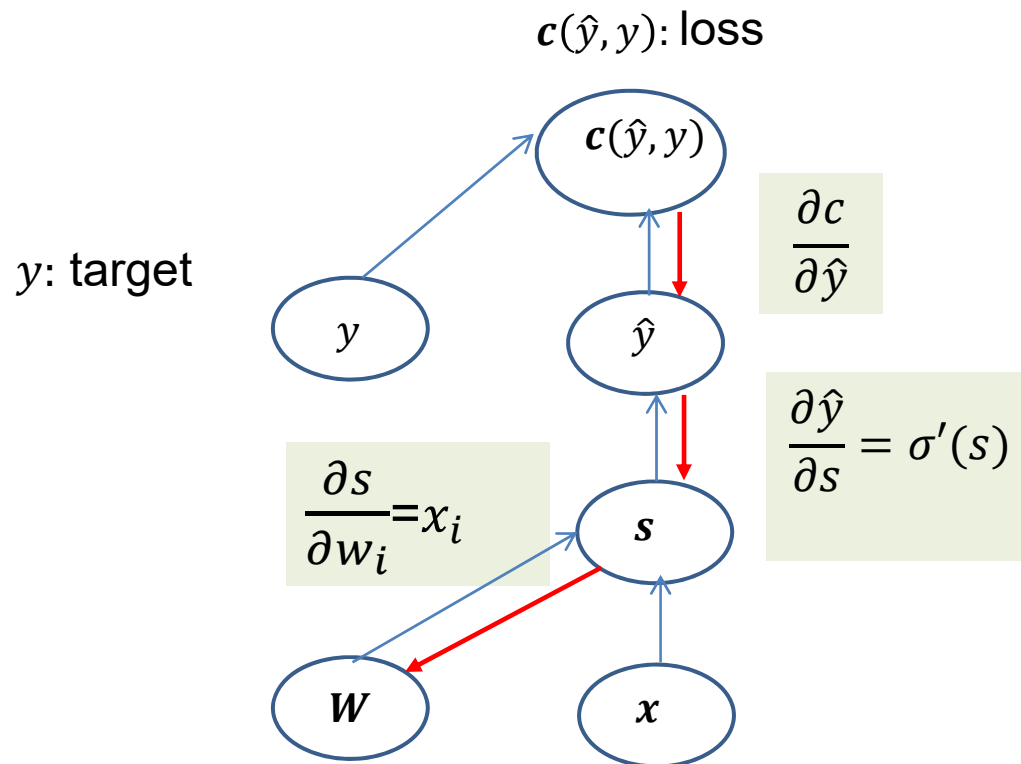
Logistic regression – Computational graph - SGD

► Forward pass



Logistic regression – Computational graph - SGD

► Backward pass



Backward propagation:

$$\frac{\partial c}{\partial s} = \frac{\partial c}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial s}$$
$$\frac{\partial c}{\partial w_i} = \frac{\partial c}{\partial s} \frac{\partial s}{\partial w_i}$$

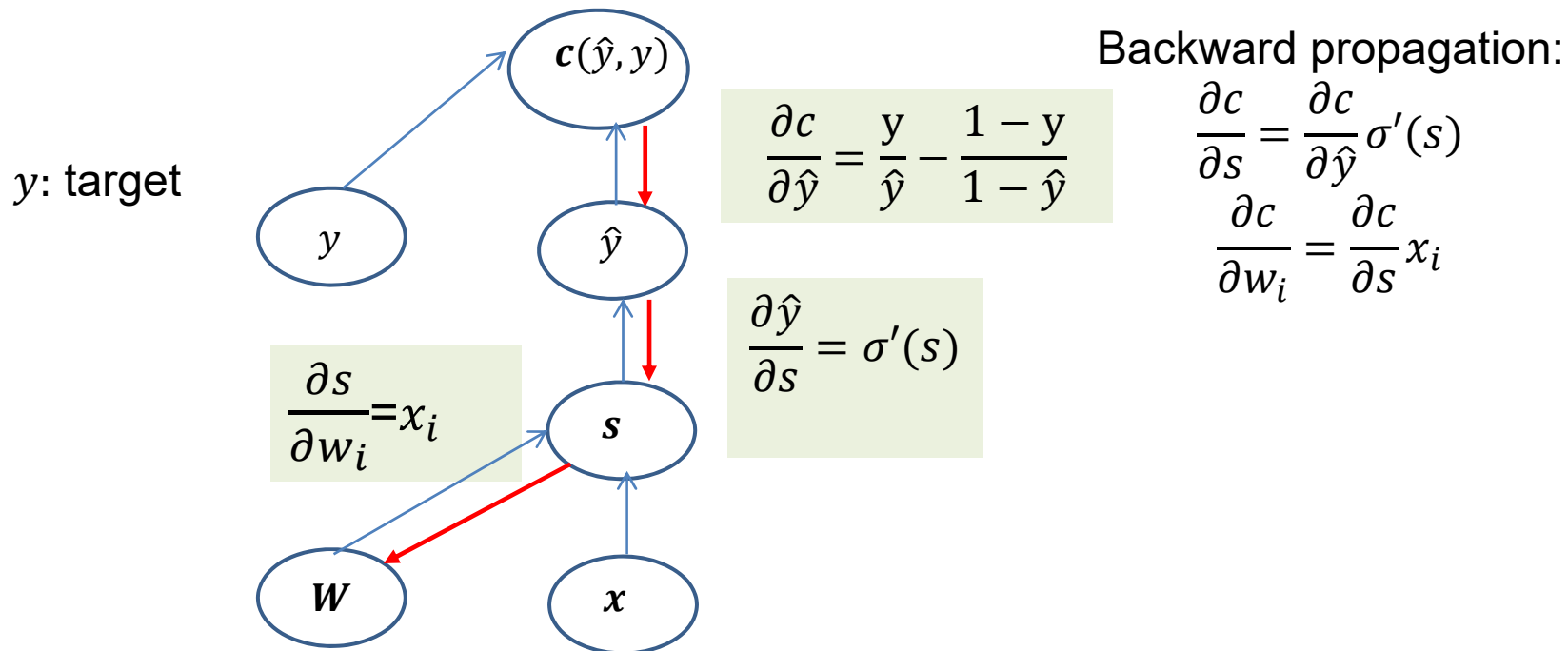
$$\frac{\partial c}{\partial w_i} = \frac{\partial c}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial s} \frac{\partial s}{\partial w_i}$$

Chain Rule

Logistic regression – Computational graph - SGD

- **Backward pass** For the cross entropy loss
 $l(\mathbf{w}) = \sum_{i=1}^N y^i \log \hat{y}^i + (1 - y^i) \log(1 - \hat{y}^i) = \sum_{i=1}^N c(\hat{y}^i, y^i)$

$c(\hat{y}, y)$: loss



$$\frac{\partial c}{\partial w_i} = \left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}} \right) \sigma'(s) x_i$$

Probabilistic interpretation of NN outputs

Mean Square loss

- ▶ Derived here for multivariate regression (1 output), trivial extension to multiple outputs
- ▶ Holds for any continuous functional (regression, logistic regression, NNs, etc)
- ▶ Risk $R = E_{x,y} [(y - h(x))^2]$
- ▶ The minimum of R , $\text{Min}_h R$, is obtained for $h^*(x) = E_y[y|x]$
- ▶ The risk R pour the family of functions $F_w(x)$ decomposes as follows:
 - ▶ $R = E_{x,y} [(y - F_w(x))^2]$
 - ▶ $R = E_{x,y} [(y - E_y[y|x])^2] + E_{x,y} [(E_y[y|x] - F_w(x))^2]$
- ▶ Let us consider $E_y [(y - E_y[y|x])^2]$
 - ▶ This term is independent of the model $F_w(\cdot)$ and only depends on the problem characteristics (the data distribution).
 - ▶ It represents the min error that could be obtained for this data distribution
 - ▶ $h^*(x) = E_y[y|x]$ est the optimal solution to $\text{Min}_h R$
- ▶ Minimizing $E_{x,y} [(y - F_w(x))^2]$ is equivalent to minimizing $E_{x,y} [(E_y[y|x] - F_w(x))^2]$
 - ▶ The optimal solution $F_{w*}(x) = \text{argmin}_w E_{x,y} [(E_y[y|x] - F_w(x))^2]$ is the best mean square approximation of $E[y|x]$

Probabilistic interpretation of NN outputs

► Classification

- Let us consider multi-class classification with one hot encoding of the target outputs
 - i.e. $\mathbf{y} = (0, \dots, 0, 1, 0, \dots, 0)^T$ with a 1 at position i if the target is class i and zero everywhere else
 - $h_i^* = E_y[y|x] = 1 * P(C_i|x) + 0 * (1 - P(C_i|x)) = P(C_i|x)$
 - i.e. $F_{w^*}()$ is the best LMS approximation of the Bayes discriminant function (which is the optimal solution for classification with 0/1 loss)
- More generally with binary targets
 - $h_i^* = P(y_i = 1|x)$

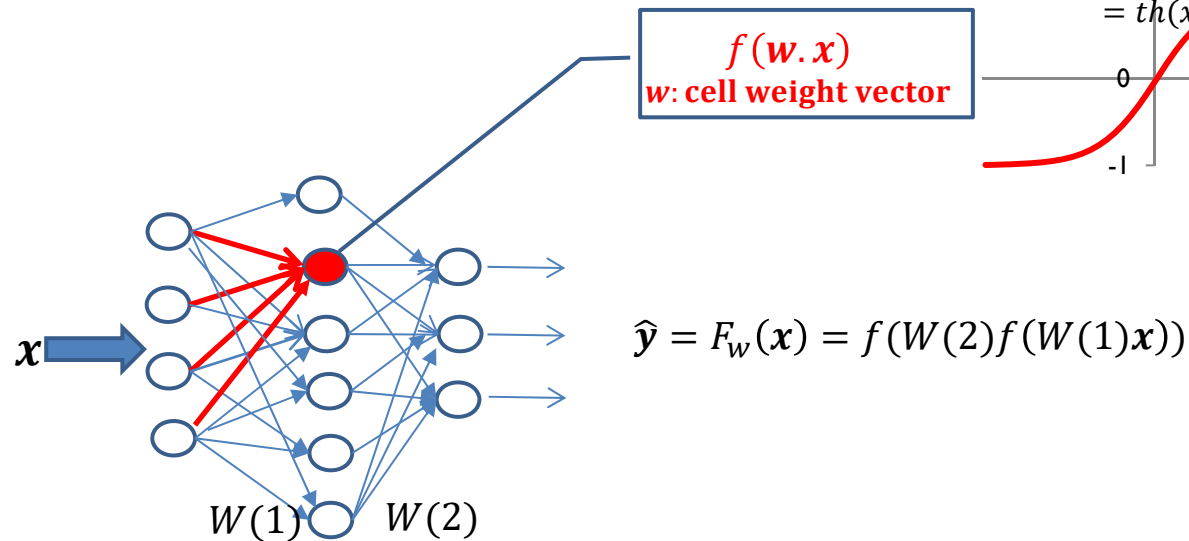
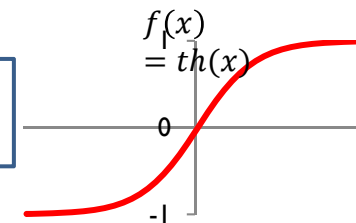
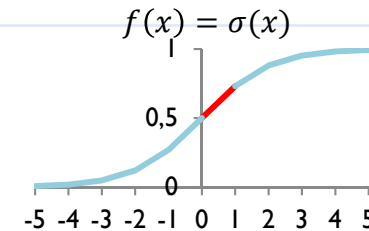
► Note

- Similar results hold for the cross entropy criterion
- Precision on the computed outputs depends on the task
 - Classification: precision might not be so important (max decision rule, one wants the correct class to be ranked above all others)
 - Posterior probability estimation: precision is important

Multi-layer Perceptron

Multi-layer Perceptron (Hinton – Sejnowski – Williams 1986)

- ▶ Neurons arranged into layers
- ▶ Each neuron is a non linear unit, e.g.



<http://playground.tensorflow.org/>

Note: here f applies pointwise, if $x = (x_1, x_2)$, $f(x_1, x_2) = (f(x_1), f(x_2))$

Multi-layer Perceptron - Training

▶ **Stochastic Gradient Descent** - The algorithm is called **Back-Propagation**

- ▶ Pick one example (x, y) or a **Mini Batch** $\{(x^i, y^i)\}$ sampled from the training set
 - ▶ Here the algorithm is described for 1 example (pure SGD) and for the sigmoid ($f(\cdot) = \sigma(\cdot)$) non linearity
- ▶ **Forward pass**
 - $\hat{y} = F_w(x) = f(W(2)f(W(1)x))$
- ▶ **Compute error**
 - $c(y, \hat{y})$, e.g. mean square error or cross entropy
- ▶ **Backward pass**
 - ▶ efficient implementation of chain rule
 - ▶ $W = W - \epsilon \frac{\partial c(y, \hat{y})}{\partial W}$

Algorithmic differentiation

- ▶ Back-Propagation is an instance of **automatic differentiation / algorithmic differentiation - AD**
 - ▶ A mathematical expression can be written as a **computation graph**
 - ▶ i.e. graph decomposition of the expression into elementary computations
 - ▶ **AD** allows to **compute** efficiently the derivatives of every element in the graph w.r.t. any other element.
 - ▶ **AD** transforms a programs computing a numerical funtion into the program for computing the derivatives
- ▶ All DL framework implement AD
 - ▶ There are multiple instances of AD – we will use here reverse AD

Notations – vector calculus - derivatives

- ▶ Different conventions exist for vector/ matrix derivatives
- ▶ We use here the « denominator layout » convention most often used for BP derivation
 - ▶ Gradients and their arguments keep the same shape

Matrix cookbooks

<http://www.cs.toronto.edu/~roweis/notes/matrixid.pdf> –
http://www.imm.dtu.dk/pubdb/views/edoc_download.php/3274/pdf/imm3274.pdf
https://en.wikipedia.org/wiki/Matrix_calculus

$$y = f(x), f: \mathbb{R}^n \rightarrow \mathbb{R}$$

$$\nabla_x y = \frac{\partial f}{\partial x} = \begin{pmatrix} \frac{\partial y}{\partial x_1} \\ \vdots \\ \frac{\partial y}{\partial x_n} \end{pmatrix} \in \mathbb{R}^n$$

Gradient

$$y = f(x), f: \mathbb{R}^n \rightarrow \mathbb{R}^m$$

$$J_{y(x)} = \frac{\partial y}{\partial x} = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_m}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_1}{\partial x_n} & \dots & \frac{\partial y_m}{\partial x_n} \end{pmatrix} \in \mathbb{R}^{n \times m} \text{ Jacobian}$$

$$y = f(W), f: \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$$

$$\nabla_W y = \frac{\partial y}{\partial W} = \begin{pmatrix} \frac{\partial y}{\partial w_{11}} & \dots & \frac{\partial y}{\partial w_{1n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial y}{\partial w_{m1}} & \dots & \frac{\partial y}{\partial w_{mn}} \end{pmatrix} \in \mathbb{R}^{m \times n} \text{ Gradient}$$

Notations – vector calculus - derivatives

- ▶ Additional useful derivations
- ▶ For matrix W and vector x
 - ▶ $\frac{\partial Wx}{\partial x} = W^T$
- ▶ For $u = f(x); f: \mathbb{R}^n \rightarrow \mathbb{R}^p$; $y = g(u); g: \mathbb{R}^p \rightarrow \mathbb{R}$
 - ▶ $\frac{\partial y}{\partial x_i} = \sum_{j=1}^p \frac{\partial y}{\partial u_j} \frac{\partial u_j}{\partial x_i}$

Vector chain rules – denominator layout

Vector case

$$u = f(x); f: \mathbb{R}^n \rightarrow \mathbb{R}^p$$
$$y = g(u); g: \mathbb{R}^p \rightarrow \mathbb{R}^m$$

$$\frac{\partial y}{\partial x} = \frac{\partial u}{\partial x} \frac{\partial y}{\partial u} \in \mathbb{R}^{n \times m}$$

Note:

$$\frac{\partial u}{\partial x} \in \mathbb{R}^{n \times p}; \frac{\partial y}{\partial u} \in \mathbb{R}^{p \times m}$$

Scalar case

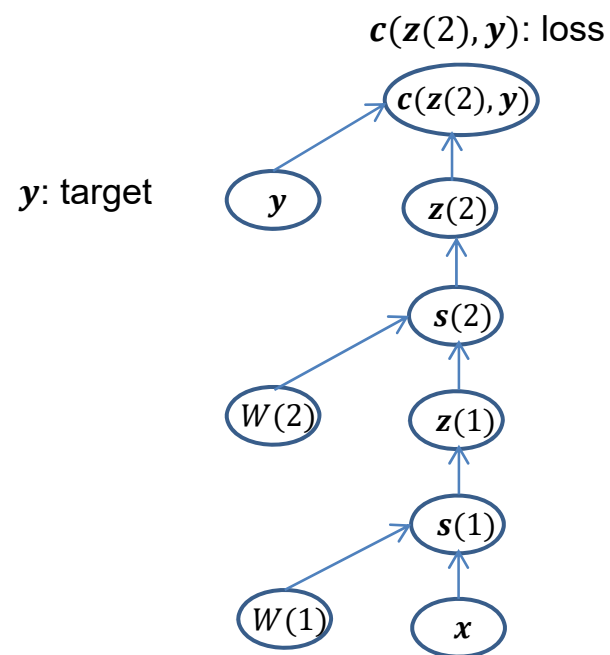
$$u = f(x); f: \mathbb{R}^n \rightarrow \mathbb{R}^p$$
$$y = g(u); g: \mathbb{R}^p \rightarrow \mathbb{R}$$

$$\nabla_x y = \frac{\partial y}{\partial x} = \frac{\partial u}{\partial x} \nabla_u y \in \mathbb{R}^n$$

$$\frac{\partial u}{\partial x} \in \mathbb{R}^{n \times p}; \frac{\partial y}{\partial u} \in \mathbb{R}^{p \times 1}$$

Multi-layer Perceptron - Training

► Computational graph

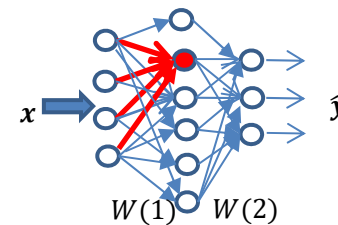


Here, $z(0) = x, z(2) = \hat{y}$

Forward propagation, $l = 1, 2$:

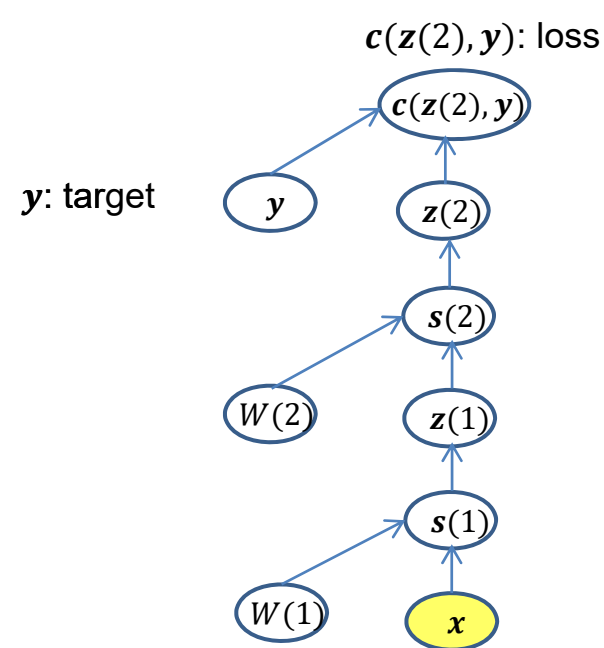
$$s(l) = W(l)z(l-1)$$

$$z(l) = \sigma(s(l))$$



Multi-layer Perceptron - Training

► Forward pass



Here, $z(0) = x, z(2) = \hat{y}$

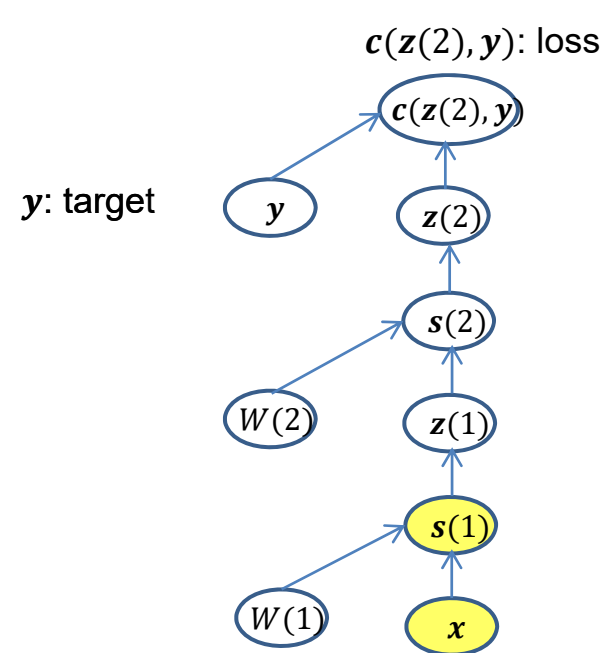
Forward propagation, $l = 1, 2$:

$$s(l) = W(l)z(l-1)$$

$$z(l) = \sigma(s(l))$$

Multi-layer Perceptron - Training

► Forward pass



Here, $z(0) = x, z(2) = \hat{y}$

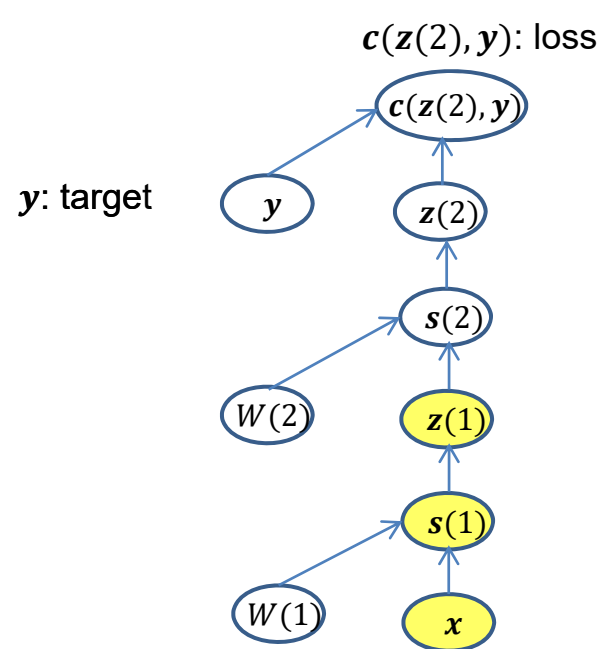
Forward propagation, $l = 1, 2$:

$$s(l) = W(l)z(l-1)$$

$$z(l) = \sigma(s(l))$$

Multi-layer Perceptron - Training

► Forward pass



Here, $z(0) = x, z(2) = \hat{y}$

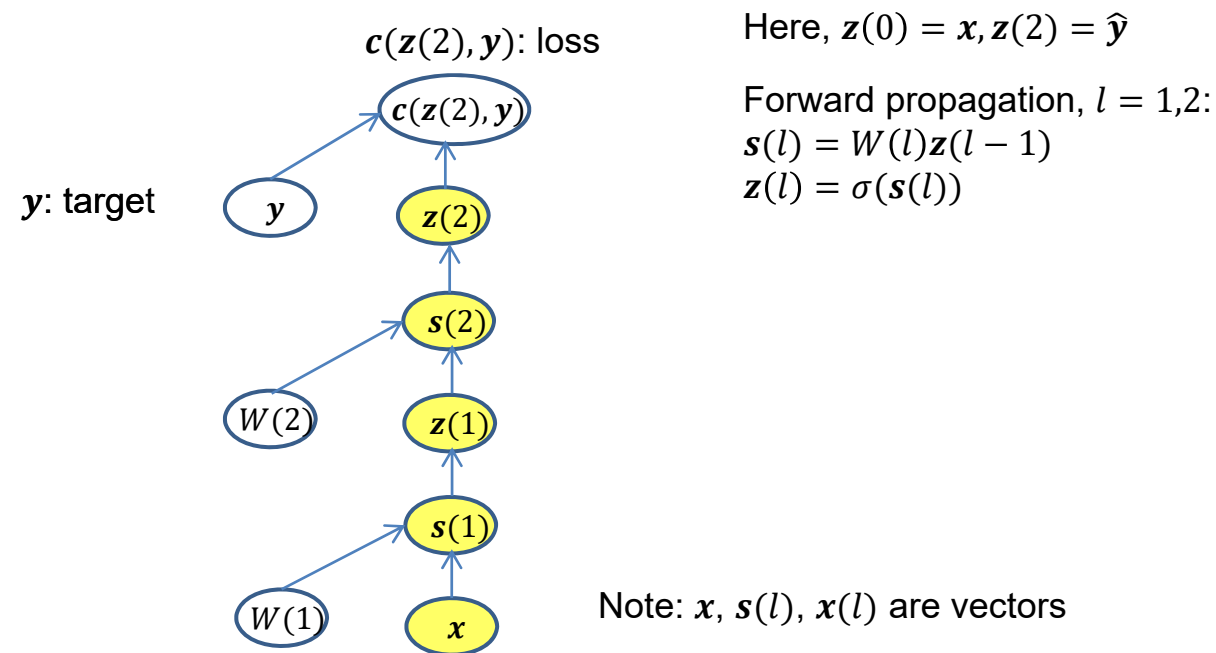
Forward propagation, $l = 1, 2$:

$$s(l) = W(l)z(l-1)$$

$$z(l) = \sigma(s(l))$$

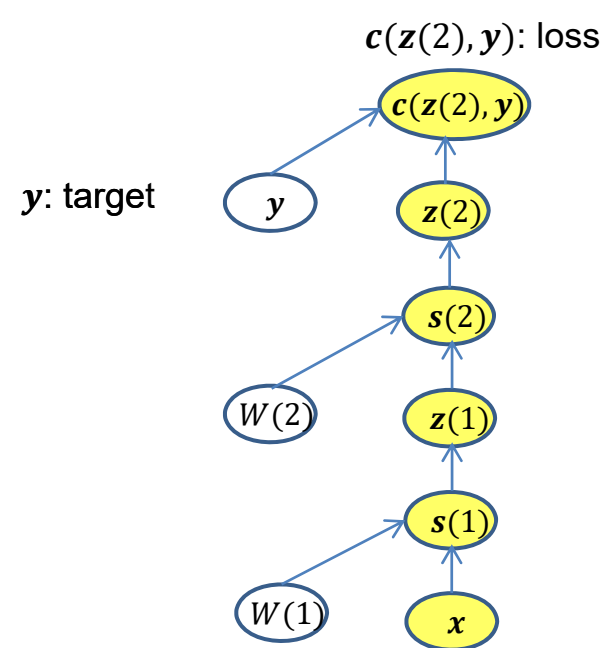
Multi-layer Perceptron - Training

► Forward pass



Multi-layer Perceptron - Training

► Forward pass



Here, $z(0) = x, z(2) = \hat{y}$

Forward propagation, $l = 1, 2$:

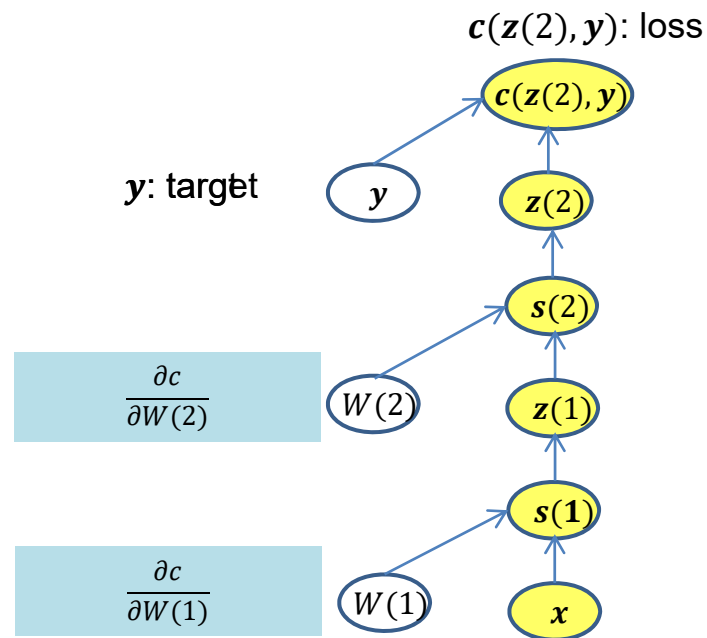
$$s(l) = W(l)z(l-1)$$

$$z(l) = \sigma(s(l))$$

Note: $x, s(l), z(l)$ are vectors

Multi-layer Perceptron - Training

► Back Propagation: Reverse Mode Differentiation

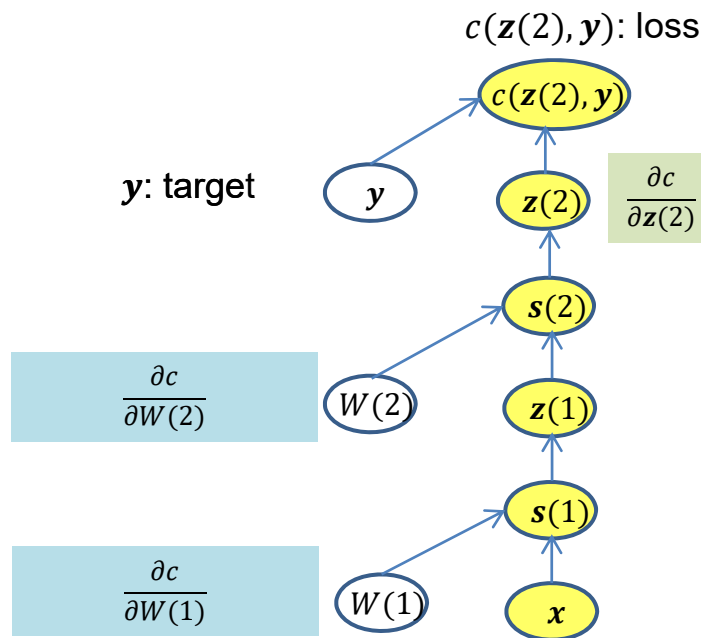


$$W = W - \epsilon \frac{\partial c}{\partial W}$$

Note: notations are in vector form, $\frac{\partial c}{\partial W}$ is a matrix of the same shape as W

Multi-layer Perceptron - Training

► Back propagation: Reverse Mode Differentiation



Backward propagation, $l = 1, 2$

$$\frac{\partial c}{\partial \mathbf{s}(l)} = \frac{\partial c}{\partial \mathbf{z}(l)} \odot \sigma'(\mathbf{s}(l))$$

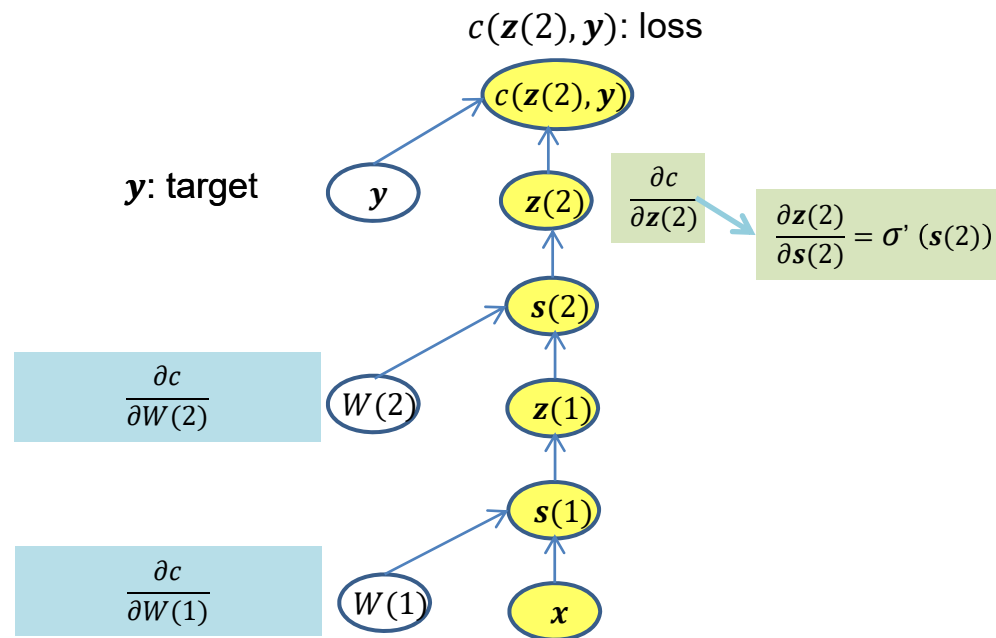
$$\frac{\partial c}{\partial \mathbf{W}(l)} = \frac{\partial c}{\partial \mathbf{s}(l)} \mathbf{z}(l-1)^T$$

$$\frac{\partial c}{\partial \mathbf{z}(l-1)} = \mathbf{W}(l)^T \frac{\partial c}{\partial \mathbf{s}(l)}$$

Note: notations are in vector form, $\frac{\partial c}{\partial \mathbf{W}}$ is a matrix of the same shape as $\mathbf{W}$, $\frac{\partial c}{\partial \mathbf{z}}$ and $\frac{\partial c}{\partial \mathbf{s}}$ are column vectors of the same size as $\mathbf{z}$ and $\mathbf{s}$

Multi-layer Perceptron - Training

► Back propagation: Reverse Mode Differentiation



Backward propagation, $l = 1, 2$

$$\frac{\partial c}{\partial s(l)} = \frac{\partial c}{\partial z(l)} \odot \sigma'(s(l))$$

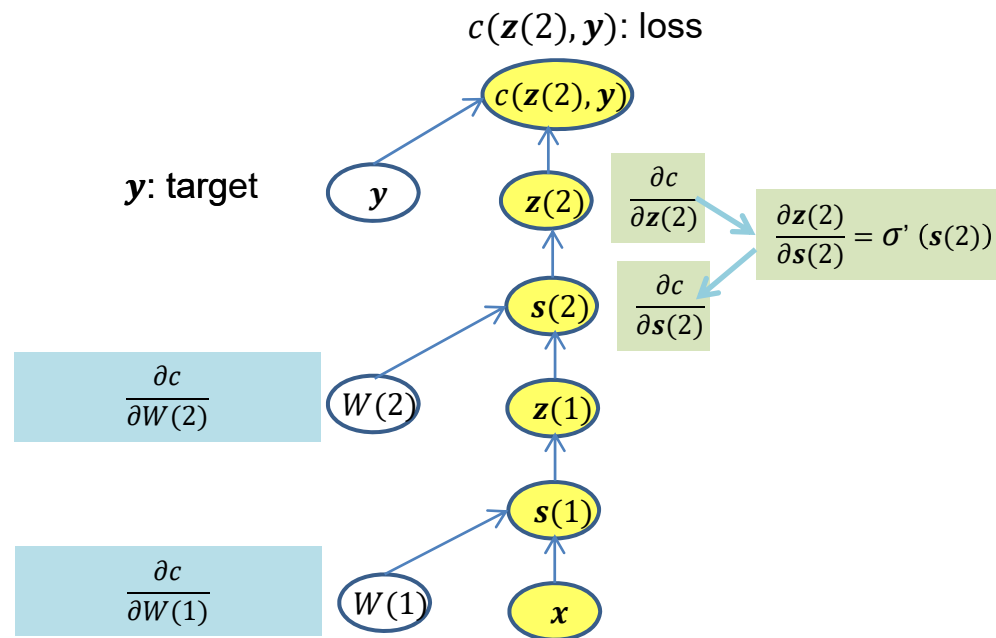
$$\frac{\partial c}{\partial W(l)} = \frac{\partial c}{\partial s(l)} z^{(l-1)T}$$

$$\frac{\partial c}{\partial z(l-1)} = W(l)^T \frac{\partial c}{\partial s(l)}$$

Note: notations are in vector form, $\frac{\partial c}{\partial W}$ is a matrix of the same shape as W , $\frac{\partial c}{\partial z}$ and $\frac{\partial c}{\partial s}$ are column vectors of the same size as z and s

Multi-layer Perceptron - Training

► Back propagation: Reverse Mode Differentiation



Backward propagation, $l = 1, 2$

$$\frac{\partial c}{\partial s(l)} = \frac{\partial c}{\partial z(l)} \odot \sigma'(s(l))$$

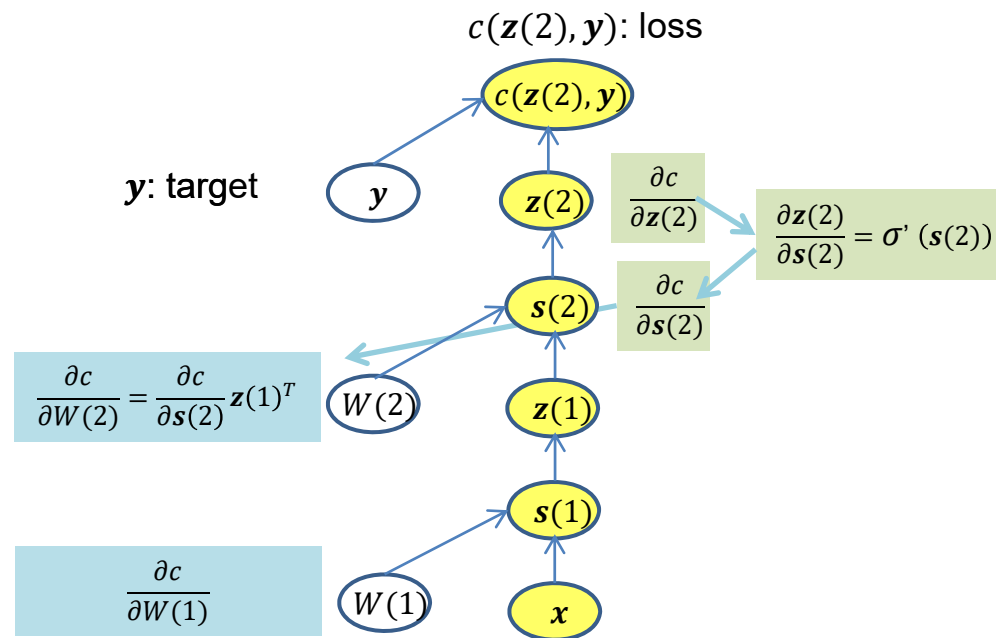
$$\frac{\partial c}{\partial W(l)} = \frac{\partial c}{\partial s(l)} z^{(l-1)T}$$

$$\frac{\partial c}{\partial z(l-1)} = W(l)^T \frac{\partial c}{\partial s(l)}$$

Note: notations are in vector form, $\frac{\partial c}{\partial W}$ is a matrix of the same shape as W , $\frac{\partial c}{\partial z}$ and $\frac{\partial c}{\partial s}$ are column vectors of the same size as z and s

Multi-layer Perceptron - Training

► Back propagation: Reverse Mode Differentiation



Backward propagation, $l = 1, 2$

$$\frac{\partial c}{\partial s(l)} = \frac{\partial c}{\partial z(l)} \odot \sigma'(s(l))$$

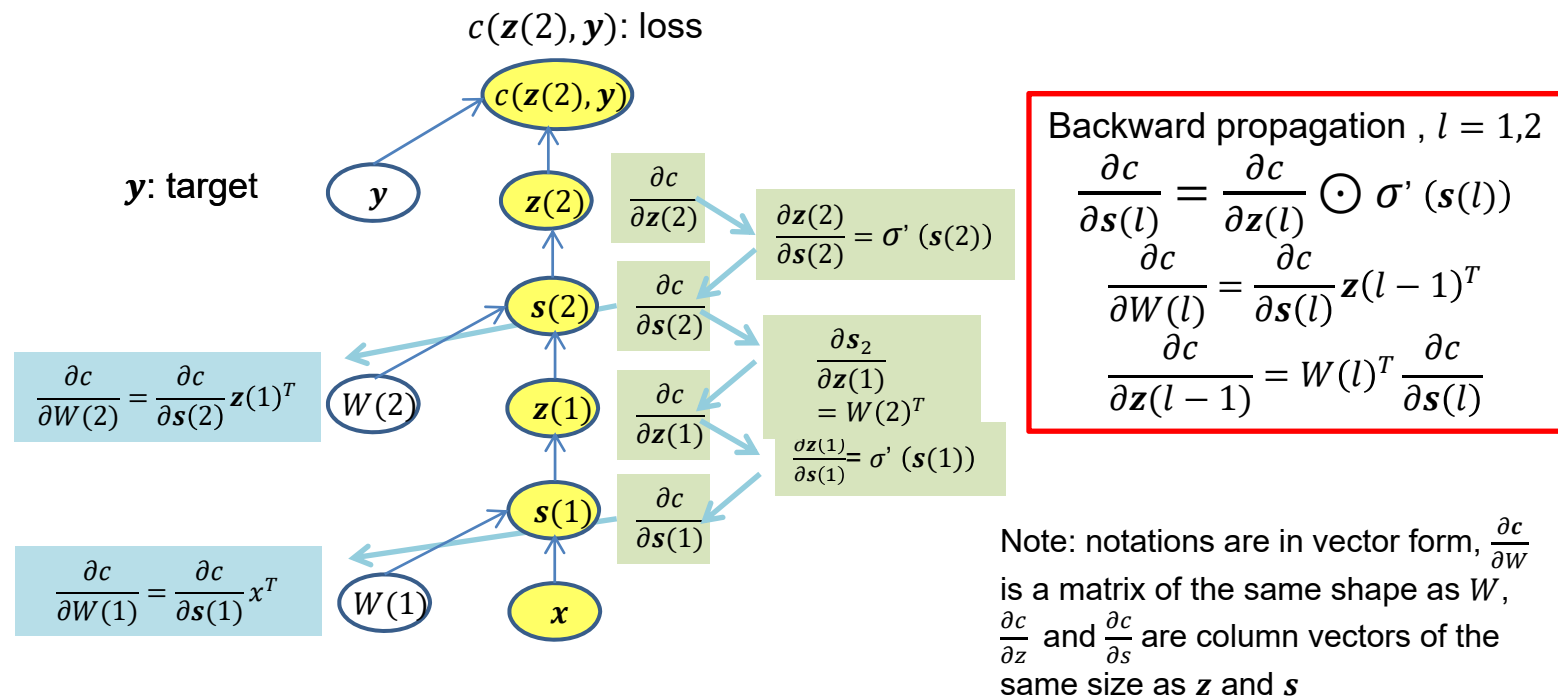
$$\frac{\partial c}{\partial W(l)} = \frac{\partial c}{\partial s(l)} z(l-1)^T$$

$$\frac{\partial c}{\partial z(l-1)} = W(l)^T \frac{\partial c}{\partial s(l)}$$

Note: notations are in vector form, $\frac{\partial c}{\partial W}$ is a matrix of the same shape as W , $\frac{\partial c}{\partial z}$ and $\frac{\partial c}{\partial s}$ are column vectors of the same size as z and s

Multi-layer Perceptron - Training

► Back propagation: Reverse Mode Differentiation

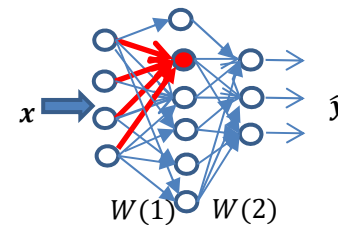


Chain rule derivation – vector form

MLP architecture

► Let us consider a two layers fully connected MLP

- $x \in \mathbb{R}^n$
- $s(1) = W(1)x \in \mathbb{R}^p$
- $z(1) = \sigma(s(1)) \in \mathbb{R}^p$
- $s(2) = W(2)z(1) \in \mathbb{R}^m$
- $z(2) = \sigma(s(2)) \in \mathbb{R}^m$
- $c = c(y, z(2)) \in \mathbb{R}$



Chain rule derivation – vector form

► Error signal layer 2

- $\delta_2 = \frac{\partial c}{\partial s(2)} = \frac{\partial z(2)}{\partial s(2)} \frac{\partial c}{\partial z(2)}$
- $\frac{\partial z(2)}{\partial s(2)} = \text{diag}(\sigma'(s(2))) \in \mathbb{R}^{m \times m}$
- $\delta_2 = \frac{\partial c}{\partial z(2)} \odot \sigma'(s(2)) \in \mathbb{R}^m$

► Error signal layer 1

- $\delta_1 = \frac{\partial c}{\partial s(1)} = \frac{\partial z(1)}{\partial s(1)} \frac{\partial s(2)}{\partial z(1)} \frac{\partial c}{\partial s(2)}$
- $s(2) = W(2)z(1) \Rightarrow \frac{\partial s(2)}{\partial z(1)} = W(2)^T$
- $\delta_1 = W(2)^T \delta_2 \odot \sigma'(s(1)) \in \mathbb{R}^n$

► Gradient w.r.t. $W(2)$

- $\frac{\partial c}{\partial W(2)} = \frac{\partial s(2)}{\partial W(2)} \frac{\partial c}{\partial s(2)}$
- $\frac{\partial c}{\partial W(2)} = \delta_2 z(1)^T$

► Gradient w.r.t. $W(1)$

- $\frac{\partial c}{\partial W(1)} = \frac{\partial s(1)}{\partial W(1)} \frac{\partial c}{\partial s(1)}$
- $\frac{\partial c}{\partial W(1)} = \delta_1 x^T$

Detailed derivations: https://www.kamperh.com/notes/kamper_backprop17.pdf

Chain rule derivation – vector form

- ▶ $\frac{\partial c}{\partial W(t)} = \frac{\partial s(t)}{\partial W(t)} \frac{\partial c}{\partial s(t)}$
 - ▶ $s_i(t) = \sum_j W_{ij} z_j(t-1)$
 - ▶ $\frac{\partial s_i}{\partial W_{kl}(t)} = z_l(t-1)$ if $i = k$, 0 otherwise
 - ▶ $\frac{\partial c}{\partial W_{kl}(t)} = \sum_i \frac{\partial c}{\partial s_i(t)} \frac{\partial s_i(t)}{\partial W_{kl}(t)}$
 - ▶ $\frac{\partial c}{\partial W_{kl}(t)} = [\delta_t]_k z_l(t-1)$
 - ▶ This is the classical form of the chain rule since the derivation is wrt a scalar W_{kl}
- ▶ $\frac{\partial c}{\partial W(t)} = \delta_t z_l^T(t-1)$

- ▶ $\frac{\partial c}{\partial W} = \begin{pmatrix} \frac{\partial c}{\partial W_{11}} & \cdots & \frac{\partial c}{\partial W_{1n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial c}{\partial W_{m1}} & \cdots & \frac{\partial c}{\partial W_{mn}} \end{pmatrix}$
- ▶ $\frac{\partial c}{\partial W} = \begin{pmatrix} [\delta]_1 z_1 & \cdots & [\delta]_1 z_n \\ \vdots & \ddots & \vdots \\ [\delta]_m z_1 & \cdots & [\delta]_m z_n \end{pmatrix}$
- ▶ $\frac{\partial c}{\partial W} = \delta Z^T$

Chain rule derivation – vector form

- ▶ Let us now consider a MLP with L layers and σ as activation function
- ▶ For an arbitrary layer, the recursion takes the following form

$$\begin{aligned}\delta_l &= \frac{\partial c}{\partial s(l)} = \frac{\partial c}{\partial \mathbf{z}(l)} \odot \sigma'(s(l)) \\ \delta_l &= W(l+1)^T \delta_{l+1} \odot \sigma'(s(l)) \text{ for } l < L \\ \frac{\partial c}{\partial W(l)} &= \delta_l \mathbf{z}(l-1)^T \\ \frac{\partial c}{\partial \mathbf{z}(l-1)} &= W^T(l)^T \delta_l\end{aligned}$$

Back Propagation and Adjoint

- ▶ BP is an instance of a more general technique: the Adjoint method
- ▶ Adjoint method
 - ▶ has been designed for computing **efficiently** the sensitivity of a loss to the parameters of a function (e.g. weights, inputs or any cell value in a NN).
 - ▶ Can be used to solve different constrained optimization problems (including BP)
 - ▶ Is used in many fields like control, geosciences
 - ▶ Interesting to consider the link with the adjoint formulation since this opens the way to generalization of the BP technique to more general problems
 - ▶ e.g. continuous NNs (Neural ODE)

Back Propagation and Adjoint

- ▶ Learning problem

- ▶ $Min_W c = \frac{1}{N} \sum_{k=1}^N c(F(x^k), y^k)$

- ▶ With $F(x) = F_l \circ \dots \circ F_1(x)$

- ▶ Rewritten as a constrained optimisation problem

- ▶ $Min_W c = \frac{1}{N} \sum_{k=1}^N c(z^k(l), y^k)$

- ▶ Subject to $\forall k = 1 \dots N$
$$\begin{cases} z^k(l) = F_l(z^k(l-1), w(l)) \\ z^k(l-1) = F_{l-1}(z^k(l-2), w(l-1)) \\ \dots \\ z^k(1) = F_1(x^k, w(1)) \end{cases}$$

- ▶ Note

- ▶ z and w are vectors of the appropriate size

- $z(i)$ is $n_z(i) \times 1$ and $w(i)$ is $n_w(i) \times 1$

- note: here we consider a flattened vector w instead of a matrix W

Back Propagation and Adjoint

- ▶ For simplifying, one considers pure SGD, i.e. $N = 1$
 - ▶ So that we drop the index k
- ▶ The Lagrangian associated to the optimization problem is
 - ▶ $\mathcal{L}(x, w) = c(z(l), y) - \sum_{i=1}^l \lambda_i^T (z(i) - F_i(z(i-1), w(i)))$
 - ▶ λ_i is a vector with the same size as $z(i)$
 - ▶ Unknowns to be estimated:
 - ▶ $z(i), w(i), \lambda_i, i = 1 \dots l,$

Back Propagation and Adjoint

► We want to solve for the Lagrangian

- $\mathcal{L}(x, W) = c(z(l), y) - \sum_{i=1}^l \lambda_i^T (z(i) - F_i(z(i-1), w(i)))$
- with unknowns: $z(i), w(i), \lambda_i, i = 1, \dots, l$

► The partial derivatives of the Lagrangian are

- $\frac{\partial \mathcal{L}}{\partial z(l)} = -\lambda_l + \frac{\partial c(z(l), y)}{\partial z(l)}$ for the last layer l
- $\frac{\partial \mathcal{L}}{\partial z(i)} = -\lambda_i + \frac{\partial F_{i+1}(z(i), w(i+1))}{\partial z(i)} \lambda_{i+1}, i = 1, \dots, l-1$ for intermediate layer i
- $\frac{\partial \mathcal{L}}{\partial w(i)} = \frac{\partial F_i(z(i-1), w(i))}{\partial w(i)} \lambda_i, i = 1 \dots l$
- $\frac{\partial \mathcal{L}}{\partial \lambda_i} = z(i) - F_i(z(i-1), w(i)), i = 1 \dots l$

► Note

- $\frac{\partial \mathcal{L}}{\partial z(i)}$ is $n_z(i) \times 1$, $\frac{\partial \mathcal{L}}{\partial w(i)}$ is $n_w(i) \times 1$, $\frac{\partial \mathcal{L}}{\partial \lambda_i}$ is $n_z(i) \times 1$, λ_i is $n_z(i) \times 1$, $\frac{\partial F_{i+1}(z(i), w(i+1))}{\partial z(i)}$ is $n_z(i) \times n_z(i+1)$, $\frac{\partial c(z(l), y)}{\partial z(l)}$ is $n_z(l) \times 1$, $\frac{\partial F_i(z(i-1), w(i))}{\partial w(i)}$ is $n_w(i) \times n_z(i)$

Back Propagation and Adjoint

▶ Forward equation

- ▶ $\frac{\partial \mathcal{L}}{\partial \lambda_i} = z(i) - F_i(z(i-1), w(i))$, $i = 1 \dots l$, represent the constraints
- ▶ One wants $\frac{\partial \mathcal{L}}{\partial \lambda_i} = 0$, $i = 1 \dots l$
- ▶ Starting from $i = 1$ up to $i = l$, this is exactly the forward pass of BP

▶ Backward equation

- ▶ Remember the Lagrangian
 - ▶ $\mathcal{L}(x, W) = c(z(l), y) - \sum_{i=1}^l \lambda_i^T (z(i) - F_i(z(i-1), w(i)))$
- ▶ Since one imposes $(z(i) - F_i(z(i-1), w(i))) = 0$ (forward pass), one can choose λ_i^T as we want
- ▶ Let us choose the λ s such that $\frac{\partial \mathcal{L}}{\partial z(i)} = 0, \forall i$
- ▶ The λ s can be computed backward Starting at $i = l$ down to $i = 1$
 - ▶ $\lambda_l = \frac{\partial c(z(l), y)}{\partial z(l)}$
 - ▶ ...
 - ▶ $\lambda_i = \frac{\partial F_{i+1}(z(i), w(i+1))}{\partial z(i)} \lambda_{i+1} = \frac{\partial z(i+1)}{\partial z(i)} \lambda_{i+1}$

Back Propagation and Adjoint

► Derivatives

- All that remains is to compute the derivatives of $\mathcal{L}$ wrt the W_i

- $\frac{\partial \mathcal{L}}{\partial w(i)} = \frac{\partial F_i(z(i-1), w(i))}{\partial w(i)} \lambda_{i+1} , \forall i$

- $\frac{\partial F_i(z(i-1), w(i))}{\partial w(i)} = \frac{\partial z(i)}{\partial w(i)}$ easy to compute

Back Propagation and Adjoint – Algorithm Recap

▶ Recap, BP algorithm with Adjoint

▶ Forward

▶ Solve forward $\frac{\partial \mathcal{L}}{\partial \lambda_i} = 0$

▶ $z(1) = F_1(z(0), w(1))$

▶ ...

▶ $z(i) = F_i(z(i-1), w(i))$

▶ Backward

▶ Solve backward $\frac{\partial \mathcal{L}}{\partial z(i)} = 0$

▶ $\lambda_l = \frac{\partial c(z(l), y)}{\partial z(l)}$

▶ ...

▶ $\lambda_i = \frac{\partial F_{i+1}(z(i), w(i+1))}{\partial z(i)} \lambda_{i+1} = \frac{\partial z(i+1)}{\partial z(i)} \lambda_{i+1}$

▶ Derivatives

□ $\frac{\partial \mathcal{L}}{\partial w(i)} = \frac{\partial F_i(z(i-1), w(i))}{\partial w(i)} \lambda_{i+1}, \forall i$

Multi-layer Perceptron – stochastic gradient

► Note

- The algorithm has been detailed for « pure » SGD, i.e. one datum at a time
- In practical applications, one uses mini-batch implementations
- This accelerates GPU implementations
- The algorithm holds for any differentiable loss/ model
- Deep Learning on large architectures makes use of SGD variants, e.g. Adam

Multi-layer Perceptron – stochastic gradient

► Note

- The algorithm has been detailed for « pure » SGD, i.e. one datum at a time
- In practical applications, one uses mini-batch implementations
- This accelerates GPU implementations
- The algorithm holds for any differentiable loss/ model
- Deep Learning on large architectures makes use of SGD variants, e.g. Adam

Approximation properties of MLPs

► Results based on functional analysis

► (Cybenko 1989)

- Theorem 1 (regression): Let f be a continuous saturating function, then the space of functions $g(x) = \sum_{j=1}^n v_j f(\mathbf{w}_j \cdot \mathbf{x})$ is dense in the space of continuous functions on the unit cube $C(I)$. i.e. $\forall h \in C(I)$ et $\forall \epsilon > 0, \exists g : |g(x) - h(x)| < \epsilon$ on I
- Theorem 2 (classification): Let f be a continuous saturating function. Let F be a decision function defining a partition on I . Then $\forall \epsilon > 0$, there exists a function $g(x) = \sum_{j=1}^n v_j f(\mathbf{w}_j \cdot \mathbf{x})$ and a set $D \subset I$ such that $\text{measure}(D) = 1 - \epsilon$ and $|g(x) - F(x)| < \epsilon$ on D
- .

► (Hornik et al., 1989)

- Theorem 3 : For any increasing saturating function f , and any probability measure m on R^n , the space of functions $g(x) = \sum_{j=1}^n v_j f(\mathbf{w}_j \cdot \mathbf{x})$ is uniformly dense on the compact sets $C(R^n)$ - the space of continuous functions on R^n

► Notes:

- None of these result is constructive
- Recent review of approximation properties of NN: Guhring et al., 2020, Expressivity of deep neural networks, arXiv:2007.04759

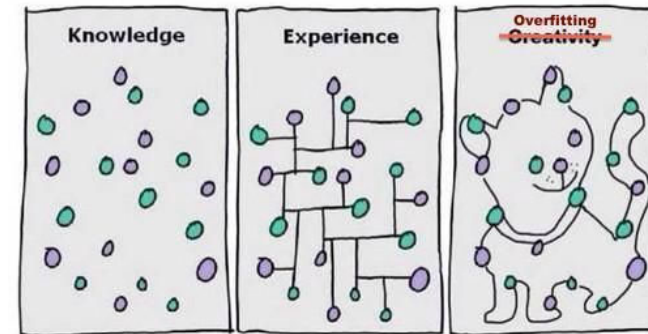
Complexity control

Bias – Variance

Overtraining and regularization

Generalization and Model Selection

- ▶ Complex models sometimes perform worse than simple linear models
 - ▶ Overfitting/ generalization problem

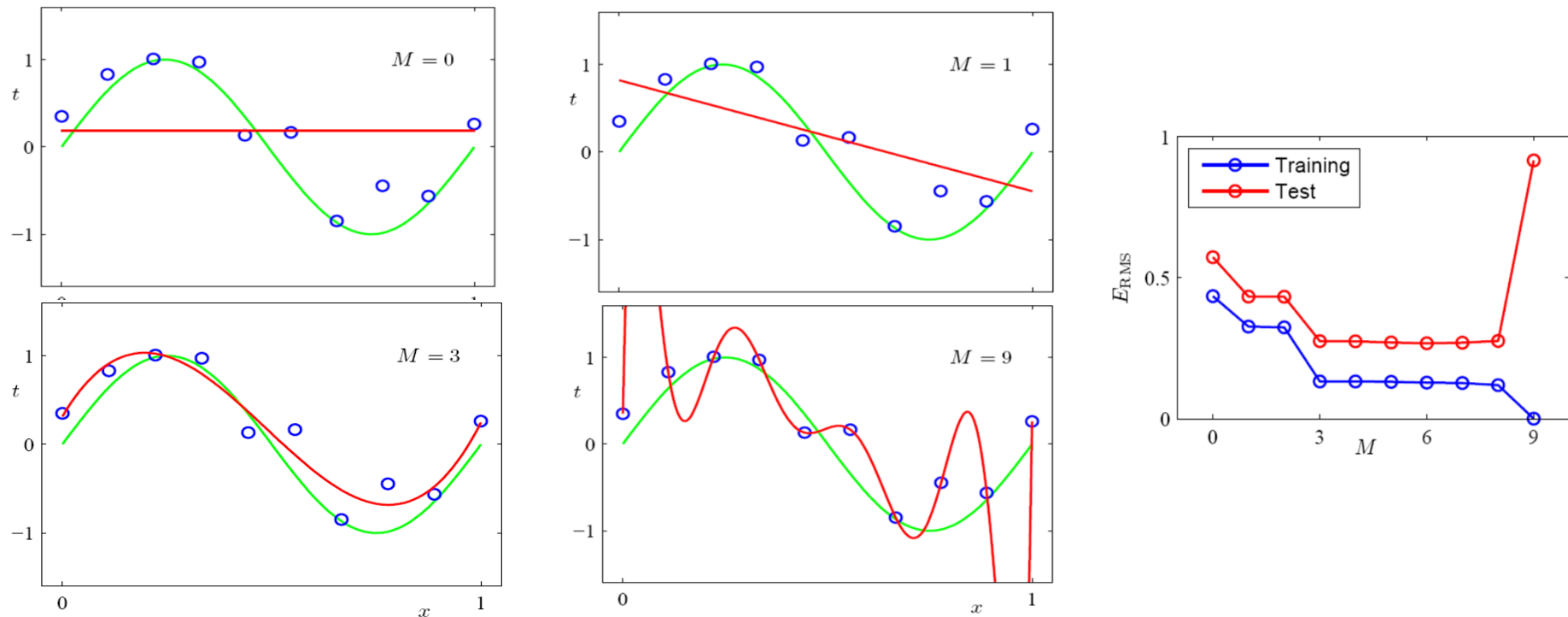


- ▶ Empirical Risk Minimization is not sufficient
 - ▶ The model complexity should be adjusted both to the task and to the information brought by the examples
 - ▶ Both the model parameters and the model capacity should be learned
 - ▶ Lots of practical method and of theory has been devoted to this problem

Complexity control

Overtraining / generalization for regression

- **Example** (Bishop 06) fit of a sinusoid with polynomials of varying degrees



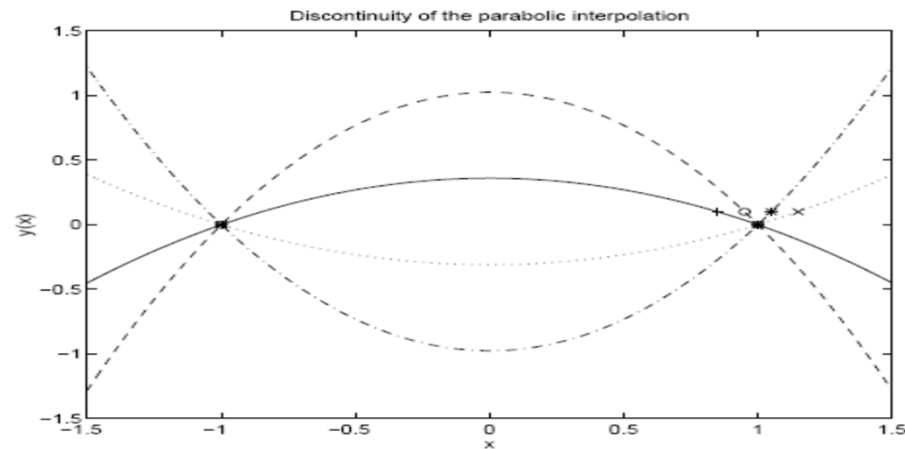
- Model complexity shall be controlled (learned) during training
 - How?

Complexity control

- ▶ One shall optimize the risk while controlling the complexity
- ▶ Several methods
 - ▶ Régularisation (Hadamard ...Tikhonov)
 - ▶ Theory of ill posed problems
 - ▶ Minimization of the structural risk (Vapnik)
 - ▶ Algebraic estimators of generalization error (AIC, BIC, LOO, etc)
 - ▶ Bayesian learning
 - ▶ Provides a statistical explanation of regularization
 - ▶ Regularization terms appear as priors on the parameter distribution
 - ▶ Ensemble methods
 - ▶ Boosting, bagging, etc
 - ▶ Many others especially in the Deep NN literature (seen later)

Regularisation

- ▶ Hadamard
 - ▶ A problem is well posed if
 - ▶ A solution exists
 - ▶ It is unique and stable
 - ▶ Example of ill posed problem (Goutte 1997)



- ▶ Tikhonov
 - ▶ Proposes methods pour transforming a ill posed problem into a “well” posed one

Bias-variance decomposition

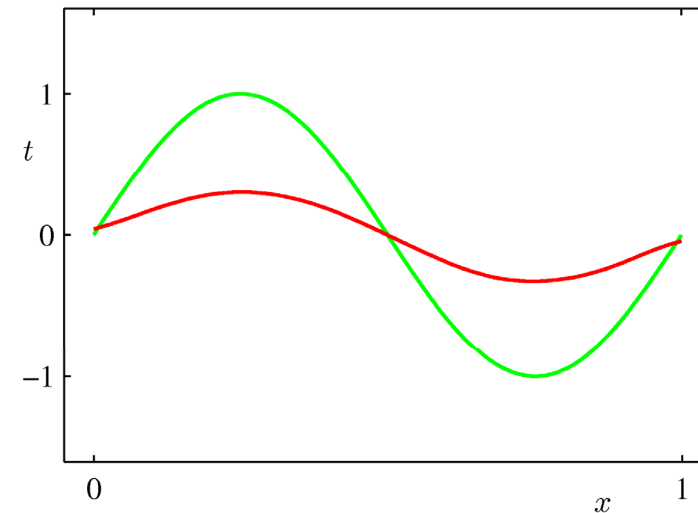
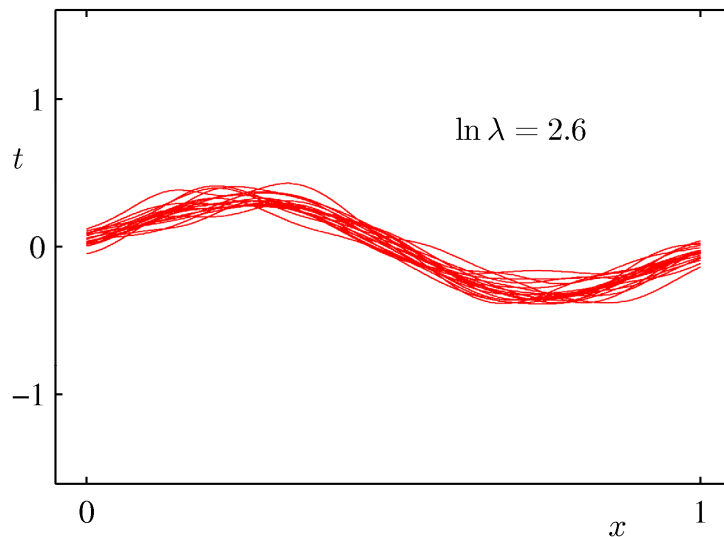
- ▶ Illustrates the problem of model selection, puts in evidence the influence of the complexity of the model
 - ▶ Remember: MSE risk decomposition
 - ▶ $E_{x,y} \left[(y - F_w(x))^2 \right] = E_{x,y} \left[(y - E_y[y|x])^2 \right] + E_{x,y} \left[(E_y[y|x] - F_w(x))^2 \right]$
 - ▶ Let $h^*(x) = E_y[y|x]$ be the optimal solution for the minimization of this risk
 - ▶ In practice, the number of training data for estimating $E_y[y|x]$ is limited
 - ▶ The estimation will depend on the training set D
 - ▶ Uncertainty due to the training set choice for this estimator can be measured as follows:
 - Sample a series of training sets, all of size N : $D_1, D_2, \dots$
 - Learn $F_w(x, D)$ for each of these datasets
 - Compute the mean of the empirical errors obtained on these different datasets

Bias-variance decomposition

- ▶ Let us consider the quadratic error $(F(x; D) - h^*(x))^2$ for a datum x and for the solution $F_w(x; D)$ obtained with the training set D (in order to simplify, we consider a 1 dimensional real output, extension to multidimensional outputs is trivial)
 - ▶ Let $E_{D \sim p(D)}[F_w(x; D)]$ denote the expectation w.r.t. the distribution of $D, p(D)$
- ▶ $(F_w(x; D) - h^*(x))^2$ decomposes as:
 - ▶ $(F_w(x; D) - h^*(x))^2 = (F_w(x; D) - E_D[F_w(x; D)] + E_D[F_w(x; D)] - h^*(x))^2$
 - ▶ $(F_w(x; D) - h^*(x))^2 = (F_w(x; D) - E_D[F_w(x; D)])^2 + (E_D[F_w(x; D)] - h^*(x))^2 + 2(F_w(x; D) - E_D[F_w(x; D)])(E_D[F_w(x; D)] - h^*(x))$
- ▶ Expectation w.r.t. D distribution decomposes as:
 - ▶ $E_D[(F_w(x; D) - h^*(x))^2] = (E_D[F_w(x; D)] - h^*(x))^2 + E_D[(F_w(x; D) - E_D[F_w(x; D)])^2]$
 - ▶ $\qquad \qquad \qquad = \qquad \qquad \qquad \text{bias}^2 \qquad \qquad \qquad + \qquad \qquad \qquad \text{variance}$
- ▶ Intuition
 - ▶ Choosing the right model requires a compromise between flexibility and simplicity
 - *Flexible model* : low bias – strong variance
 - *Simple model* : strong bias – low variance

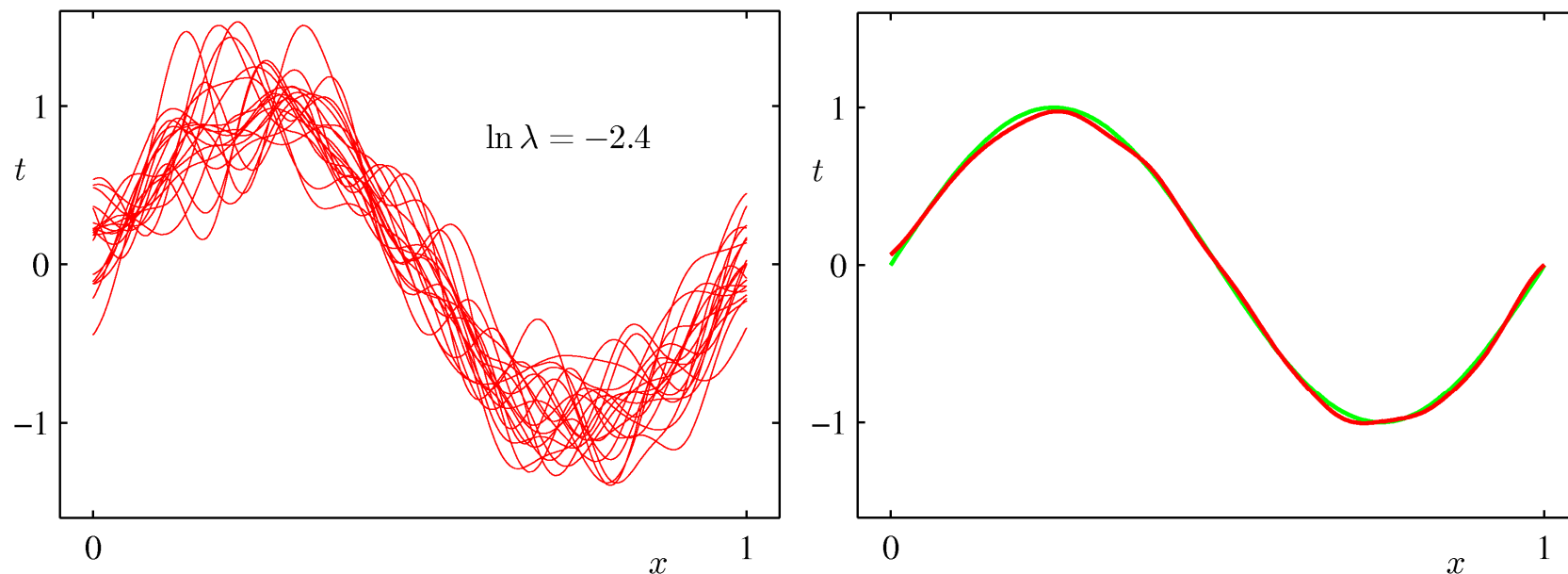
The Bias-Variance Decomposition (Bishop PRML 2006)

- ▶ Example: 100 data sets from the sinusoidal, varying the degree of regularization
 - ▶ Model: gaussian basis function, Learning set size = 25, λ is the regularization parameter
 - High values of λ correspond to simple models, low values to more complex models
 - ▶ Left 20 of the 100 models shown
 - ▶ Right : average of the 100 models (red), true sinusoid (green)
 - ▶ Figure illustrates high bias and low variance ($\lambda = 13$)



The Bias-Variance Decomposition (Bishop PRML 2006)

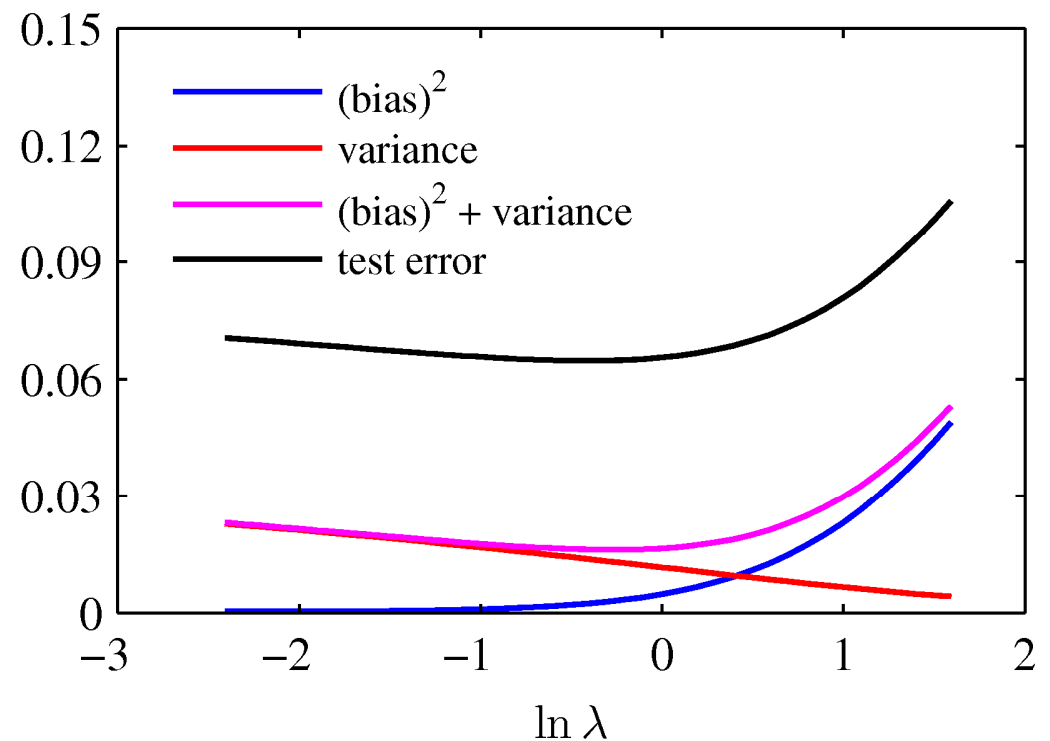
- ▶ Example: 100 data sets from the sinusoidal, varying the degree of regularization
 - ▶ Same setting as before
 - Figure illustrates low bias and high variance ($\lambda = 0.09$)



- ▶ Remark
 - The mean of several complex models behaves well here (reduced variance)
 - $\rightarrow$ leads to ensemble methods

The Bias-Variance Decomposition (Bishop PRML 2006)

- ▶ From these plots, we note that an over-regularized model (large λ) will have a high bias, while an under-regularized model (small λ) will have a high variance.



Regularisation

- ▶ Principle: control the solution variance by constraining function F
 - ▶ Optimise $C = C_1 + \lambda C_2$
 - ▶ C is a compromise between
 - ▶ C_1 : reflects the objective e.g. MSE, Entropie, ...
 - ▶ C_2 : constraints on the solution (e.g. weight distribution)
 - ▶ λ : constraint weight
- ▶ Regularized mean squares
 - ▶ For the linear multivariate regression
 - ▶ $C = \frac{1}{N} \sum_{i=1}^N (y^i - \mathbf{w} \cdot \mathbf{x}^i)^2 + \frac{\lambda}{2} \sum_{j=1}^n |w_j|^q$
 - ▶ $q = 2$ regularization L_2 , $q = 1$ regularization L_1 also known as « Lasso »

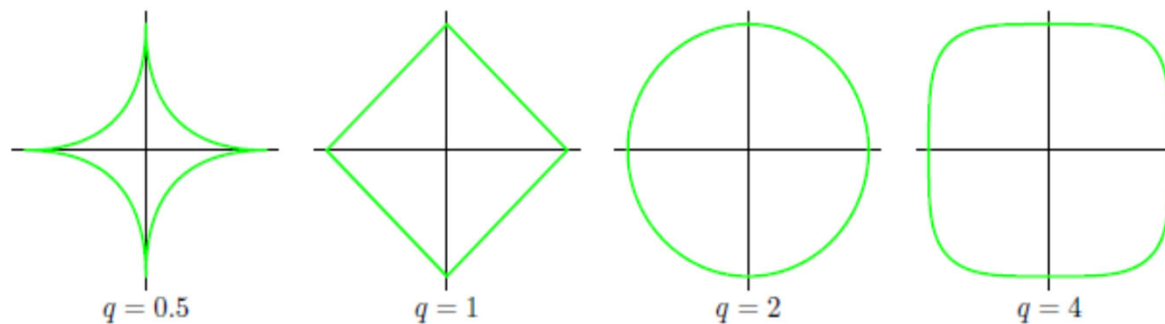


Figure 3.3 Contours of the regularization term in (3.29) for various values of the parameter q .

Fig. from Bishop 2006

Régularisation

► Solve

$$\text{► } \text{Min}_{\mathbf{w}} C = \frac{1}{N} \sum_{i=1}^N (y^i - \mathbf{w} \cdot \mathbf{x}^i)^2 + \frac{\lambda}{2} \sum_{j=1}^n |w_j|^q, \lambda > 0$$

► Amounts at solving the following constrained optimization problem

$$\text{► } \text{Min}_{\mathbf{w}} C = \frac{1}{N} \sum_{i=1}^N (y^i - \mathbf{w} \cdot \mathbf{x}^i)^2$$

► Under constraint $\sum_{j=1}^n |w_j|^q \leq s$ for a given value of s

► Effect of this constraint

Figure 3.4 Plot of the contours of the unregularized error function (blue) along with the constraint region (3.30) for the quadratic regularizer $q = 2$ on the left and the lasso regularizer $q = 1$ on the right, in which the optimum value for the parameter vector $\mathbf{w}$ is denoted by $\mathbf{w}^*$. The lasso gives a sparse solution in which $w_1^* = 0$.

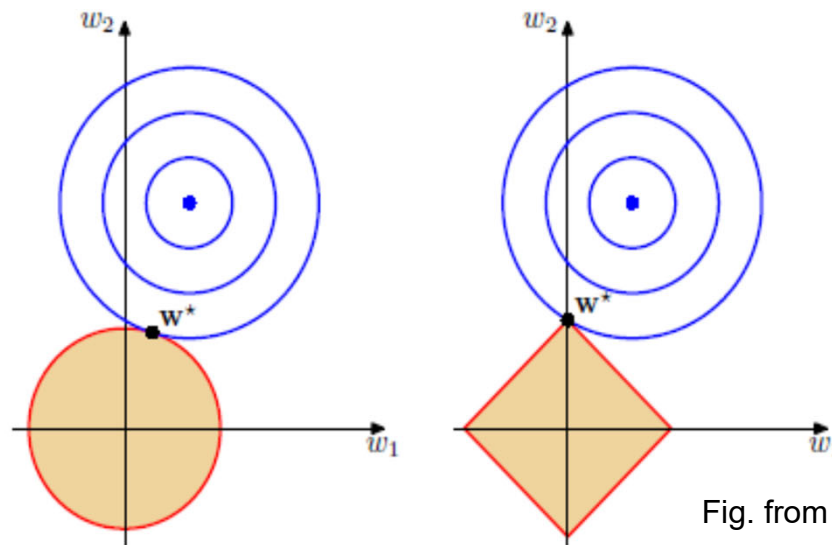


Fig. from Bishop 2006

Regularization

► Penalization L_2

► Loss

$$C = C_1 + \lambda \sum_{j=1}^n |w_j|^2$$

► Gradient

$$\nabla_{\mathbf{w}} C = \lambda \mathbf{w} + \nabla_{\mathbf{w}} C_1$$

► Update

$$\mathbf{w} = \mathbf{w} - \epsilon \nabla_{\mathbf{w}} C = (1 - \epsilon \lambda) \mathbf{w} - \epsilon \nabla_{\mathbf{w}} C_1$$

► Penalization is proportional to $\mathbf{w}$

► Penalization L_1

► Loss

$$C = C_1 + \lambda \sum_{j=1}^n |w_j|$$

► Gradient

$$\nabla_{\mathbf{w}} C = \lambda \text{sign}(\mathbf{w}) + \nabla_{\mathbf{w}} C_1$$

► $\text{sign}(\mathbf{w})$ is the sign of $\mathbf{w}$ applied to each component of $\mathbf{w}$

► Update

$$\mathbf{w} = \mathbf{w} - \epsilon \nabla_{\mathbf{w}} C = \mathbf{w} - \epsilon \lambda \text{sign}(\mathbf{w}) - \epsilon \nabla_{\mathbf{w}} C_1$$

► Penalization is constant with sign $\text{sign}(\mathbf{w})$

Other ideas for improving generalization in NNs

- ▶ Several heuristics have been developed in order to force inductive biases for NNs – some
 - ▶ Gradient descent and stochastic gradient descent perform implicit regularization
 - ▶ Weights initialization
 - ▶ Early stopping
 - ▶ Data augmentation
 - ▶ By adding noise
 - with early work from Matsuoka 1992 ; Grandvallet and Canu 1994 ; Bishop 1994
 - and many new developments for Deep learning models
 - ▶ By generating new examples (synthetic, or any other way)
 - ▶ Note: Bayesian learning and regularization
 - ▶ Regularization parameters correspond to priors on these model variables
 - ▶ Ensembling
 - ▶ Model averaging
 - Average models outputs: reduces the variance
 - ▶ Functional ensembling (recently developed)
 - Average the network weights on the training trajectory
 - As for 2022: SOTA in classification (e.g. vision tasks)

Generalization in modern Deep Learning

- ▶ Deep Learning models often do not follow the common complexity / performance wisdom
 - ▶ Extremely large models / with no complexity control (like e.g. regularization or early stopping), may reach good performance, better than models trained with the usual complexity control ingredients
 - ▶ Observed in modern deep learning
 - ▶ High complexity models with zero train error may not overfit and lead to accurate predictions on unseen data
 - This observation questions the usual claim and the theoretical beliefs such as Bias – Variance dilemma
- ▶ Example
 - ▶ Double descent phenomenon
 - ▶ Based on (Belkin 2019) and (Nakkiran 2020)

Generalization in modern Deep Learning - Double Descent

- ▶ Observed by different authors but formalized as a general concept in (Belkin 2019)
- ▶ General message
 - ▶ Learning curves as a function of model capacity (complexity) exhibit a two regimes phenomenon coined as « double descent »
 - ▶ Classical regime corresponds to under-parameterized models and exhibits the classical U shaped curve corresponding to the bias-variance intuition
 - ▶ Models do not achieve perfect interpolation
 - ▶ The test risk first decreases and then increases when the model starts interpolating
 - ▶ Modern interpolation regime corresponds to over-parameterized models
 - ▶ Models may achieve near zero train error, i.e. near perfect interpolation
 - ▶ Test risk value may decrease below the level of the best classical regime risk value

Generalization in modern Deep Learning - Double Descent Intuition (Belkin 2019)

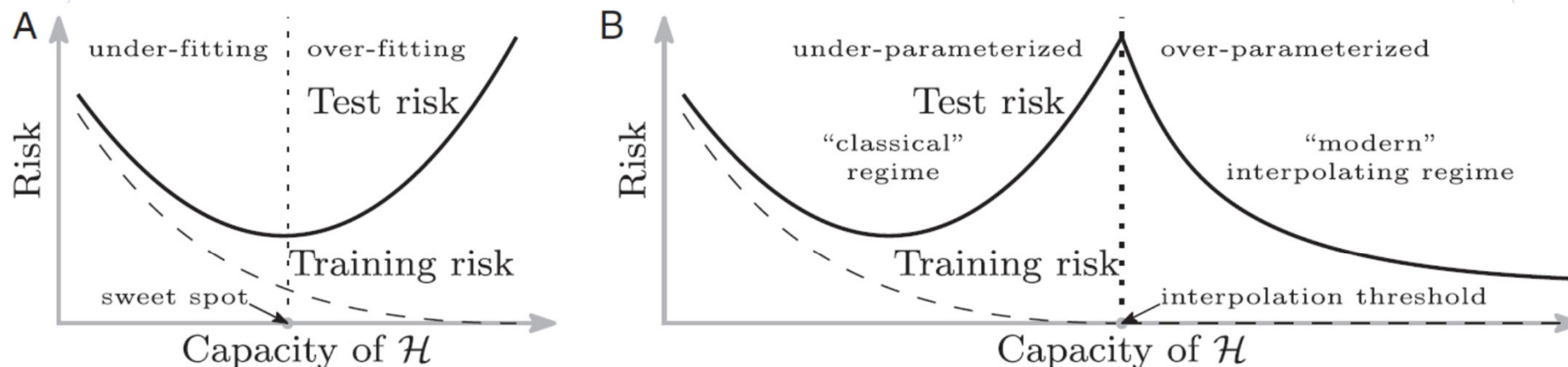


Fig. 1. Curves for training risk (dashed line) and test risk (solid line). (A) The classical U-shaped risk curve arising from the bias-variance trade-off. (B) The double-descent risk curve, which incorporates the U-shaped risk curve (i.e., the “classical” regime) together with the observed behavior from using high-capacity function classes (i.e., the “modern” interpolating regime), separated by the interpolation threshold. The predictors to the right of the interpolation threshold have zero training risk.

- ▶ All the models to the right of the interpolation threshold have a zero training error
- ▶ Tentative explanation
 - ▶ The notion of « capacity of the function class » does not fit the inductive bias appropriate for the problem and cannot explain the observed behavior
 - ▶ The **inductive bias** seems to be the **smoothness of a function** as measured by a certain function space norm

Generalization in modern Deep Learning - Double Descent Intuition (Belkin 2019)

- ▶ Characterization on classification problems
 - ▶ Model: Random Fourier Features
 - ▶ Equivalent to 1 hidden layer NN with fixed weights in the first layer
 - ▶ i.e. only the last weight layers are learned, i.e. convex problem
 - ▶ Because of the linearity of the trainable component, the complexity can be measured by the number of basis functions (nb of hidden cells)
 - Or at least this provides a proxy for the complexity
- ▶ Random Fourier Features
 - ▶ Consider a class of function denoted $\mathcal{H}_N : h(x) : R^d \rightarrow R$
 - ▶ With $h(x) = \sum_{k=1}^N a_k \phi(x; v_k)$ with $\phi(x; v) = \exp(i \langle v, x \rangle)$ - (the complex exponential)
 - ▶ Where the $v_1, \dots, v_N$ are sampled independently from the standard normal distribution in R^d
 - ▶ The $\phi(x; v)$ are N complex basis functions
 - ▶ This may be implemented as a NN with $2N$ basis functions corresponding to the real and imaginary parts of ϕ
 - ▶ Learning procedure
 - ▶ Given a training set $(x^1, y^1) \dots (x^n, y^n)$, train via ERM, i.e. minimize $\frac{1}{n} \sum_{i=1}^n (h(x^i) - y^i)^2$
 - ▶ When the minimizer is not unique (always the case when $N > n$) **choose the one with coefficients $(a_1, \dots, a_N)$ of minimum l_2 norm, i.e. the smoothest one**

Generalization in modern Deep Learning - Double Descent Intuition (Belkin 2019)

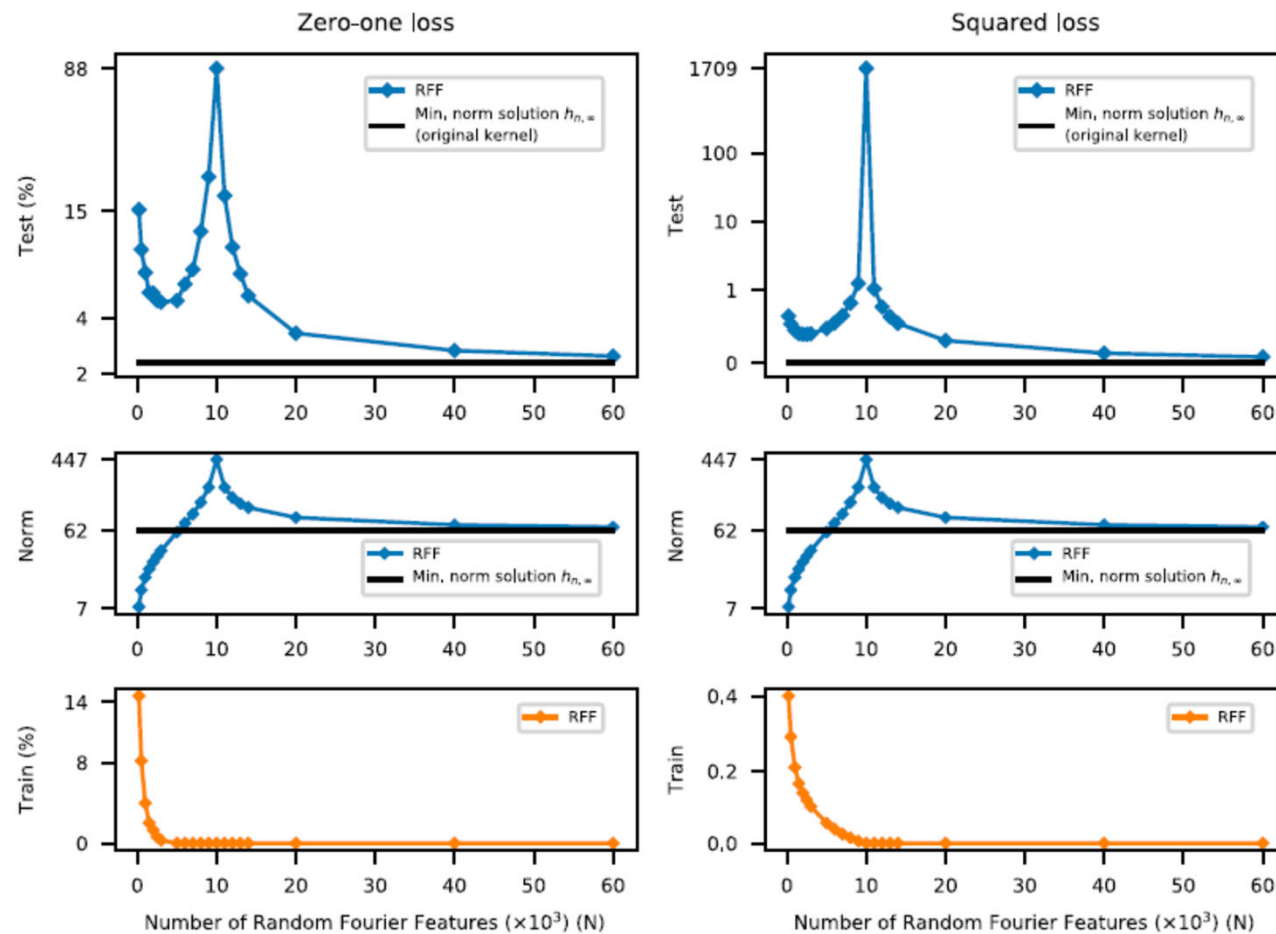


Fig. 2. Double-descent risk curve for the RFF model on MNIST. Shown are test risks (log scale), coefficient ℓ_2 norms (log scale), and training risks of the RFF model predictors $h_{n,N}$ learned on a subset of MNIST ($n = 10^4$, 10 classes). The interpolation threshold is achieved at $N = 10^4$.

Generalization in modern Deep Learning - Double Descent Intuition (Nakkiran 2020)

- ▶ Characterize the double descent phenomenon for
 - ▶ A large variety of NN models: CNN, ResNet, Transformers
 - ▶ Several settings: model-wise, epoch-wise, sample-wise (defined later)
- ▶ Propose a measure of complexity called « effective model complexity »
 - ▶ For non linear models, the number of parameters is not a characterization of the function class complexity

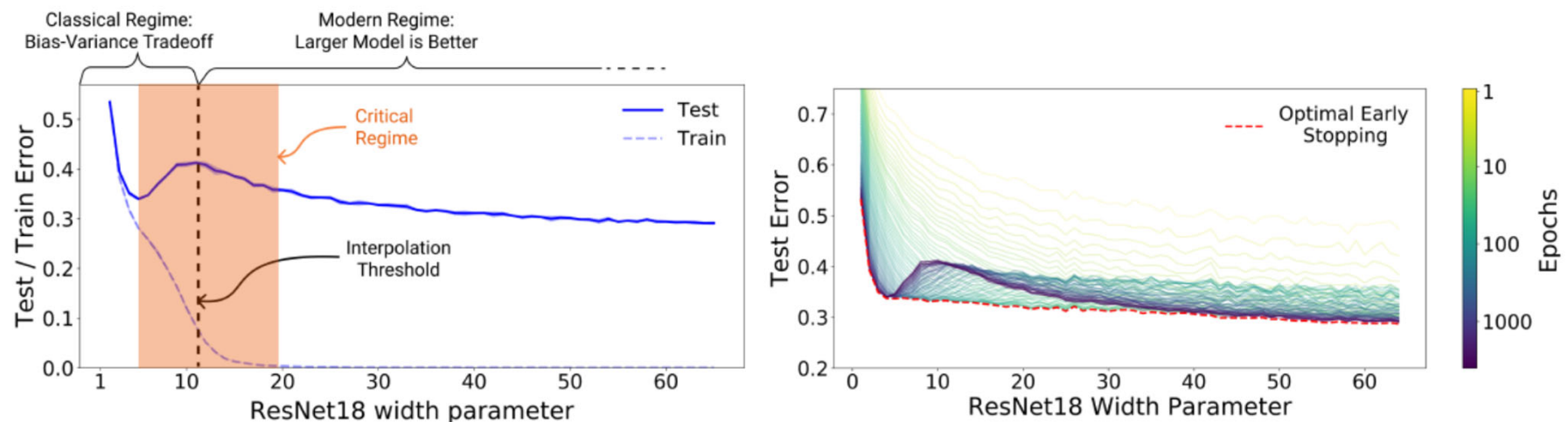


Figure 1: **Left:** Train and test error as a function of model size, for ResNet18s of varying width on CIFAR-10 with 15% label noise. **Right:** Test error, shown for varying train epochs. All models trained using Adam for 4K epochs. The largest model (width 64) corresponds to standard ResNet18.

Generalization in modern Deep Learning - Double Descent Intuition (Nakkiran 2020)

▶ Effective model complexity (EMC)

- ▶ A training procedure $\mathcal{T}$ is any procedure that takes as input a training set $D = \{(x^1, y^1), \dots, (x^n, y^n)\}$ and outputs a classifier $\mathcal{T}(D)$ mapping data to labels
- ▶ The effective model complexity of $\mathcal{T}$ w.r.t. the distribution $\mathcal{D}$ of D is the maximum number of samples n' on which $\mathcal{T}$ achieves on average a zero **training** error

▶ The EMC of training procedure $\mathcal{T}$ w.r.t. distribution $\mathcal{D}$ and parameter $\epsilon > 0$, is defined as:

- ▶ $EMC_{\mathcal{D},\epsilon}(\mathcal{T}) = \max \left\{ n' \mid E_{D \sim \mathcal{D}^{n'}} [Error_D(\mathcal{T}(D))] \leq \epsilon \right\}$
 - ▶ with $Error_D(\mathcal{T}(D))$ is the mean error on D .

▶ Regimes

- ▶ **Assumption: the classifier $\mathcal{T}(D)$ is trained on a dataset of size n**
- ▶ Under-parameterized: $EMC_{\mathcal{D},\epsilon}(\mathcal{T})$ smaller than n , i.e. $\mathcal{T}$ achieves 0 error only on training sets of size smaller than n , increasing EMC will decrease the **test** error
- ▶ Over-parameterized: $EMC_{\mathcal{D},\epsilon}(\mathcal{T})$ larger than n , increasing EMC will decrease the **test** error
- ▶ Critical: $EMC_{\mathcal{D},\epsilon}(\mathcal{T})$ around n , increasing EMC may decrease or increase the **test** error (see figure)

Generalization in modern Deep Learning - Double Descent Intuition (Nakkiran 2020)

- ▶ Different settings for characterizing the double-descent phenomenon
 - ▶ i.e. the phenomenon appears under each setting and not only under the Model-wise setting characterized by Belkin et al.
 - ▶ Model-wise
 - ▶ Fixed large number of training steps, models of increasing size,
 - ▶ Epoch-wise
 - ▶ Fixed large architecture, increase the number of training epochs
 - ▶ Sample-wise
 - ▶ Fixed model and training procedure, change the number of training samples



Deep learning

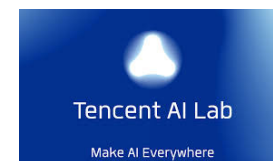


Interlude: new actors – new practices

- ▶ GAFA (Google, Apple, Facebook, Amazon) , BAT (Baidu, Tencent, Alibaba), ..., Startups, are shaping the data world
- ▶ Research
 - ▶ Big Tech. actors are leading the research in DL
 - ▶ Large research groups
 - ▶ Google Brain, Google Deep Mind, Facebook FAIR, Baidu AI lab, Baidu Institute of Deep Learning, etc
 - ▶ Standard development platforms, dedicated hardware, etc
 - ▶ DL research requires access to resources
 - ▶ sophisticated libraries
 - ▶ large computing power e.g. GPU clusters
 - ▶ large datasets, ...

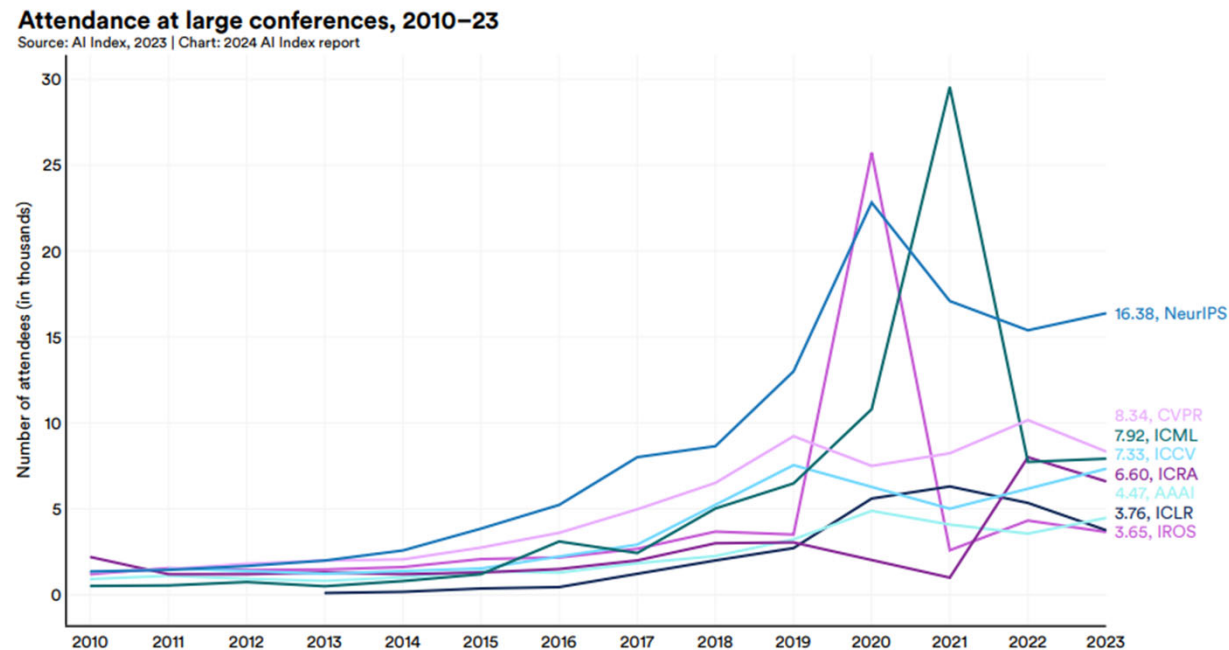


Facebook AI
Research



Interlude – ML conference attendance growth

► ML and AI conference Attendance



► NeurIPS - Breakpoints

- 2017 sold out 1 week after registration opening, 7000 participants
- 2018, 2k inscriptions sold in 11 mn!

Interlude – Deep Learning platforms

- ▶ Deep Learning platforms offer
 - ▶ Classical DL models
 - ▶ Optimization algorithms
 - ▶ Automatic differentiation
 - ▶ Popular options/ tricks
 - ▶ Pretrained models
 - ▶ CUDA/ GPU/ CLOUD support
 - ▶ Contributions by large open source communities: lots of code available
 - ▶ Easy to build/ train sophisticated models
- ▶ Among the most populars platforms:
- ▶ TensorFlow - Google Brain - Python, C/C++
 - ▶ PyTorch – Facebook- Python
 - ▶ JAX - Google
 - ▶ Higher order interfaces
 - ▶ e.g. Keras

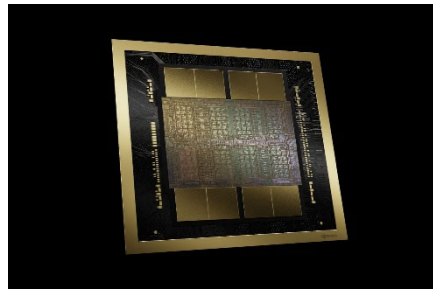


PYTORCH



Interlude – Hardware

- ▶ 2024 – NVIDIA B200 GPU

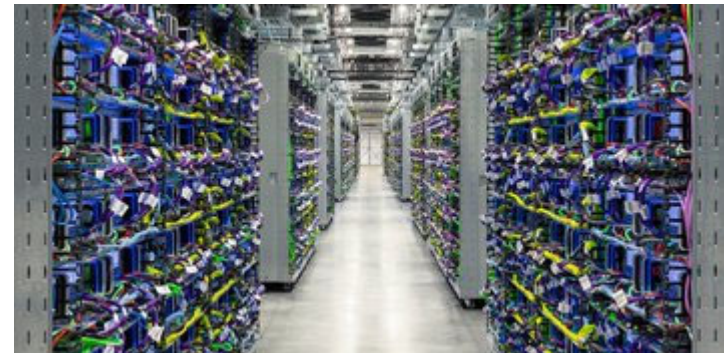


- ▶ From “The verge 2024-3-18
- ▶ Nvidia says the new B200 GPU offers up to 20 *petaflops* of FP4 horsepower from its 208 billion transistors. Also, it says, a GB200 that combines two of those GPUs with a single Grace CPU can offer 30 times the performance for LLM inference workloads while also potentially being substantially more efficient. It “reduces cost and energy consumption by up to 25x” over an H100, says Nvidia, though there’s a questionmark around cost — [Nvidia’s CEO has suggested](#) each GPU might cost between \$30,000 and \$40,000.

- ▶ Google Tensor Processor Unit – TPU V6 - 2024



- ▶ Cloud TPU



Useful Deep Learning heuristics

Deep NN make use of several (essential) heuristics for training large architecture: type of units, normalization, optimization...

We introduce some of these ideas

Deep Learning heuristics -Activation functions

Figures from:

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/tutorial3/Activation_Functions.html

- ▶ In addition to the logistic or tanh units,, other forms are used in deep architectures – Some of the popular forms are:

- ▶ Let $z = b + \mathbf{w} \cdot \mathbf{x}$

- ▶ RELU - Rectified linear units (used for internal layers)

- $g(\mathbf{z}) = \max(0, \mathbf{z})$
 - Rectified units allow to draw activations to 0 (used for sparse representations) + derivative remain large when unit is active

- ▶ Leaky RELU (used for internal layers)

- $g(\mathbf{z}) = \begin{cases} \mathbf{z} & \text{if } b + \mathbf{w} \cdot \mathbf{x} > 0 \\ 0.01(\mathbf{z}) & \text{otherwise} \end{cases}$
 - Introduces a small derivative when $b + \mathbf{w} \cdot \mathbf{x} < 0$

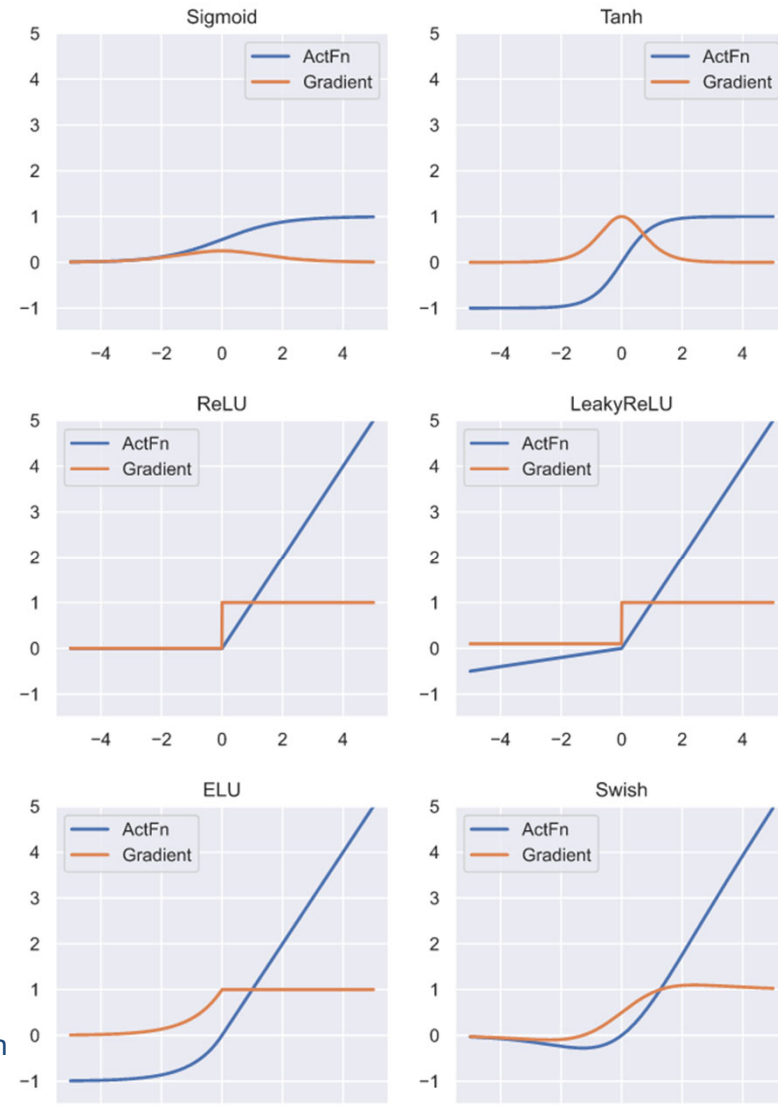
- ▶ ELU (used for internal layers)

- $g(\mathbf{z}) = \begin{cases} \mathbf{z} & \text{if } \mathbf{z} > 0 \\ \alpha(\exp(b + \mathbf{w} \cdot \mathbf{x}) - 1) & \text{otherwise} \end{cases}$

- ▶ Swish

- $g(\mathbf{z}) = \frac{\mathbf{z}}{1 + \exp(-\mathbf{z})}$

x axis $b + \mathbf{w} \cdot \mathbf{x}$, y axis $g(\mathbf{x})$

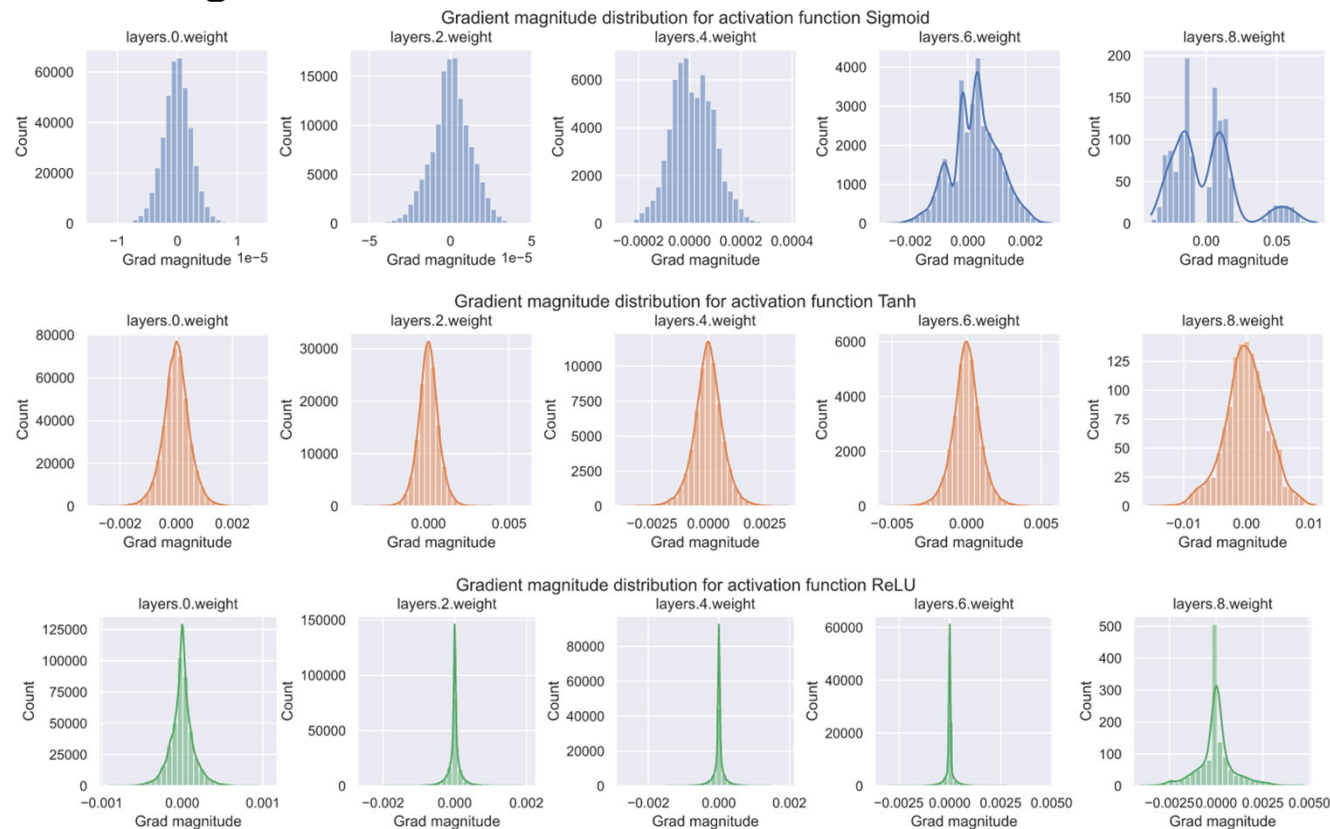


Deep Learning heuristics -Activation functions

Figures from:

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/tutorial3/Activation_Functions.html

- Visualisation of the gradient at different layers of a NN after initialisation of the weights
- Dataset : FashionMNIST (images) 10 classes, gradient computed on a batch of 256 images



Deep Learning heuristics - Activation functions

- ▶ In addition to the logistic or tanh units, other forms are used in deep architectures – Some of the popular forms are:

- ▶ **Maxout**

- ☐ $g(\mathbf{x}) = \max_i (b_i + \mathbf{w}_i \cdot \mathbf{x})$
- ☐ Generalizes the rectified unit
- ☐ There are multiple weight vectors for each unit

- ▶ **Softmax (used for output layer)**

- ▶ Used for classification with a 1 out of p coding (p classes)
 - ☐ Ensures that the sum of predicted outputs sums to 1
 - ☐ $g(\mathbf{x}) = \text{softmax}(\mathbf{b} + W\mathbf{x}) = \frac{e^{b_i + (W\mathbf{x})_i}}{\sum_{j=1}^p e^{b_j + (W\mathbf{x})_j}}$

Deep Learning heuristics

Normalisation

► Units: Batch Normalization (Ioffe 2015)

- Normalize the activations of the units (hidden units) so as to coordinate the gradients across layers
- Let $B = \{x^1, \dots, x^N\}$ be a mini batch, $h_i(x^j)$ the activation of hidden unit i for input x^j before non linearity
- Training
 - Set $h'_i(x^j) = \frac{h_i(x^j) - \mu_i}{\sigma_i + \epsilon}$ where μ_i is the mean of the activities of hidden unit i on batch B , and σ_i its standard deviation
 - μ_i and σ_i are estimated on batch B , ϵ is a small positive number
 - The output of unit i is then $z_i = \gamma_i h'_i(x^j) + \beta_i$
 - Where γ and β are learned via SGD
- Testing
 - μ_i and σ_i for test are estimated as a moving average during training, and need not be recomputed on the whole training dataset

Deep Learning heuristics

Normalization

► Note on B.N.

- No clear agreement if BN should be performed before or after non linearity
- L^2 normalization could be used together with BN but reduced
- One of the most effective tricks for learning with deep NNs
- Other types of normalization have been proposed e.g. Layerwise Normalization similar to BN, but layerwise and datum wise, etc.

► Gradient/ gradient clipping

- Avoid very large gradient steps when the gradient becomes very large - different strategies work similarly in practice.
- Let $\nabla_w c$ be the gradient computed over a minibatch
- A possible clipping strategy is (Pascanu 2013)
 - $\nabla_w c = \frac{\nabla_w c}{\|\nabla_w c\|} v$, with v a norm threshold

Deep Learning heuristics

Dropout

▶ Dropout (Srivastava 2014)

▶ Training

- Randomly drop units at training time
- Parameter: dropout percentage p
- Each unit is dropped with probability p

▶ This means that it is inactive in the forward and backward pass

▶ Testing

- Initial paper (Srivastava 2014)
 - Keep all the units
 - Multiply the units activation by p during test
- ▶ The expected output for a given layer during the test phase should be the same as during the training phase

Figure from Srivastava 2014

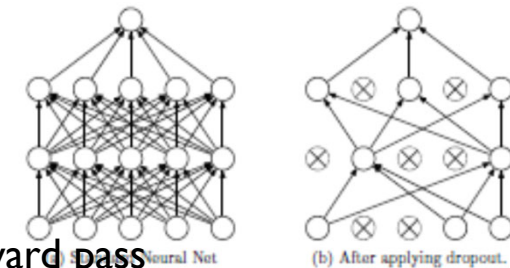


Figure 1: Dropout Neural Net Model. Left: A standard neural net with 2 hidden layers. Right: An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Deep Learning heuristics

Dropout

▶ Inverted Dropout

- ▶ Current implementations use « inverted dropout » - easier implementation: the network does not change during the test phase (see next slide)
 - Units are dropped with probability p
 - Multiplies activations by $\frac{1}{1-p}$ during training, and keep the network untouched during testing

▶ Effects

- ▶ Increases independence between units and better distributes the representation
- ▶ Interpreted as an ensemble model; reduces model variance

Deep Learning heuristics

Dropout

▶ Dropout for a single unit

- ▶ Let p be the dropout probability
- ▶ Consider a neuron i with inputs $\mathbf{x} \in R^n$ and weight vector $\mathbf{w} \in R^n$ including the bias term
- ▶ The activation of neuron i is $z_i = f(\mathbf{w} \cdot \mathbf{x})$ with f a non linear function (e.g. Relu)
- ▶ Let b_i a binomial variable of parameter $1 - p$

▶ Original dropout

- ▶ Training phase
 - $z_i = b_i f(\mathbf{w} \cdot \mathbf{x}), b_i \in \{0,1\}$
- ▶ Test phase
 - $z_i = \frac{1}{1-p} f(\mathbf{w} \cdot \mathbf{x})$

▶ Inverted dropout

- ▶ Training phase
 - $z_i = \frac{1}{1-p} b_i f(\mathbf{w} \cdot \mathbf{x}), b_i \in \{0,1\}$
- ▶ Test phase
 - $z_i = f(\mathbf{w} \cdot \mathbf{x})$

▶ Note

- ▶ The total number of neurons dropped at each step is the sum of Bernoullis b_i , it follows a binomial distribution $B(m, p)$ where m is the number of neurons on the layer of neuron i .
- ▶ Its expectation is the $E[B(m, p)] = mp$
- ▶ L^2 normalization could be used together with dropout but reduced

The loss landscape of deep neural networks

from Li et al. 2018, <https://arxiv.org/pdf/1712.09913.pdf>

- ▶ Developed a method for visualizing the loss landscape that allows to compare different NNs
- ▶ Hints
 - ▶ Given θ^* a solution learned by a NN and δ, η two random vectors of the same size as θ^* , plus normalization heuristics on these vectors, plot the surface $f(\alpha, \beta) = L(\theta^* + \alpha\delta + \beta\eta)$
- ▶ Examples
 - ▶ Networks trained on CIFAR-10 (image dataset for classification)
- ▶ Some messages
 - ▶ NN depth has a dramatic effect on loss surface when no skip connection is used
 - ▶ Wide models tend to have smoother surfaces
 - ▶ Landscape geometry has a dramatic effect on generalization. Flat minimizers tend to have lower test errors

The loss landscape of deep neural networks

from Li et al. 2018, <https://arxiv.org/pdf/1712.09913.pdf>

► 3-D plots

► ResNet-56 without and with skip connections

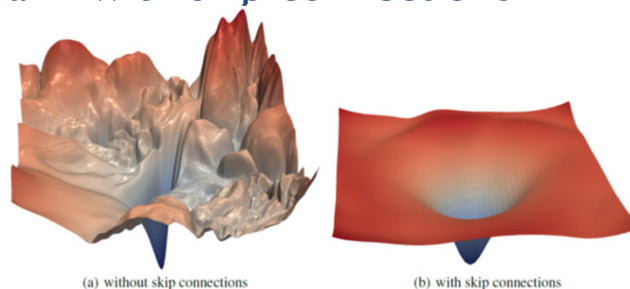


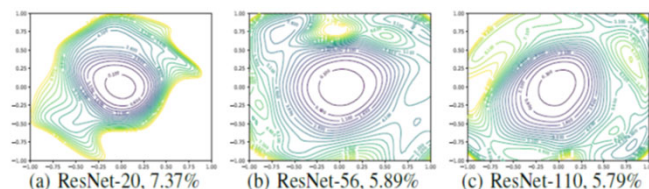
Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

► 2-D plots

► Resnets of different sizes (20, 56, 110 layers) without and with skip connections

► Centered on the learned min θ^*

Skip connections



No skip connections

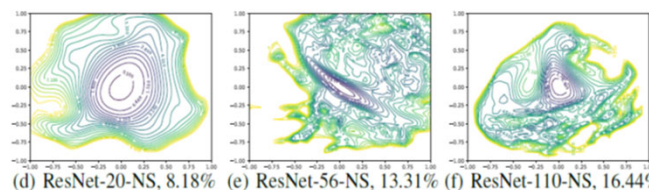


Figure 5: 2D visualization of the loss surface of ResNet and ResNet-noshort with different depth.

Convex landscape for small (20 layers) NNs and for Skip connections

Highly non convex landscape for noSkip NNs when size increases.

The loss landscape of deep neural networks

Mode connectivity (Garipov et al. 2018)

- ▶ **Mode connectivity** different local minima (modes) in the loss landscape are not isolated but can be connected by paths along which the loss remains relatively low.

If you have two trained models that correspond to different minima, mode connectivity suggests that there exists a continuous path in the parameter space linking these models without encountering significant increases in loss.

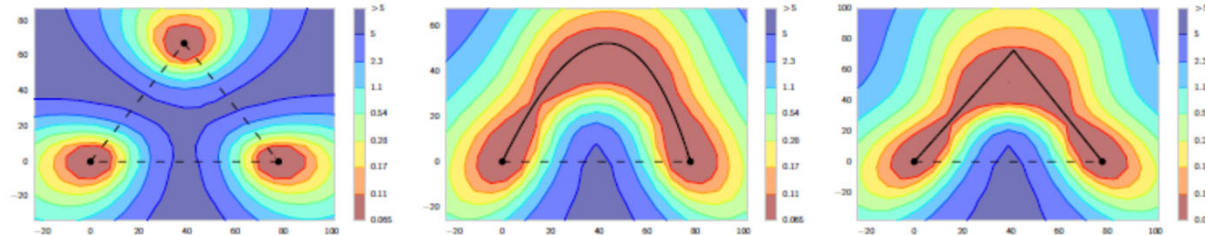


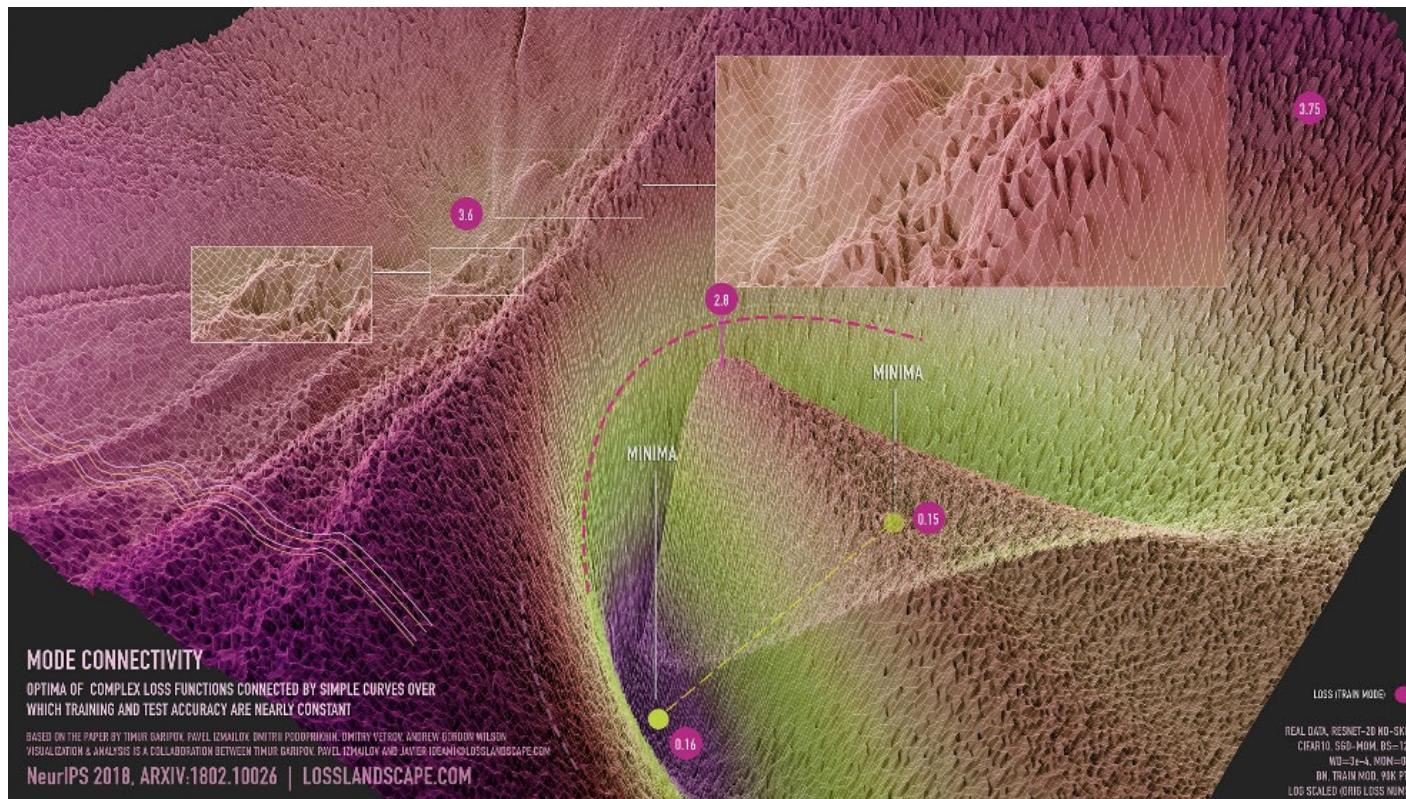
Figure 1: The ℓ_2 -regularized cross-entropy train loss surface of a ResNet-164 on CIFAR-100, as a function of network weights in a two-dimensional subspace. In each panel, the horizontal axis is fixed and is attached to the optima of two independently trained networks. The vertical axis changes between panels as we change planes (defined in the main text). **Left:** Three optima for independently trained networks. **Middle and Right:** A quadratic Bezier curve, and a polygonal chain with one bend, connecting the lower two optima on the left panel along a path of near-constant loss. Notice that in each panel a direct linear path between each mode would incur high loss.

Garipov et al. 2018

The loss landscape of deep neural networks

Mode connectivity (Garipov et al. 2018)

- ▶ **Low-Curvature Paths:** The connecting paths often traverse regions of the parameter space with low curvature, indicating stable and robust transitions between modes.



https://izmailovpavel.github.io/curves_blogpost/

Cf video illustrating the effect of learning on the loss landscape

The loss landscape of deep neural networks

Mode connectivity – implication for ensembles -

- ▶ If multiple minima are connected by such paths, it suggests that ensemble methods can benefit from the diversity of solutions.
 - ▶ This has been proposed e.g. with Stochastic Weight Averaging – SWA – Izmailov et al. 2019
- ▶ SWA
 - ▶ Start from a pretrained network
 - ▶ Continue training
 - ▶ Average the weights obtained after every epoch
 - ▶ You get an ensemble that performs as well as classical ensemble methods with a low inference cost: same complexity as a single NN

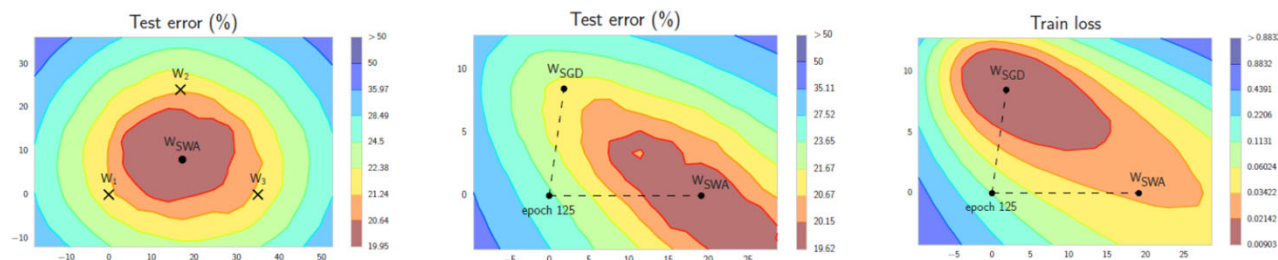


Figure 1: Illustrations of SWA and SGD with a Preactivation ResNet-164 on CIFAR-100¹. **Left:** test error surface for three FGE samples and the corresponding SWA solution (averaging in weight space). **Middle** and **Right:** test error and train loss surfaces showing the weights proposed by SGD (at convergence) and SWA, starting from the same

References

- ▶ Garipov, T., Izmailov, P., Podoprikin, D., Vetrov, D., & Wilson, A. G. (2018). Loss surfaces, mode connectivity, and fast ensembling of DNNs. *Advances in Neural Information Processing Systems*, 2018-December, 8789-8798.
- ▶ Izmailov, A. Podoprikin D., Garipov T., Vetrov D. , Gordon Wilson A., Averaging Weights Leads to Wider Optima and Better Generalization. UAI 2018: 876-885

Language models and transformers

Language models

► Objective:

- Probability models of sequences $(x^1, x^2, \dots, x^t)$
- Items may be words, characters, character ngrams, word pieces, etc
- Formally: given a sequence of items, what is the probability of the next item?
 - $p(x^t | x^{t-1}, \dots, x^1)$

► Example

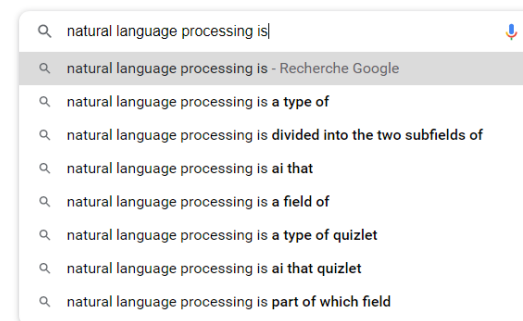
- « S'il vous plaît... dessine-moi ... » what next ?
- « $x^1 x^2 x^3 \dots \dots \dots x^{t-1} \dots$ » what is x^t ?



Google

► Language models in everyday use

- Sentence completion
 - Search engine queries
 - Smartphone messages, etc
- Speech recognition, handwriting recognition, etc



Language models

- ▶ Language models can be used to compute the probability of a piece of text
- ▶ Let $(x^1, x^2, \dots, x^T)$ be a sequence of text, its probability according to a language model is:
 - ▶ $p(x^1, x^2, \dots, x^T) = \prod_{t=1}^T p(x^t | x^{t-1}, \dots, x^1)$
 - ▶ With $p(x^t | x^{t-1}, \dots, x^1)$ computed by the language model

Language models

► Objective:

- Probability models of sequences $(x^1, x^2, \dots, x^t)$
- Items may be words, characters, character ngrams, word pieces, etc
- Formally: given a sequence of items, what is the probability of the next item?
 - $p(x^t | x^{t-1}, \dots, x^1)$

► Example

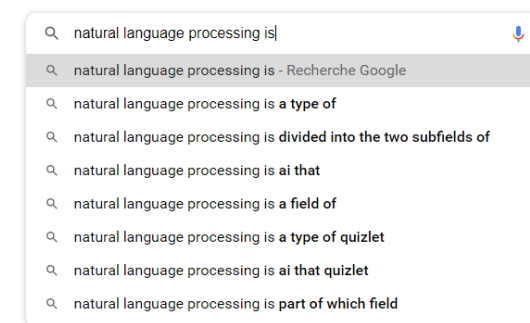
- « S'il vous plaît... dessine-moi ... » what next ?
- « $x^1 x^2 x^3 \dots \dots \dots x^{t-1} \dots$ » what is x^t ?



Google

► Language models in everyday use

- Sentence completion
 - Search engine queries
 - Smartphone messages, etc
- Speech recognition, handwriting recognition, etc



Language models

- ▶ Language models can be used to compute the probability of a piece of text
- ▶ Let $(x^1, x^2, \dots, x^T)$ be a sequence of text, its probability according to a language model is:
 - ▶ $p(x^1, x^2, \dots, x^T) = \prod_{t=1}^T p(x^t | x^{t-1}, \dots, x^1)$
 - ▶ With $p(x^t | x^{t-1}, \dots, x^1)$ computed by the language model

Language models

How to learn a language model - n-grams

▶ A simple solution: n-grams

- ▶ n-grams are sequences of n consecutive words (or characters, or any items)
- ▶ Language model is based on n-gram statistics
- ▶ Markov assumption

- ▶ x^t only depends on the $n - 1$ preceding words

- $p(x^t | x^{t-1}, \dots, x^1) = p(x^t | x^{t-1}, \dots, x^{t-n+1})$

n-gram probability

- ▶ Use Bayes formula $p(x^t | x^{t-1}, \dots, x^{t-n+1}) = \frac{p(x^t, x^{t-1}, \dots, x^{t-n+1})}{p(x^{t-1}, \dots, x^{t-n+1})}$

n-1-gram probability

- ▶ Given large text collections, it is possible to compute estimates of the posterior probabilities

- ▶ An estimate could be $\hat{p}(x^t | x^{t-1}, \dots, x^{t-n+1}) = \frac{\text{count}(x^t, x^{t-1}, \dots, x^{t-n+1})}{\text{count}(x^{t-1}, \dots, x^{t-n+1})}$

- ▶ Where $\text{count}(x^t, x^{t-1}, \dots, x^{t-n+1})$ is the number of occurrences of the sequence in the corpus

Language models

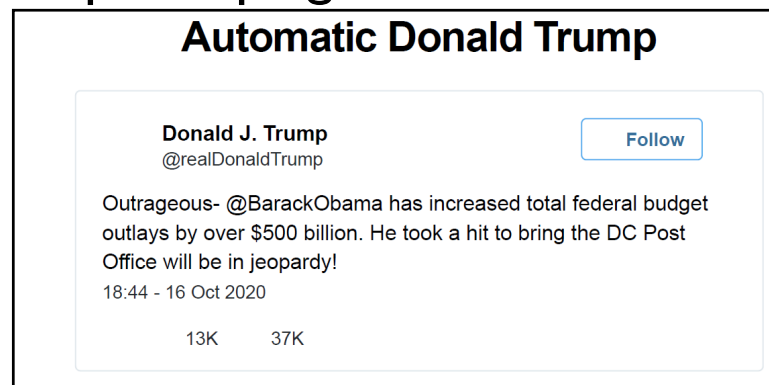
n-grams – text generation

- ▶ Example from <https://projects.haykranen.nl/markov/demo/>

- ▶ 4 gram trained on the Wikipedia article on Calvin and Hobbes
- ▶ Generated text

Rosalyn is a standary children used each otherwise as he stereotypically comic stand for an impulsive real-life Watterson's stuffed tiger, much as "grounded in reality rathmore spacious circle: because associety The club has said they have the archive shifting into low art some of the strip was one larger than Calvin anticipate indulges in his hands attribute red-and-black pants, magenta socks and Susie Derkins specifically characters like school where were printerestrainstory

- ▶ Example from <https://filiph.github.io/markov/>



Language models – Word representation

- ▶ Words, characters, n-grams, word pieces are all discrete data
- ▶ How to represent them
 - ▶ The usual way is to embed the words, etc in a continuous space of high dimension e.g. R^{200} , i.e. each word will be a vector in R^{200}
 - ▶ This could be done
 - ▶ Off line using some embedding technique (e.g. Word2Vec, see later)
 - Advantage, this can be done by using very large text collections
 - These representations could then be used for downstream tasks (e.g. classification)
 - ▶ On line while training the language model
 - In this case, the x s are initialized at random values in R^n and are learned by backpropagating the error, together with the other parameters
 - We usually lose the benefit of training on large corpora

Semantics: words

How to encode words according to their semantic meaning

▶ **Representing words as discrete symbols**

- ▶ In traditional NLP, we regard words as discrete symbols: Words can be represented by **one-hot vectors** - Each word is a distinct symbol
- ▶ **Example:** in web search, if user searches for “Seattle motel”, we would like to match documents containing “Seattle hotel”.
 - ▶ motel = [0 0 0 0 0 0 0 0 0 0 0 **1** 0 0 0 0]
 - ▶ hotel = [0 0 0 0 0 0 0 **1** 0 0 0 0 0 0 0]
 - These two vectors are orthogonal.
 - There is **no natural notion of similarity** for one-hot vectors!
- ▶ **Vector dimension** = number of words in vocabulary (e.g., 500,000)
 - ▶ Very large dimensional discrete space - Problem for machine learning - sparsity

Words in vector space

Representing words by their context

- ▶ **Distributional semantics:** A word's meaning is given by the words that frequently appear close-by
 - ▶ One of the most successful ideas of modern statistical NLP!
- ▶ When a word w appears in a text, its context is the set of words that appear nearby (within a fixed-size window).
 - ▶ Use the many contexts of w to build up a representation of w

...government debt problems turning into **banking** crises as happened in 2009...
...saying that Europe needs unified **banking** regulation to replace the hodgepodge...
...India has just given its **banking** system a shot in the arm...

context words will
represent ***banking***

Words in vector space

Representing words by their context

- ▶ Word embeddings

- ▶ We represent words w by vectors v_w so that words with similar contexts share « close » representations in the vector space

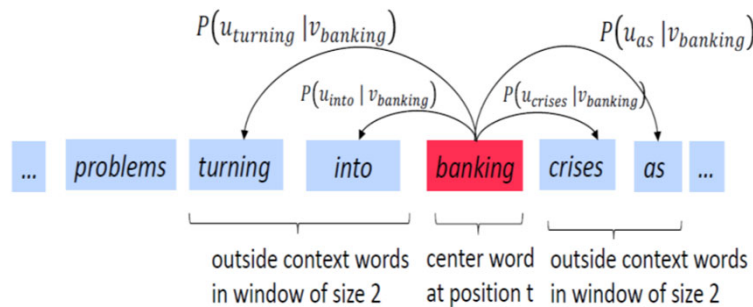
$$v_{\text{banking}} = \begin{bmatrix} 0.87 \\ 0.45 \\ -0.34 \\ -0.63 \\ 0.23 \\ 0.16 \end{bmatrix}$$

- ▶ Key idea

- ▶ These representations are learned from very large corpora for representing a large variety of situations/ contexts
 - ▶ No need for supervision
- ▶ These embeddings will be used for downstream tasks, e.g. classification
- ▶ This is an example of **self-supervised learning**

Word embeddings

Word2Vec – Mikolov et al. 2013



30

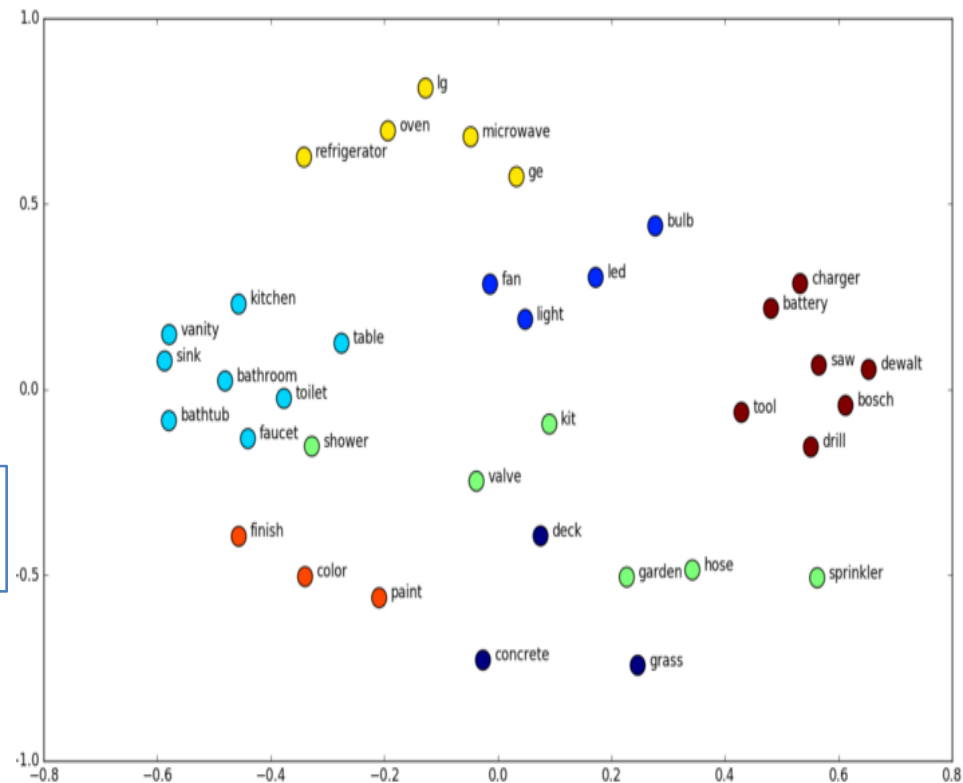
$$p(w_{out}|w_{in}) = \frac{\exp(v_{out} \cdot v_{in})}{\sum_{w \in \text{Vocabulary}} \exp(v_w \cdot v_{in})}$$

w_{out} : context word (into)

w_{in} : central word (banking)

v_{out} vector representation of w_o

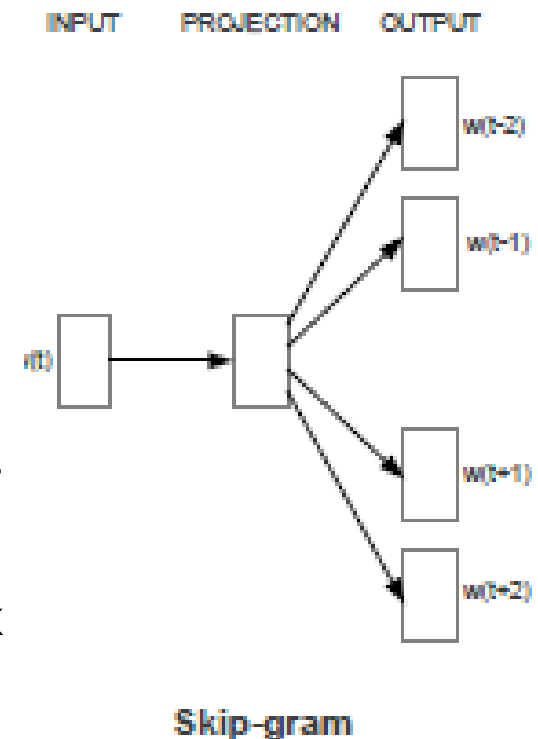
v_{in} vector representation of w_c



Word embeddings projections on 2D space:
words with similar contexts are close in the
embedding space

Learning word vector representations - Skip Gram model (Mikolov et al. 2013a, 2013b)

- ▶ Task
 - ▶ Predict the context words conditioned on the central word of a sequence
- ▶ Input v_{in} and output v_{out} word representations are learned jointly
 - ▶ (random initialization)
- ▶ The projection layer is linear followed by a sigmoid
- ▶ In the original skip gram model, input and outputs have different representations for the same word
- ▶ The output layer computes a hierarchical softmax
 - ▶ This allows computing the output in $O(\log_2(dictionary\ size))$ instead of $O(dictionary\ size)$



Learning word vector representations - Skip gram model (Mikolov et al. 2013a, 2013b)

► Average log probability

►
$$L = \frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log p(w_{t+j} | w_t)$$

- Where T is the number of words in the whole sequence used for training (roughly number of words in the corpus) and c is the context size

►
$$p(w_{out} | w_{in}) = \frac{\exp(\mathbf{v}_{out} \cdot \mathbf{v}_{in})}{\sum_{w=1}^V \exp(\mathbf{v}_w \cdot \mathbf{v}_{in})}$$

- Where $\mathbf{v}_w$ is the learned representation of the w vector (the hidden layer), $\mathbf{v}_{out} \cdot \mathbf{v}_{in}$ is a dot product and V is the vocabulary size

► Training algorithm

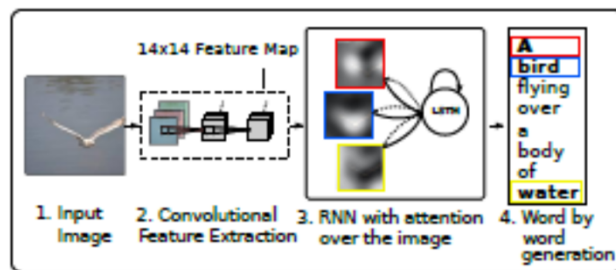
- Maximize L using e.g. SGD

Attention Mechanism

Initial historical developments and examples

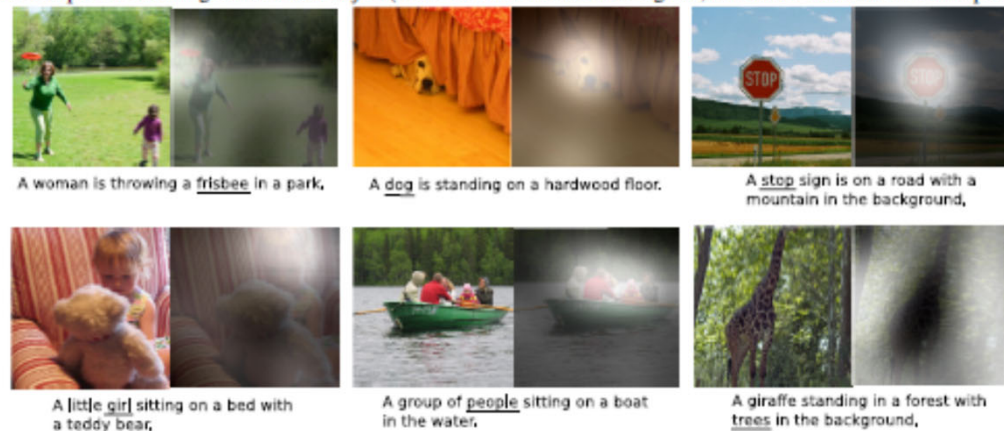
Attention mechanism

- ▶ Objective: focus on specific parts of the data representation for taking the current decision
 - ▶ Implemented as an additional differentiable modules in several architectures
- ▶ Illustration: attention on image while generating sentences

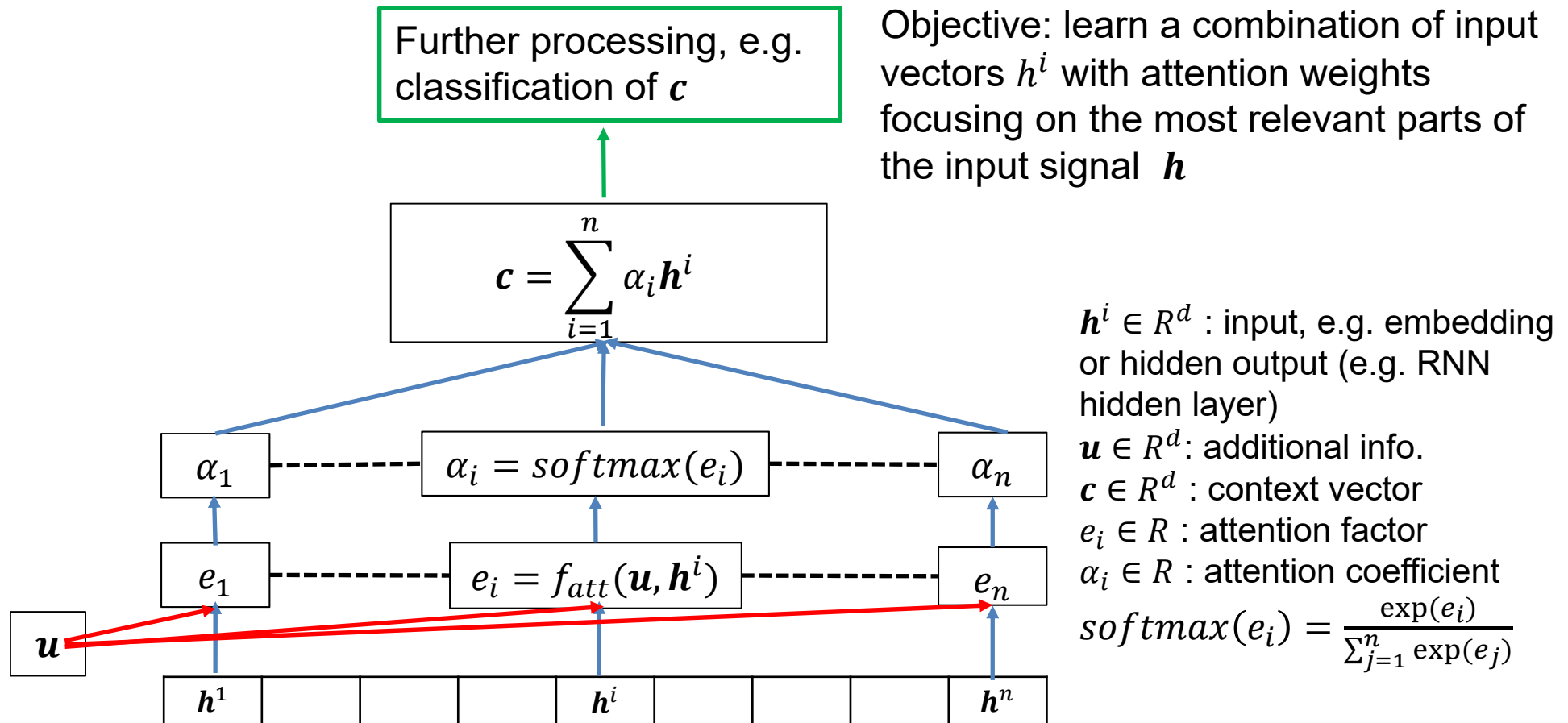


Figs. from Xu et al. 2015

Figure 4. Examples of attending to the correct object (white indicates the attended regions, underlines indicated the corresponding word)



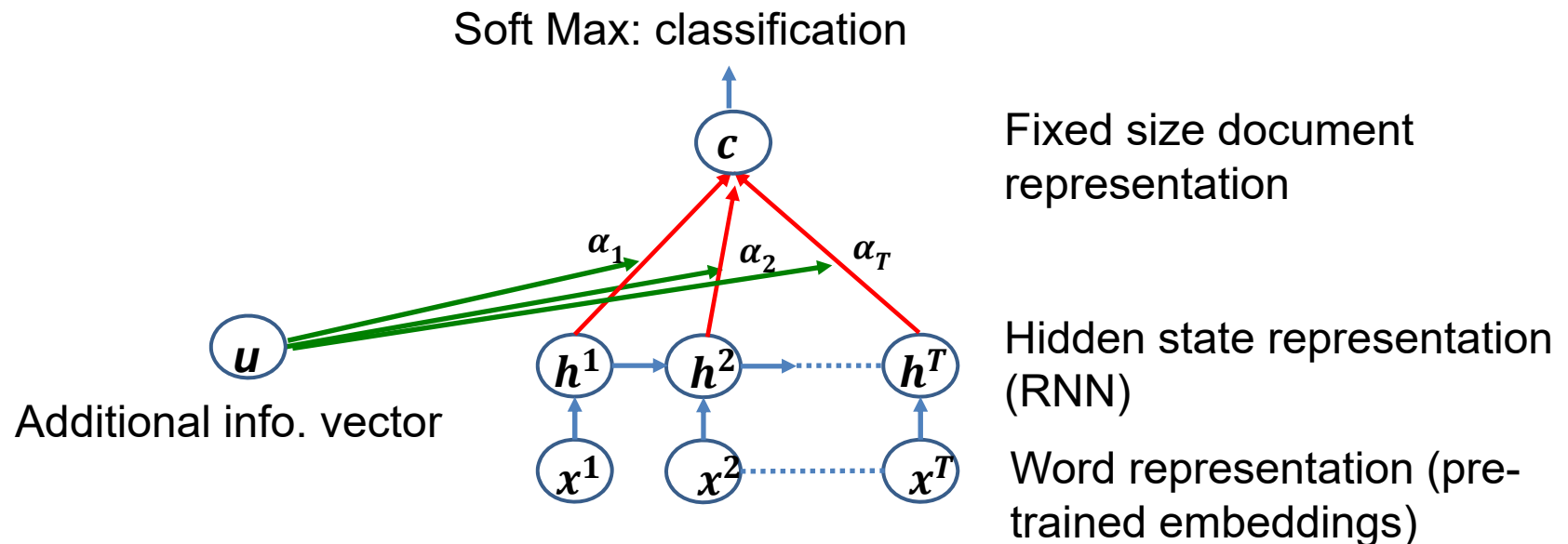
Attention mechanism



Attention mechanism

For document classification (adapted from Yang et al. 2016)

- ▶ Objective: classify documents using a sequential model of attention
 - ▶ Document : word sequence $w^1, \dots, w^T$
 - ▶ Objective: classify the document among predefined classes – learning criterion: log likelihood
 - ▶ Word sequence encodings (e.g. pretrained via Word2Vec): $x^1, \dots, x^T$
 - ▶ Corresponding hidden state sequence: $h^1, \dots, h^T$ obtained via a Recurrent NN



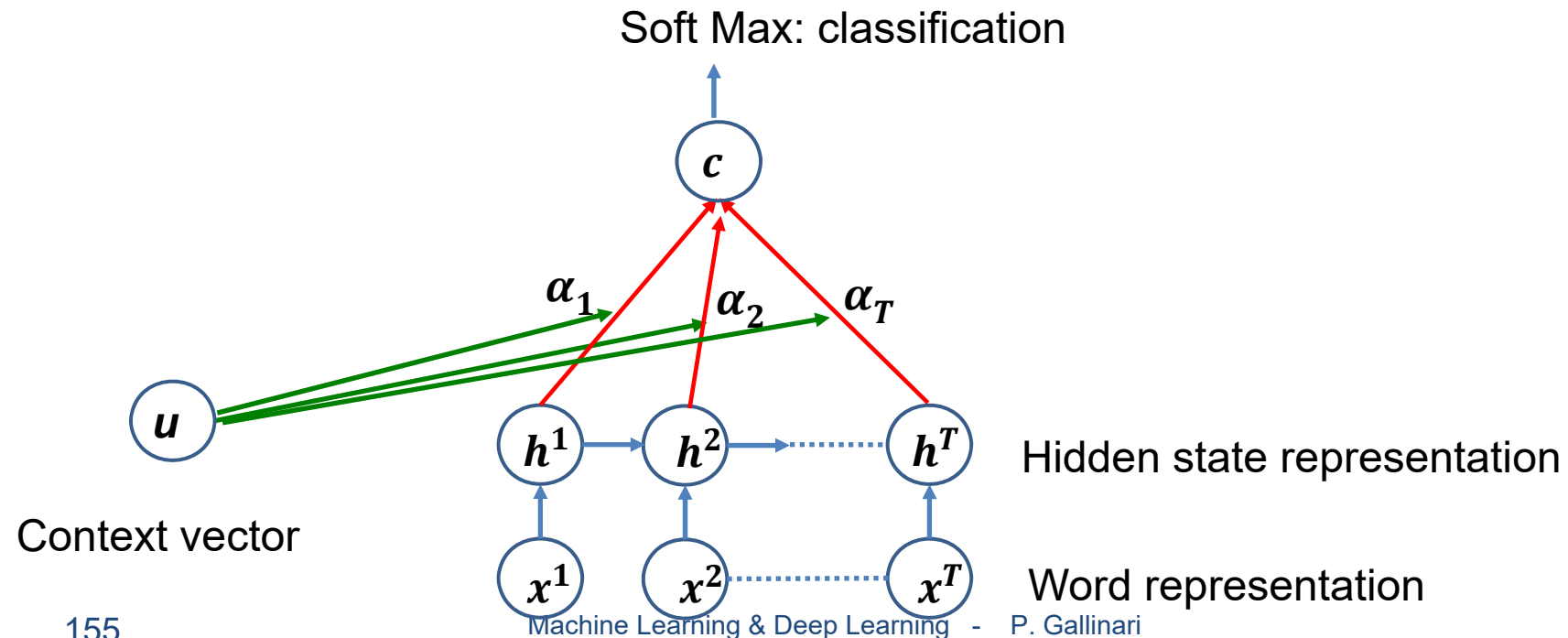
Attention mechanism

Example: document classification (adapted from Yang et al. 2016)

- ▶ $v_j = \tanh(W\mathbf{h}^j + \mathbf{b})$ (vector)
- ▶ $\alpha_j = \frac{\exp(v_j \cdot \mathbf{u})}{\sum_t \exp(v_t \cdot \mathbf{u})}$: attention weight (real value)
- ▶ $\mathbf{c} = \sum_{j=1}^T \alpha_j \mathbf{h}^j$: fixed size document representation (vector)
- ▶ $\mathbf{u}$: context vector to be learned (vector)

Parameters to be learned:

- Attention W, b, u
- Others: RNN, Softmax classifier



Attention mechanism

Example: document classification (adapted from Yang et al. 2016)

► Illustration (Yang et al. 2016)

- Yelp reviews: ratings from 1 to 5 (5 is the best)
- Classification = sentiment/ polarity classification
- Hierarchical attention: word and sentence levels
- Blue = word weight in the decision
- Red = sentence weight in the decision (hierarchical attention model – 2 levels: sentences and words within a sentence)

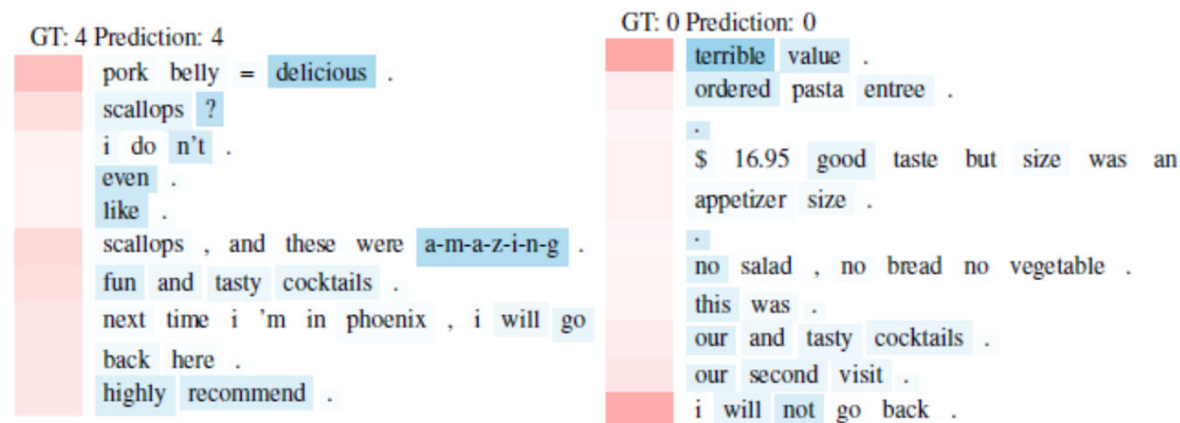


Figure 5: Documents from Yelp 2013. Label 4 means star 5, label 0 means star 1.

Transformer Networks

Initial paper: Vaswani 2017

Story Telling and Illustrations used in the slides:

J. Alammr 2018 - 2019 - <http://jalammar.github.io/illustrated-transformer/>

<http://jalammar.github.io/illustrated-gpt2/>

P. Bloem 2019 - <http://www.peterbloem.nl/blog/transformers>

Nice video on self-cross attention: <https://www.youtube.com/watch?v=aw3H-wPuRcw>

Transformer networks (Vaswani 2017, illustrations J. Alammr 2018-2019, P. Bloem 2019)

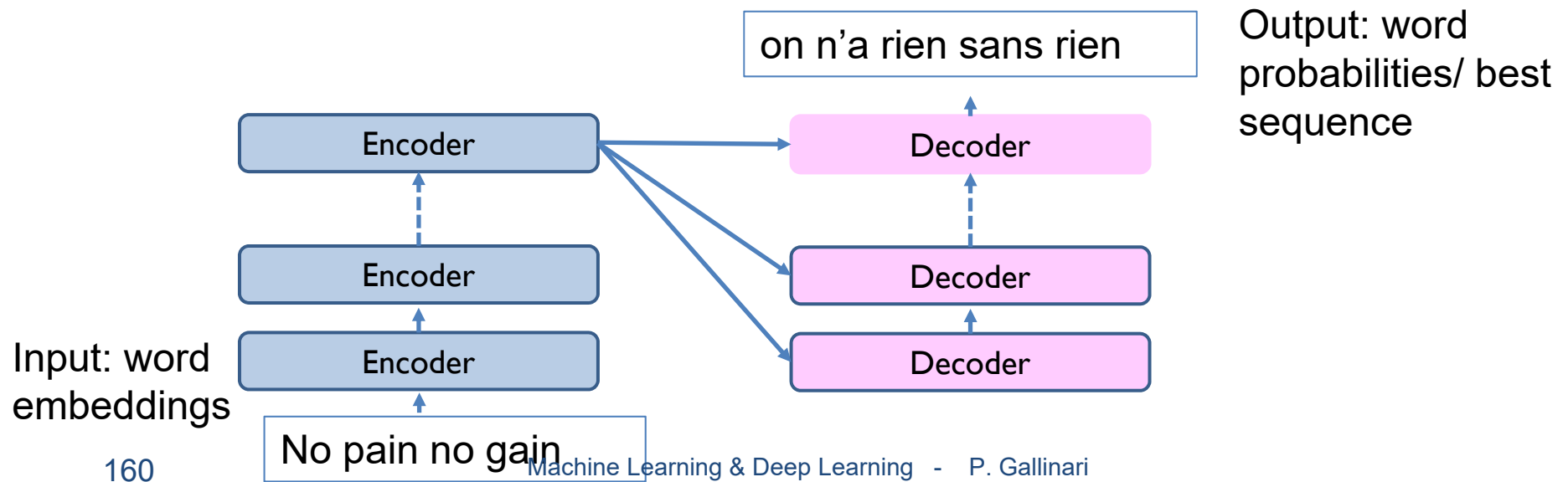
- ▶ Transformer networks were proposed in 2017
- ▶ They implement a self attention mechanism
- ▶ They became SOTA technology for many NLP problems
- ▶ Transformer blocks are now a basic component of the NN zoo
- ▶ They are key components for all the recent NLP transformer architectures
 - ▶ BERT family (Google), GPT family (OpenAI), T5 family (Google), etc
- ▶ After NLP they have been adapted by the Vision community and then by many others including weather prediction, biology, etc

Transformer networks (Vaswani 2017, illustrations J. Alammr 2018-2019, P. Bloem 2019)

- ▶ **Scalability**
 - ▶ Transformers open the way to the deployment of large scale architectures
- ▶ **Self-supervised**
 - ▶ They are usually trained in a self-supervised way using unlabeled data
- ▶ **Downstream tasks**
 - ▶ The learned representations are used for solving downstream tasks
 - ▶ Often relies on fine-tuning and/ or few-shot learning
 - ▶ This is the basis of **Foundation models**
- ▶ **Self-attention, Cross-attention**
 - ▶ Are the two fundamental mechanisms used in Transformers

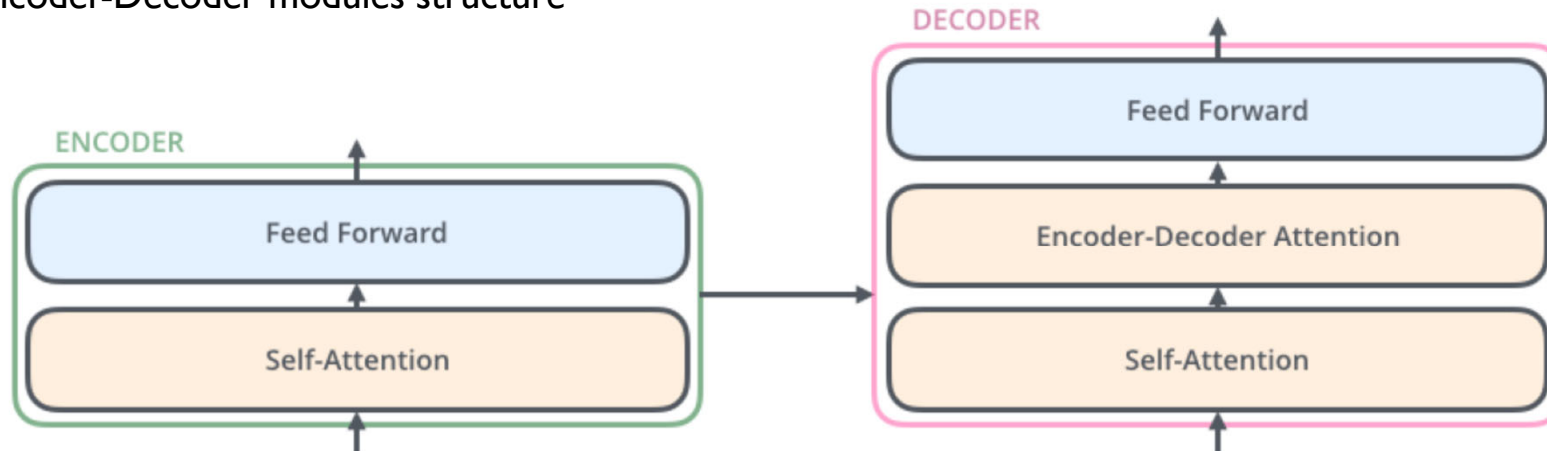
Historical side: Transformer networks (Vaswani 2017)

- ▶ The first implementation of Transformer was proposed by (Vaswani 2017) as an encoder-decoder scheme
- ▶ General scheme
 - ▶ Stacks of encoder/ decoder modules
 - ▶ Encoders (resp. decoders) have the same structure but do not share parameters



Historical side: Transformer networks (Vaswani 2017, illustrations J. Alammr 2018)

▶ Encoder-Decoder modules structure



▶ Encoder

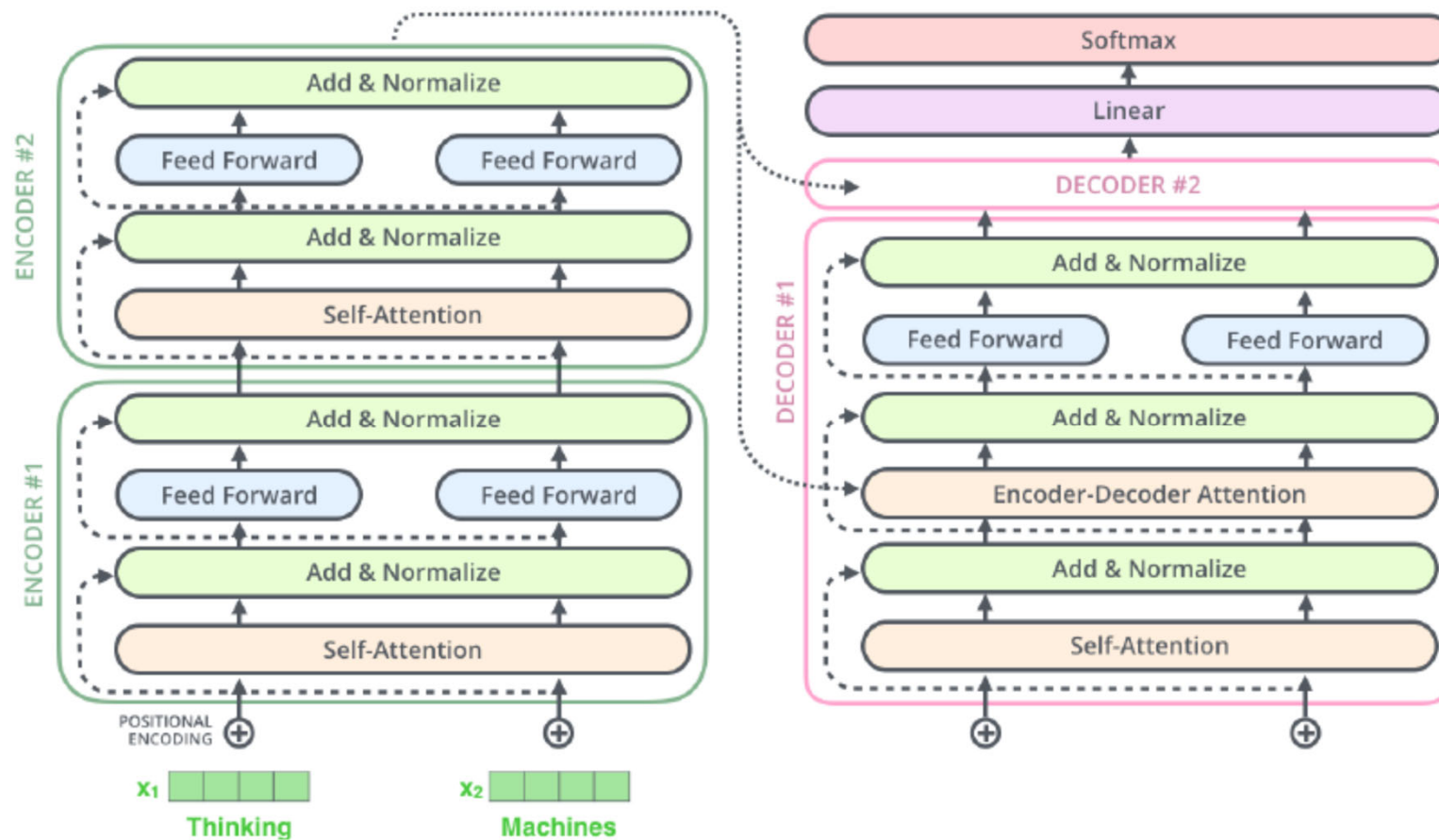
- ▶ Input flows through a self attention layer – encoding of a word in the sequence will depend on the other words
- ▶ Outputs of the self attention layer are fed in a feed-forward NN. The same network is used for each word position

▶ Decoder: 2 differences with the encoder

- ▶ 1. The decoder has an additional encoder-decoder attention layer that focuses on relevant parts of the input provided by the encoder (when the self attention module below it looks at the info from the lower layer of the decoder).
- ▶ 2. For the self attention module, the decoder can only look at past information to predict the next word – this is similar to the autoregressive example seen before

Historical side: Transformer networks (Vaswani 2017)
illustration: J. Alammari 2018

► Encoder + Decoder modules



Transformer networks (Vaswani 2017, illustrations J. Alammr 2018-2019, P. Bloem 2019)

Self Attention

- ▶ Self Attention is the fundamental operation of transformers
 - ▶ **Self attention is a sequence to sequence operation**
 - ▶ **Input and output sequences have the same length**
 - ▶ Let $x_1, x_2, \dots, x_T$ and $z_1, z_2, \dots, z_T$ be respectively the input and output vector sequence
 - ▶ Self attention computes the output sequence as:
 - ▶ $z_i = \sum_j \alpha_{ij} x_j$
 - ▶ With α_{ij} a normalized attention score
 - ▶ A simple version of the normalized score could be:
 - $e_{ij} = x_i \cdot x_j$
 - $\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp e_{ij}}{\sum_k \exp e_{ik}}$
 - ▶ α_{ij} measures how x_i **and** x_j are important for predicting z_i

Transformer networks (Vaswani 2017, illustrations J. Alammari 2018, P. Bloem 2019)
Self Attention

- ▶ Self Attention is the fundamental operation of transformers

For $i = \text{"Self" to "transformer"}: z_i = \sum_{j=\text{self} \dots \text{transformer}} \alpha_{ij} x_j$

Output
sequence

$z_{\text{self}} z_{\text{Attention}} z_{\text{is}} z_{\text{the}} z_{\text{fundamental}} z_{\text{operation}} z_{\text{of}} z_{\text{transformer}}$

Learned
embeddings

$x_{\text{self}} x_{\text{Attention}} x_{\text{is}} x_{\text{the}} x_{\text{fundamental}} x_{\text{operation}} x_{\text{of}} x_{\text{transformer}}$

Input: word
sequence

Self Attention is the fundamental operation of transformers



self attention



embedding

Transformer networks (Vaswani 2017, illustrations J. Alammari 2018, P. Bloem 2019)

Self Attention

- ▶ Self attention is the only mechanism in the transformer that propagates information **between** vectors
 - ▶ Any other operation is applied to each vector without interaction between vectors
 - ▶ In the above example $z_{fundamental}$ is a weighted sum over all embedding vectors x weighted by their normalized dot product with the embedding $x_{fundamental}$
 - ▶ The dot product expresses how related two words in the input sequence are, w.r.t. the learning task
- ▶ **Note**
- ▶ Self Attention sees the input as a set and not as a sequence
 - ▶ Permutation in the inputs simply results in a permutation of the outputs
 - ▶ An additional mechanism should be used in order to consider the sequence information (more on that later)

Transformer networks (Vaswani 2017, illustrations J. Alammari 2018, P. Bloem 2019) - Self Attention – Queries, keys, values

- ▶ Current transformers make use of a more complex self attention mechanism
- ▶ 1. For each embedding x_i create 3 vectors as a linear transformation of x_i : query, key, value

- ▶ query: $q_i = x_i W_q$

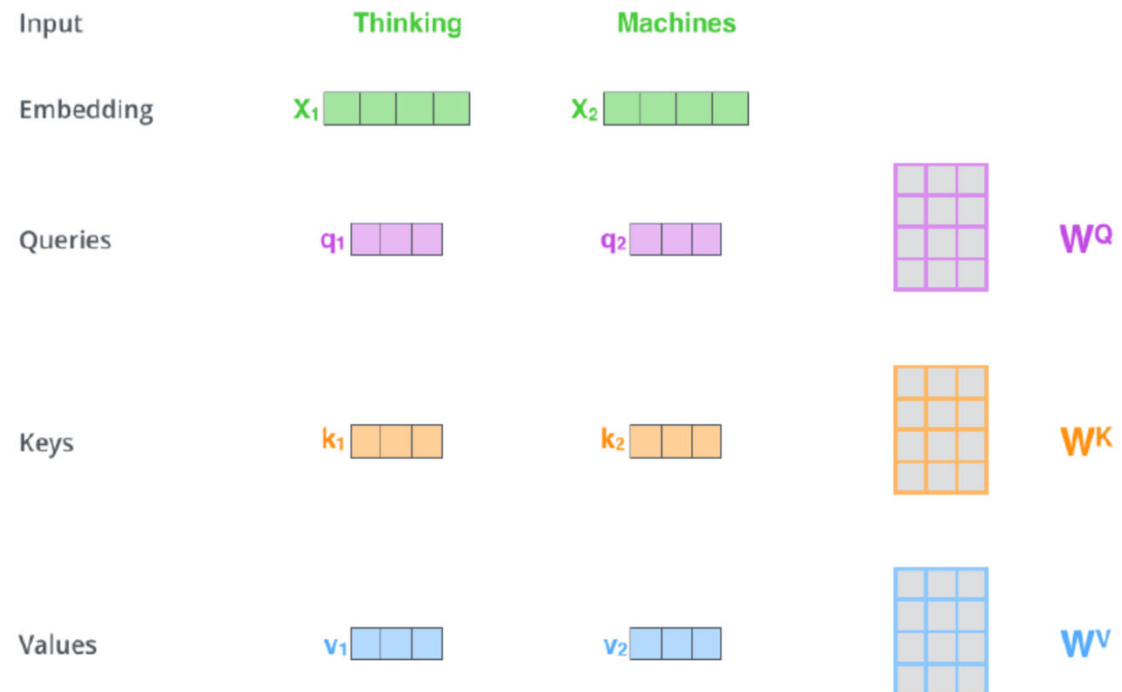
- ▶ key: $k_i = x_i W_k$

- ▶ value: $v_i = x_i W_v$

- ▶ With W_q, W_k, W_v

Matrices of the appropriate dimension

- ▶ Here x_i is a row vector



Multiplying x_1 by the W_q weight matrix produces q_1 , the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.

Transformer networks (Vaswani 2017, illustrations J. Alammari 2018, P. Bloem 2019)

Self Attention – Queries, keys, values

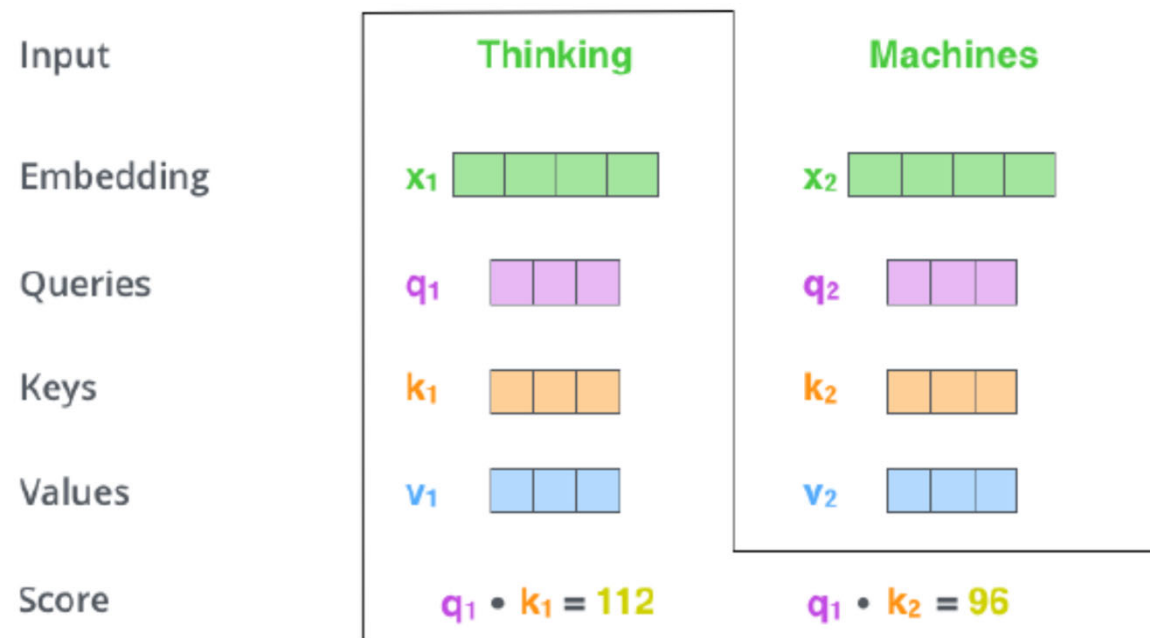
- ▶ x_i is used for three roles:
 - ▶ Query q_i : it is compared to every vector x_j to establish the weights for its **own output vector z_i**
 - ▶ Key k_i : it is compared to every vector x_j to establish the weights for the **output z_j**
 - ▶ Value v_i : it is used in the weighted sum to compute **each output vector z_j**
- ▶ Separating the roles in three vectors q_i, k_i, v_i , all linear transformations of x_i gives a more flexible model
- ▶ Illustration for computing the output vector z_i
 - ▶ **q_i and k_j will be used for computing the attention score:**
 - ▶ $e_{ij} = q_i \cdot k_j$
 - ▶ $\alpha_{ij} = \text{softmax}(e_{ij})$
 - ▶ **v_j will be used for computing the output item**
 - ▶ $z_i = \sum_j \alpha_{ij} v_j = \sum_j \text{softmax}(q_i \cdot k_j) v_j$

Transformer networks (Vaswani 2017, illustrations J. Alammari 2018, P. Bloem 2019) - – Queries, keys, values

► 2. Compute score from query and key

► Dot product of query and key value for each word

- Consider the sentence « Thinking Machines »
- $e_{ij} = q_i \cdot k_j$ - here we consider the first word **Thinking** (i.e. $i = 1, j = 1, 2$ since we have 2 words in the sentence)



Transformer networks (Vaswani 2017, illustrations J. Alammari 2018, P. Bloem 2019) - – Queries, keys, values

▶ 3. Normalize and softmax

- ▶ Divide by the square root of the dimension of the key vectors (8 in the figure)

- ▶ $e_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$, with d_k the dimension of the q, k, v vectors

- ▶ Compute softmax

- ▶ $\alpha_{ij} = \text{softmax}(e_{ij})$

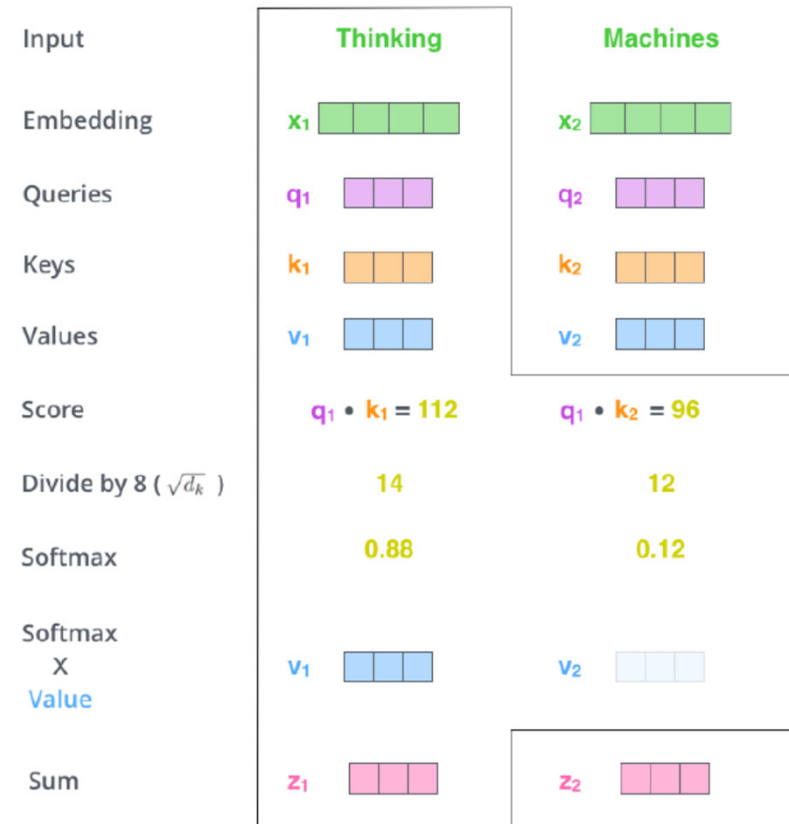
- ▶ The softmax value indicates the weight of each word in the input sequence for position 1 in the example

Input	Thinking	Machines
Embedding	x_1	x_2
Queries	q_1	q_2
Keys	k_1	k_2
Values	v_1	v_2
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Transformer networks (Vaswani 2017, illustrations J. Alammari 2018, P. Bloem 2019) - – Queries, keys, values

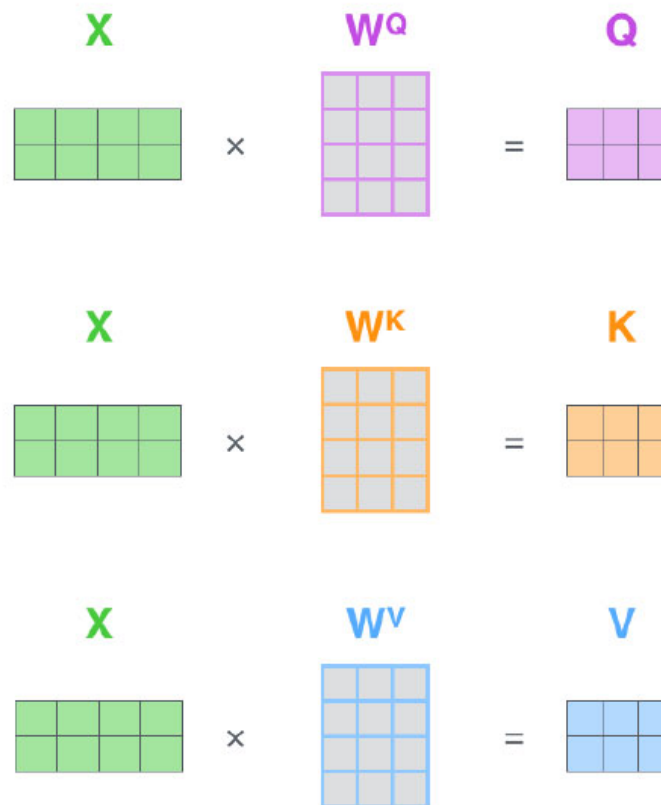
► 4. Compute the output of the self attention layer at position 1, i.e. (z_1)

- Multiply each value vector v by the softmax score
- Sum up the weighted value vectors
- $z_i = \sum_j \alpha_{ij} v_j$



Transformer networks (Vaswani 2017, illustrations J. Alammam 2018, P. Bloem 2019) – Queries, keys, values

- In matrix form for our 2 words sentence



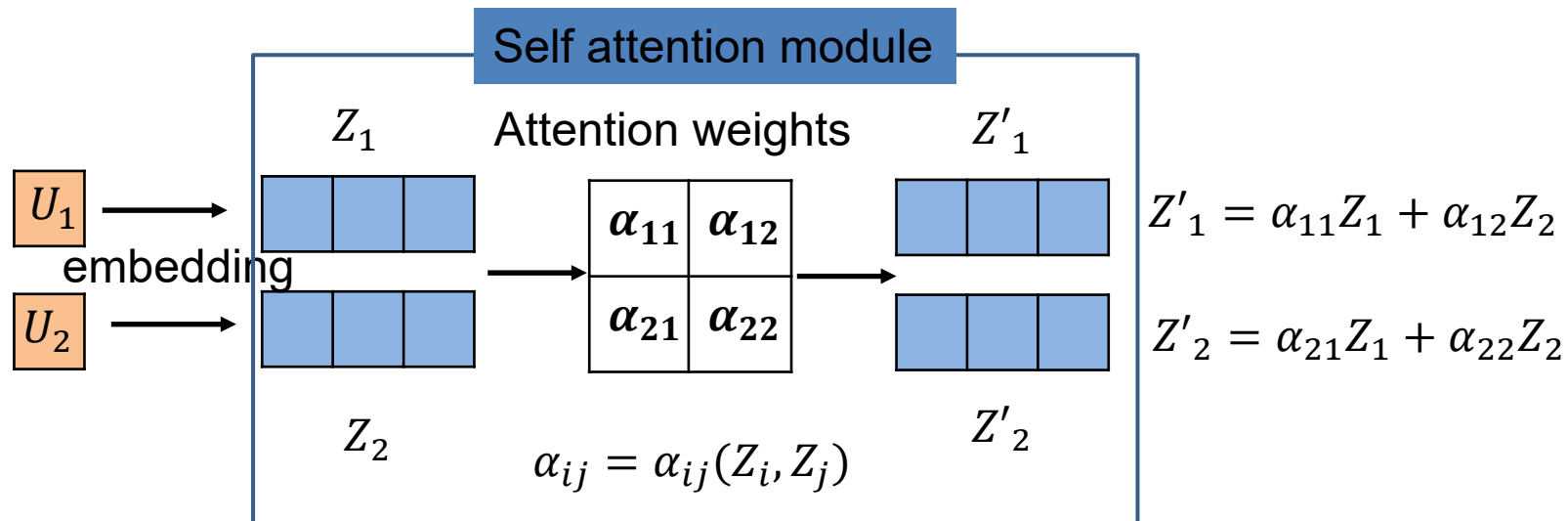
Transformer networks (Vaswani 2017, illustrations J. Alammari 2018, P. Bloem 2019) – Queries, keys, values

- ▶ Compute the output of the self attention layer at position 1
 - ▶ Matrix form

$$\text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \text{purple} & \text{purple} & \text{purple} \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \text{orange} & \text{orange} \\ \hline \end{array} \end{matrix} \right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \text{blue} & \text{blue} & \text{blue} \\ \hline \end{array} \end{matrix}$$
$$= \begin{matrix} \text{Z} \\ \begin{array}{|c|c|c|} \hline \text{pink} & \text{pink} & \text{pink} \\ \hline \end{array} \end{matrix}$$

The self-attention calculation in matrix form

Summary - Attention in transformers - Self attention

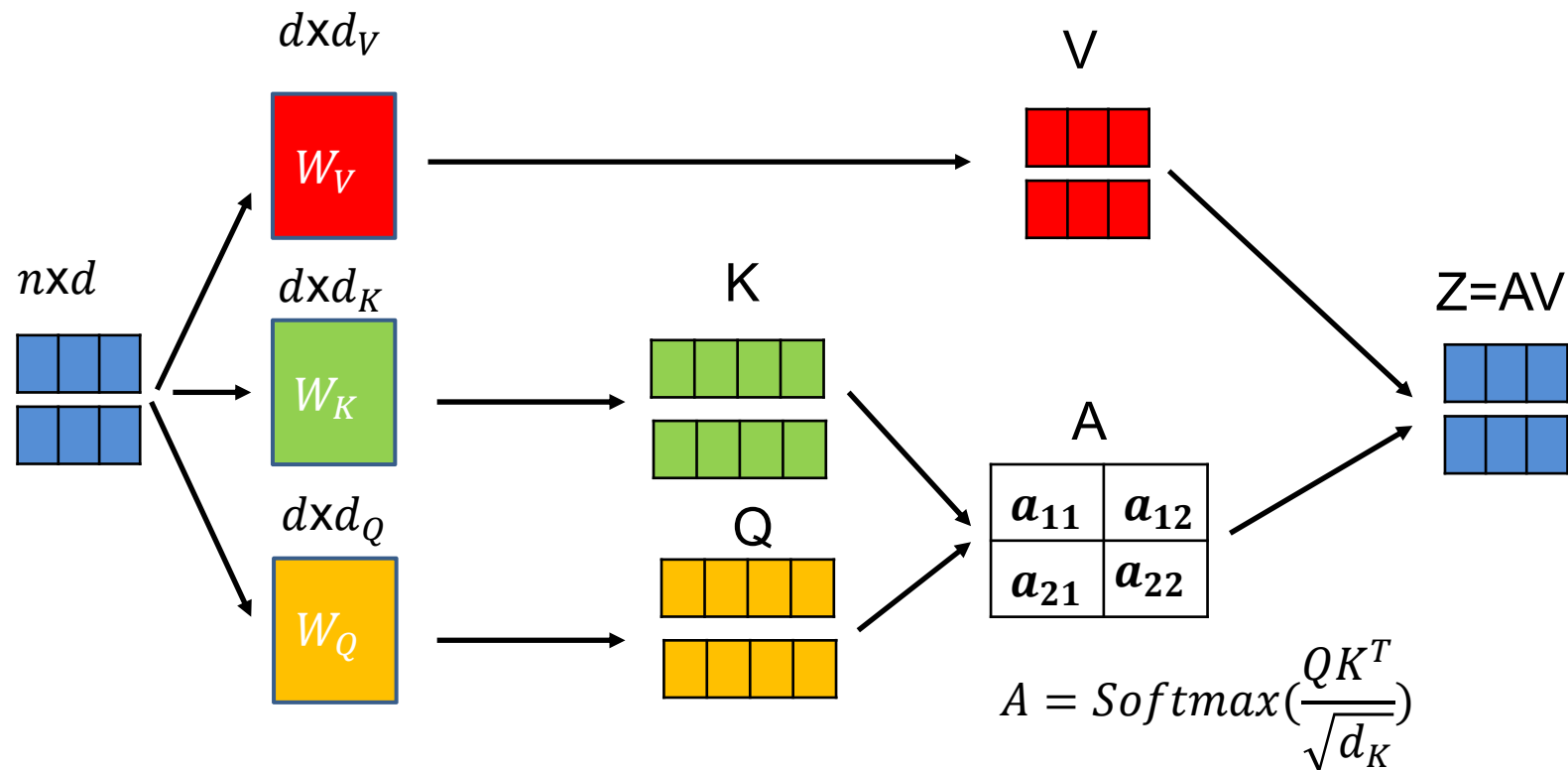


Captures **contextual representation** of the inputs

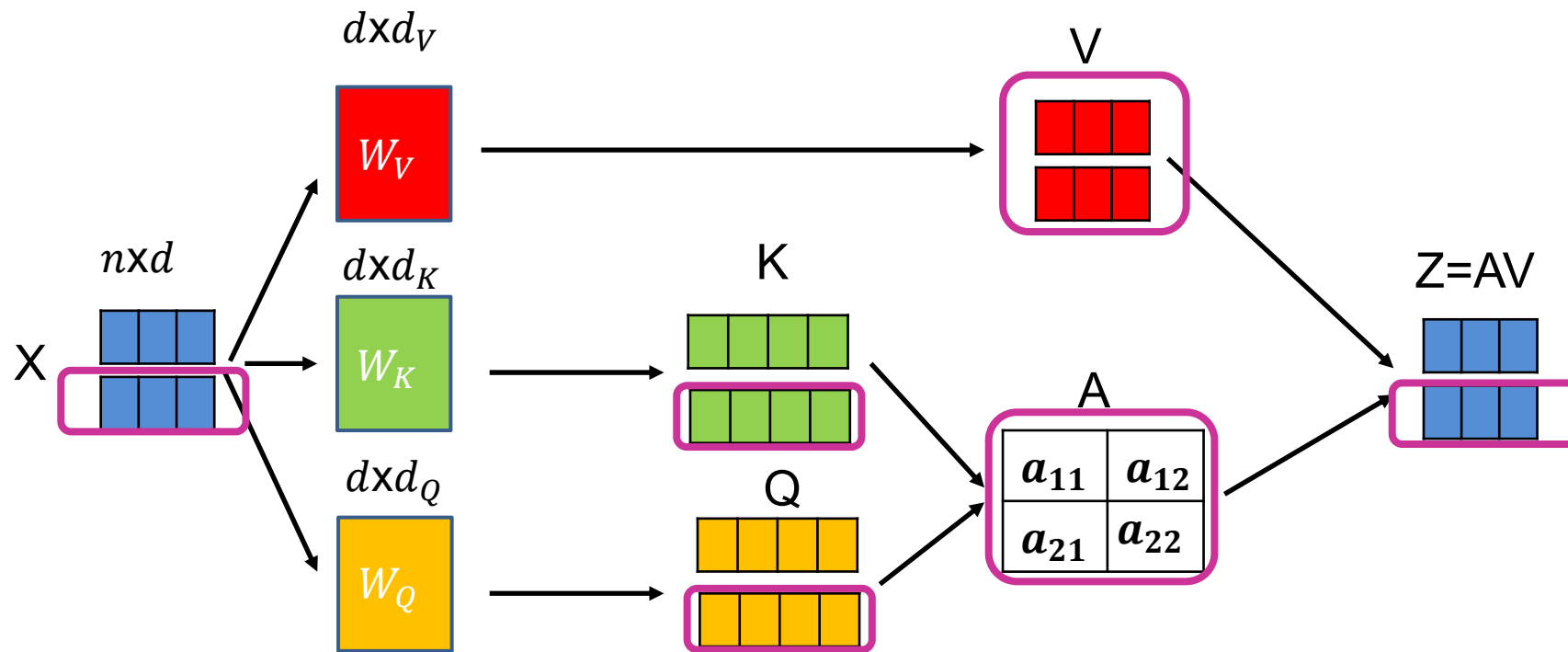
Complexity $O(N^2)$ with N the size of the input sequence

Summary - Attention in transformers – self attention detailed

- Tapez une équation ici.



Summary - Attention in transformers – self attention detailed



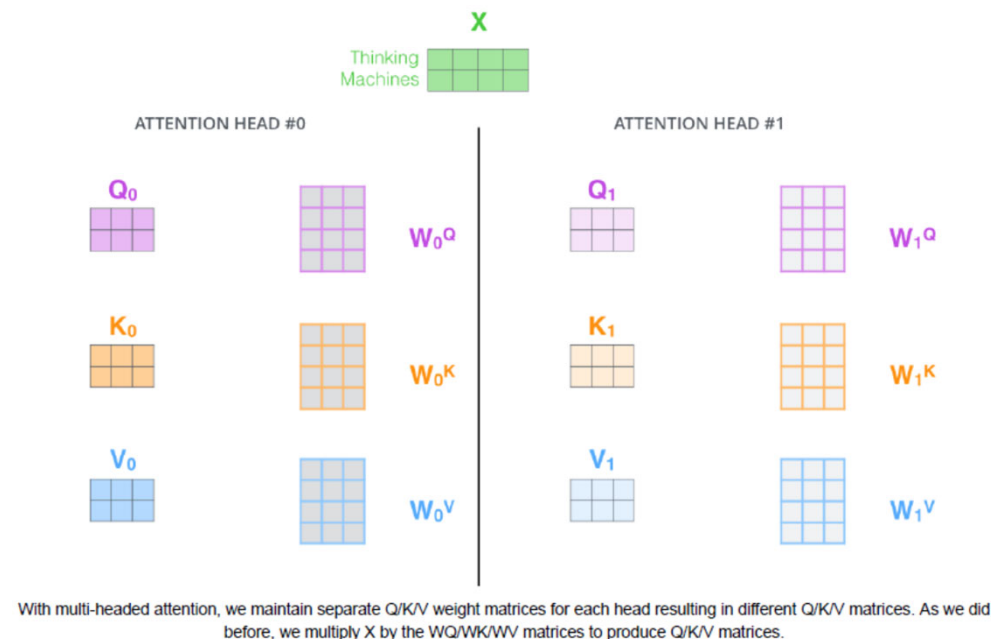
$$a_{21} = \text{softmax}\left(\frac{q_2 \cdot k_1}{\sqrt{d_k}}\right)$$

$$z_2 = a_{21}v_1 + a_{22}v_2$$

Transformer networks (Vaswani 2017, illustrations J. Alammari 2018, P. Bloem 2019)) - – Queries, keys, values

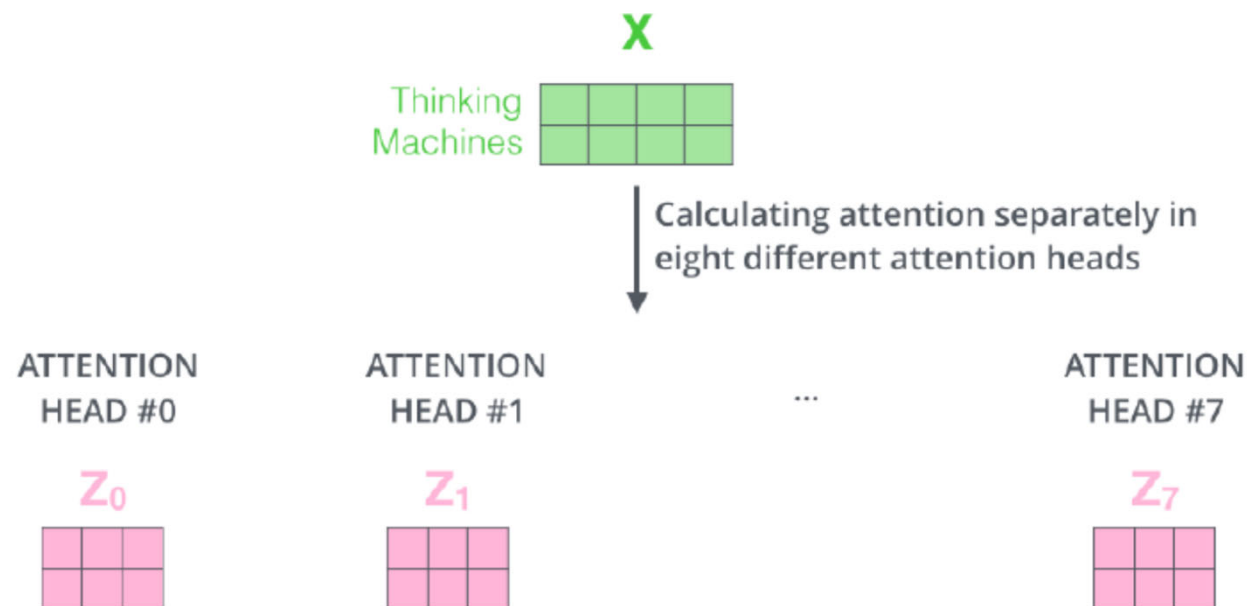
► Multi-head self attention

- Duplicate the self attention mechanism
- Allows us to focus on different parts of the input sequence and to encode different relations between elements of the input sequence
- Matrices for the different heads are denoted W_q^r, W_k^r, W_v^r with r the index of head r



Transformer networks (Vaswani 2017, illustrations J. Alammari 2018, P. Bloem 2019) - – Queries, keys, values

- Compute one output for each head



Transformer networks (Vaswani 2017, illustrations J. Alammam 2018, P. Bloem 2019))

- ▶ Multi-head self attention
- ▶ Two usual ways of applying multi-head
 - ▶ 1. Cut the embedding vector x_i into chunks and generate q, k, v from each chunk
 - ▶ e.g. if the embedding is size 256 and we have 8 heads, each chunk will be of size 32, the W_q^r, W_k^r, W_v^r are of size 32x32
 - ▶ 2. Apply each head to the whole vector
 - ▶ W_q^r, W_k^r, W_v^r are of size 256x256

Transformer networks (Vaswani 2017, illustrations J. Alammam 2018, P. Bloem 2019)

► Global output

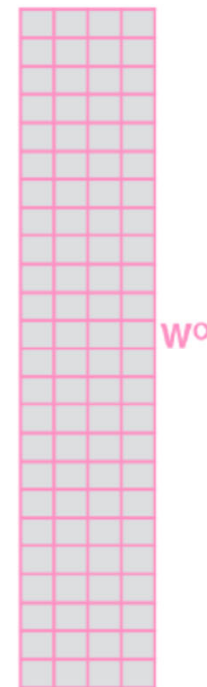
- Concatenate the individual head outputs
- Combine them with an additional matrix W^0 in order to produce an output of size k , for example the initial size of the embeddings

1) Concatenate all the attention heads



2) Multiply with a weight matrix W^0 that was trained jointly with the model

x



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



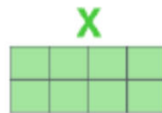
Transformer networks (Vaswani 2017, illustrations J. Alammr 2018, P. Bloem 2019)

► Summary of multi-head self attention

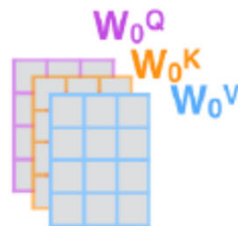
1) This is our input sentence*

Thinking
Machines

2) We embed each word*



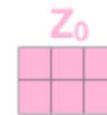
3) Split into 8 heads. We multiply X or R with weight matrices



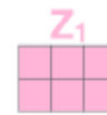
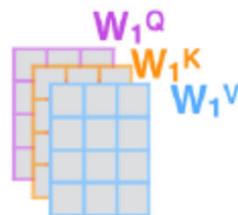
4) Calculate attention using the resulting $Q/K/V$ matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



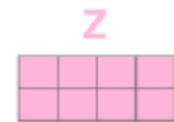
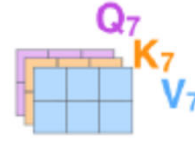
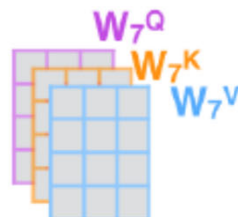
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



...

...

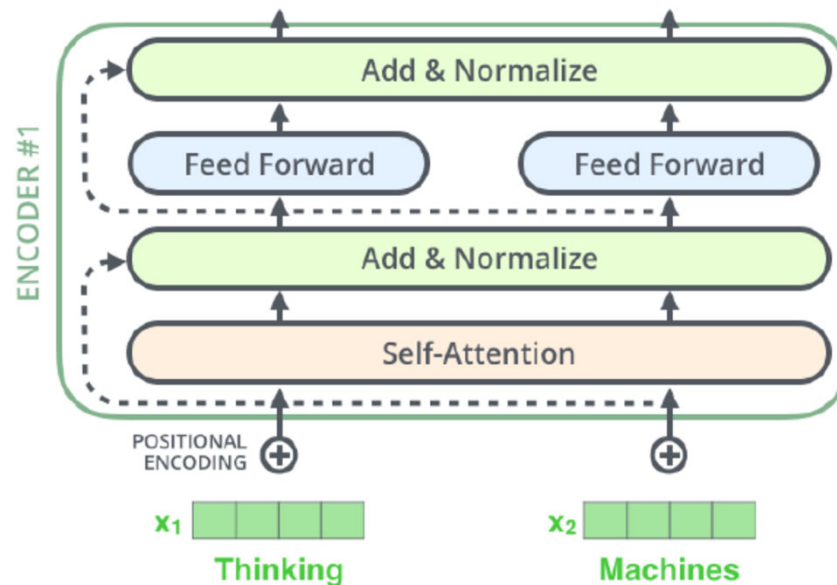
...



Transformer networks (Vaswani 2017, illustrations J. Alammari 2018, P. Bloem 2019)

Transformer module

- ▶ A transformer module combines different operations and is roughly defined as follows (several variants – here we detail an encoder module as in Vaswani 2017)
- ▶ The example takes two words as input and outputs two transformed encodings



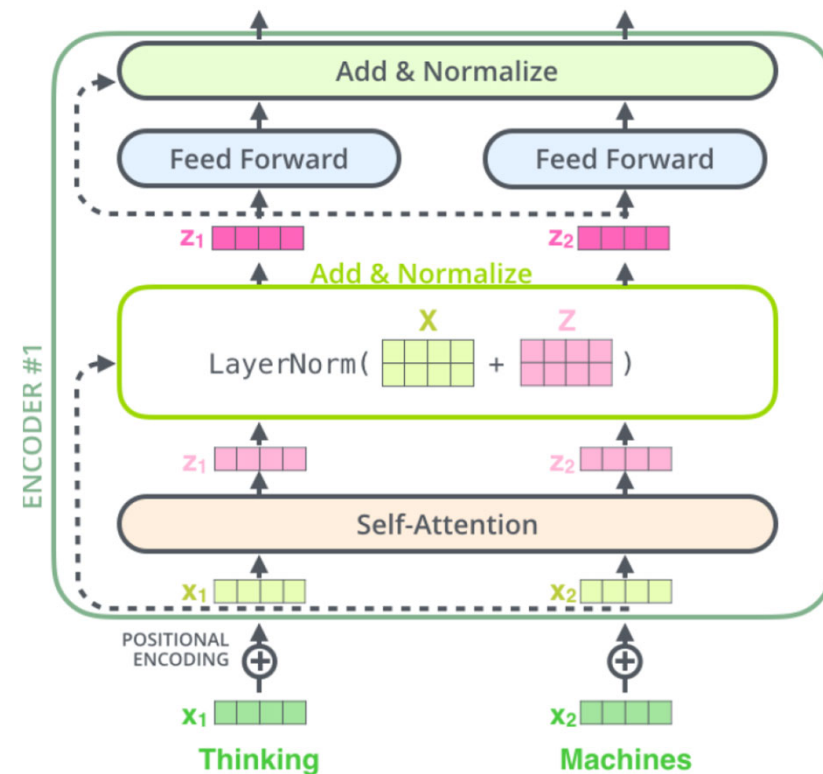
- Normalization layers (layer normalization)
- Multiple self attention modules per encoder
- Residual (skip) connections like in ResNet (see dashes --->)
- Positional encoding

Layer normalization: normalize the activations of a layer for **each sample** by **centering and reduction of the layer activation values** for that sample

Transformer networks (Vaswani 2017, illustrations J. Alammari 2018, P. Bloem 2019)

Transformer module

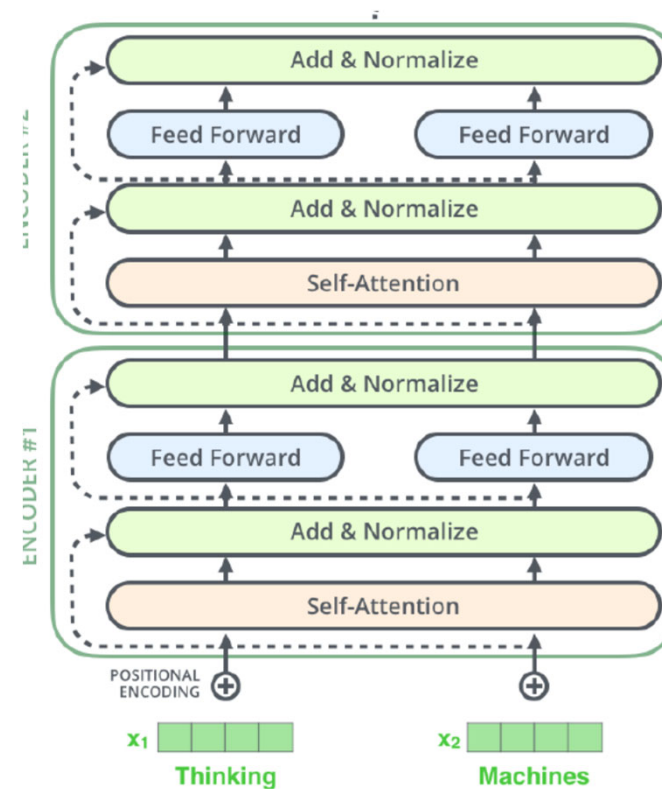
- ▶ Add and normalized detailed
- ▶ Residual connections are added before normalization
 - ▶ Helps with the gradient



Transformer networks (Vaswani 2017, illustrations J. Alammr 2018, P. Bloem 2019)

Transformer architecture

- Stack multiple transformer modules



Transformer networks (Vaswani 2017, illustrations J. Alammam 2018, P. Bloem 2019)

Transformer architecture

► Attention: word dependencies

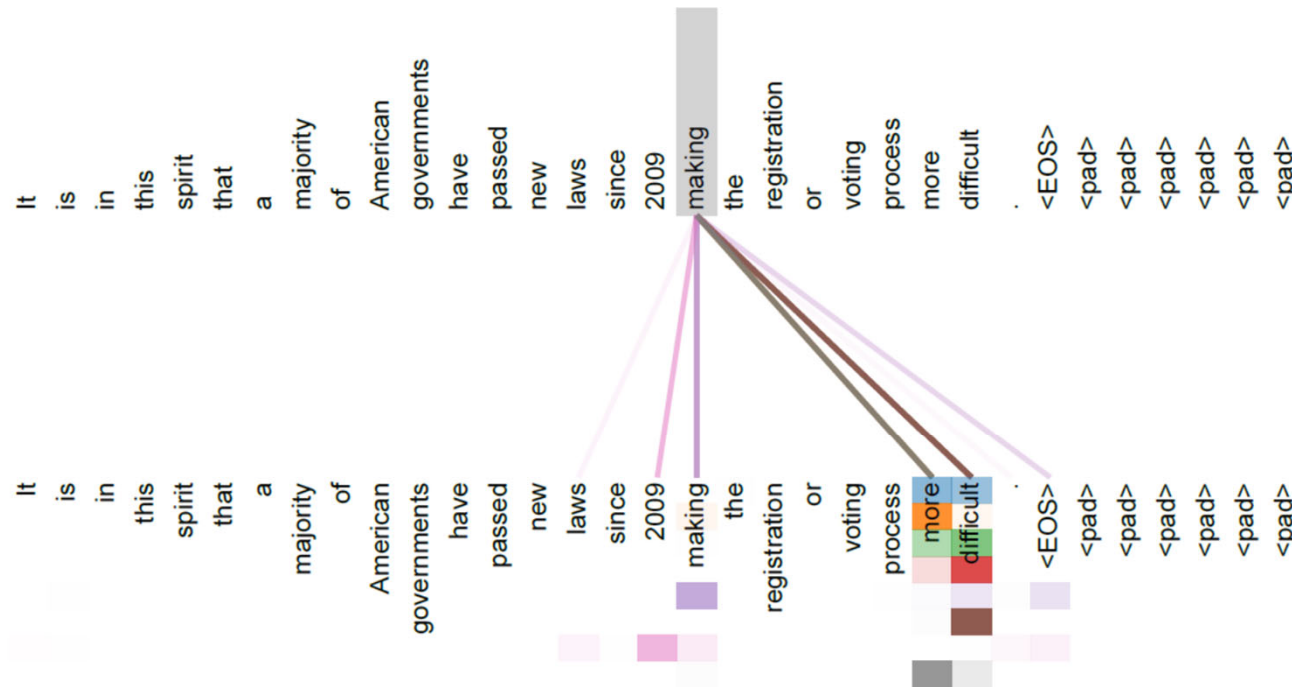


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb 'making', completing the phrase 'making...more difficult'. Attentions here shown only for the word 'making'. Different colors represent different heads. Best viewed in color.

Fig. (Vaswani 2017)

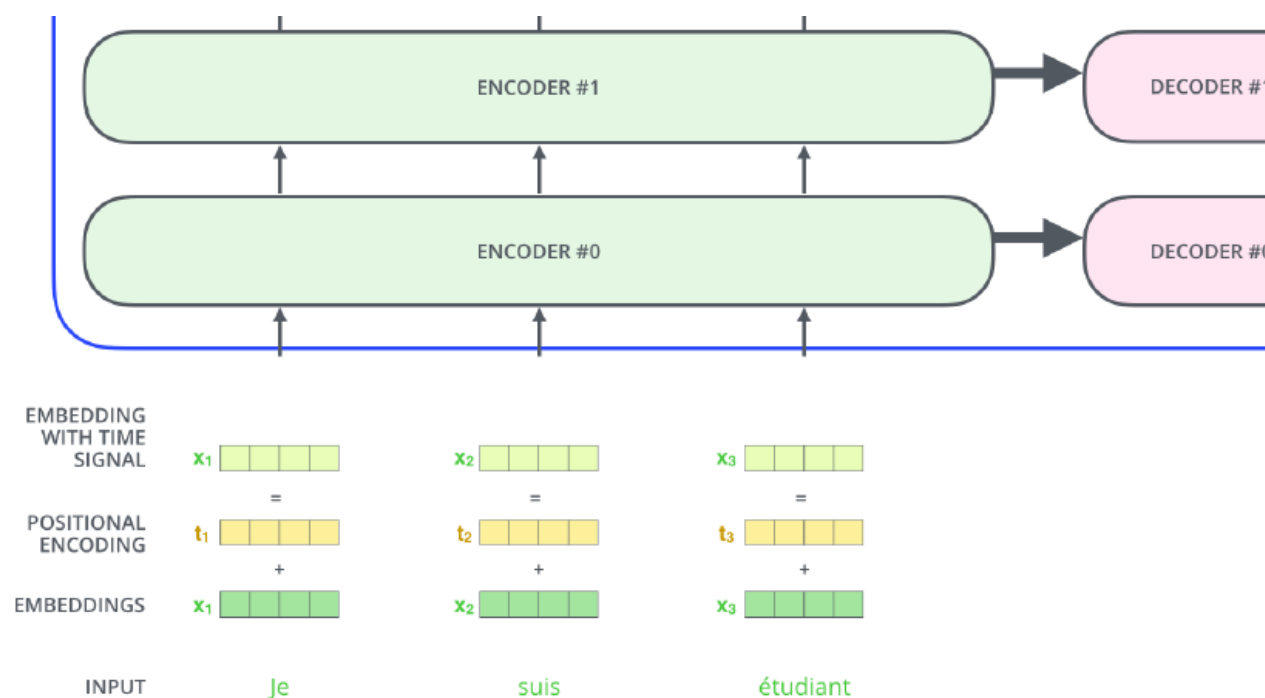
Transformer networks (Vaswani 2017, illustrations J. Alammari 2018, P. Bloem 2019)

► Positional encoding

- In order to account for the word order, the model makes use of a positional encoding together with the first word embeddings (first transformer module in the transformer multilayer architecture)
 - An information is added to each input embedding which helps determining the position of the word in the sentence.
 - This information is added to the input embeddings at the bottom of the transformer module
 - The encoding can be learned like word embeddings – this requires learning one embedding for each position
 - The encoding can be defined according to some function $f: N \rightarrow R^k$
 - In the original transformer paper, the encoding is defined as follows:
 - Let PE denote the *Positional Encoding*, $PE \in R^d \times R^n$, i.e. vector of length n , size of the sequence, and each positional encoding is of size d (same size as embeddings v).
 - $PE_{\text{pos}}(2i) = \sin(pos / 10000^{\frac{2i}{d}})$, $PE_{\text{pos}}(2i + 1) = \cos(pos / 10000^{\frac{2i}{d}})$
 - With pos the position in the sequence and $i \in \{1, \dots, d\}$ the dimension in the position vector

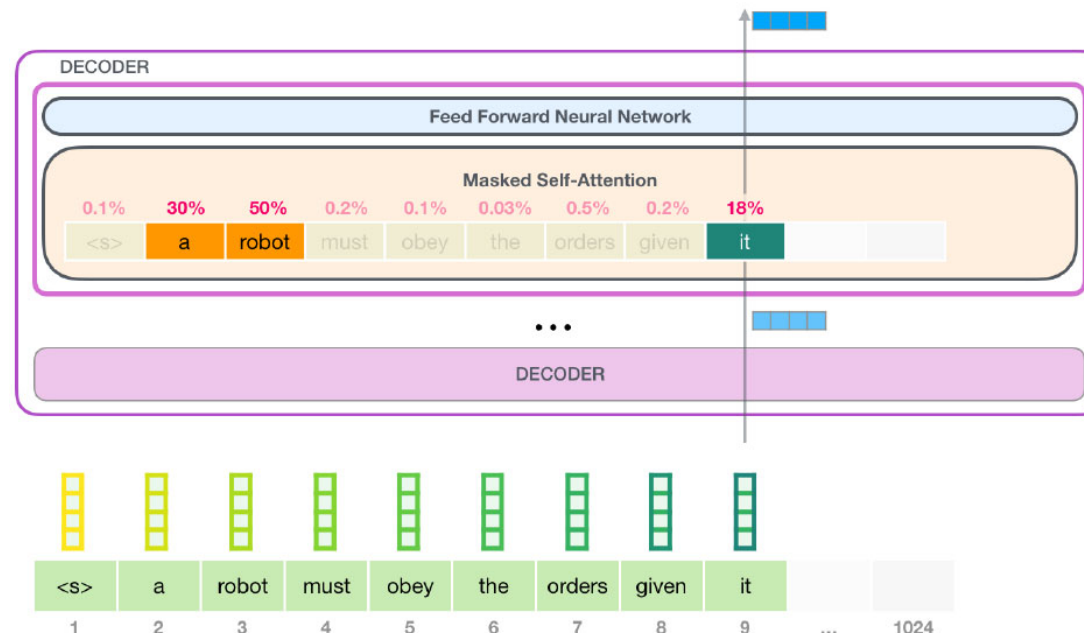
Transformer networks (Vaswani 2017, illustrations J. Alammr 2018-2019, P. Bloem 2019)

► Positional encoding



Transformer networks (Vaswani 2017, illustrations J. Alammam 2018-2019, P. Bloem 2019)

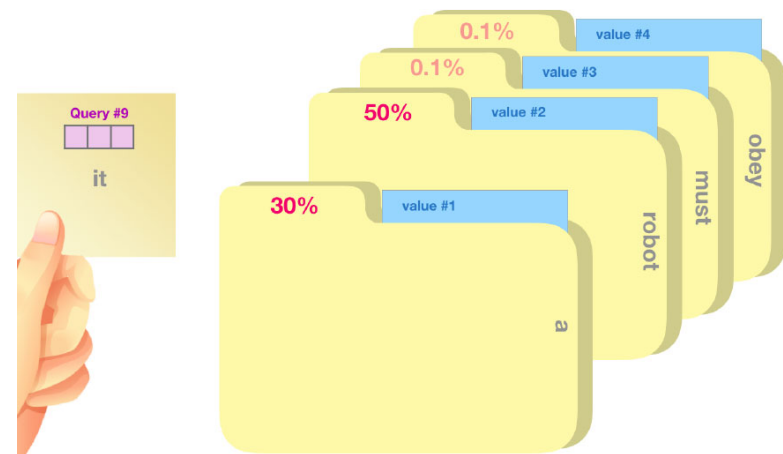
- ▶ Intuition on the Query/Key/value components (J. Alammam 2019)
- ▶ Consider the sentence
 - ▶ « a robot must obey the orders given it by human beings ... »
 - ▶ « It » refers to « a robot »
 - ▶ This is what self attention should detect
 - ▶ Consider self attention in the decoder module when processing the token « it »



Transformer networks (Vaswani 2017, illustrations J. Alammam 2018-2019, P. Bloem 2019)

- ▶ Intuition on the Query/Key/value components (J. Alammam 2019)
 - ▶ The Query is a representation of the current word used to score against all the other words (using their keys). We only care about the query of the token we're currently processing.
 - ▶ Key vectors are like labels for all the words in the segment. They're what we match against in our search for relevant words.
 - ▶ Value vectors are actual word representations, once we've scored how relevant each word is, these are the values we add up to represent the current word.

Analogy: searching through a filing cabinet. The Query is like a note with the topic you're researching. The Keys are like the labels of the folders inside the cabinet. When you match the tag with a note, we take out the contents of that folder, the Value vector. Except you're not only looking for one value, but a blend of values from a blend of folders.



The GPT family

Appendix - Historical side: Transformer networks (Vaswani 2017) illustration: J. Alammari 2018

- ▶ Transformer architectures use either encoder (e.g. BERT), decoder (e.g. GPT) or encoder-decoder (e.g. T5) schemes
 - ▶ BERT (Google) makes use of masked inputs (more on that later) and looks at the full input sequence
 - ▶ GPT (Open AI) is an autoregressive model (like a classical language model) and looks only at past items for predicting the future
 - ▶ T5 (Google) is an encoder-decoder model designed for reformulating several NLP tasks in a text to text framework

Large size language models based on transformers

Decoders - GPT family (OpenAI)

- ▶ GPT (Radford et al. 2018), GPT 2 (Radford et al. 2019), GPT 3 (Radford et al. 2020), InstructGPT, ChatGPT, GPT4. etc
 - ▶ GPT means **Generative Pre Training**
 - ▶ Language models based on transformer **decoder** architecture (Liu et al. 2018)
 - ▶ As for the other Transformer models, training proceeds in 2 steps
 - **Unsupervised language modeling**
 - **Fine tuning on downstream tasks**
 - Successive models are larger and larger and trained on larger and larger corpora
 - ▶ GPT 2 comes in different versions from 117 M parameters (12 transformer decoder blocks) to 1.542 M parameters (48 transformer decoder blocks)
 - It is trained on a corpus of 8 M documents, 40 GB of text (scraped web pages curated by humans to ensure document quality)
 - Demonstrates the ability of language models to solve tasks they are not trained on
 - Hence proposes an alternative to fine tuning
 - ▶ GPT 3: 96 Transformer decoder modules stacked, 175 Billions parameters (2020)
 - ▶ 100 times bigger than GPT2
 - ▶ Demonstrates that very large models perform well on zero shot and few shot learning
 - ▶ Started developments by different companies on LLM, chatbots, programming tools, etc

Large size language models based on transformers

Decoders - GPT family (OpenAI) – GPT2

▶ GPT 2

- ▶ Same general architecture than GPT with some modifications
 - ▶ layer normalization changed, initialization, scaling, etc...
- ▶ Training dataset
 - ▶ 40 GB of text (scraped web pages curated by humans to ensure document quality)
- ▶ Input representation
 - ▶ Modified Byte Pair Encoding (see later)
- ▶ Training
 - ▶ Language model only (unsupervised)
- ▶ Demonstrates that language models trained in an unsupervised way can achieve good performance, sometimes SOTA, on diverse tasks in few shot, zero shot learning schemes
- ▶ Generalize the use of prompting for task conditioning and for providing few shots examples
 - ▶ Language allows to provide in a natural ways task indication and task examples
 - ▶ Translation: (translate to French, English text, French text)
 - ▶ Reading comprehension: (answer the question, document, question, answer)

Large size language models based on transformers

Decoders - GPT family (OpenAI) – GPT3

- ▶ GPT3 is 100 times larger than GPT2 – 175 B parameters for the larger model @year 2020
- ▶ Training dataset
 - ▶ Same as for GPT2 – about 3 B words cleaned and augmented
- ▶ Model
 - ▶ Same general architecture as GPT2 – auto-regressive decoder
- ▶ Demonstrates that very large models are able to perform SOTA on few shot and zero shot learning
 - ▶ Size change qualitatively the ability of the model: Emergence capacities with in-context learning
 - ▶ Starts the exploration of LLM for solving a variety of language tasks
 - ▶ At the core of later developments like ChatGPT

Large size language models based on transformers

GPT family (OpenAI) – GPT3

► Importance of size

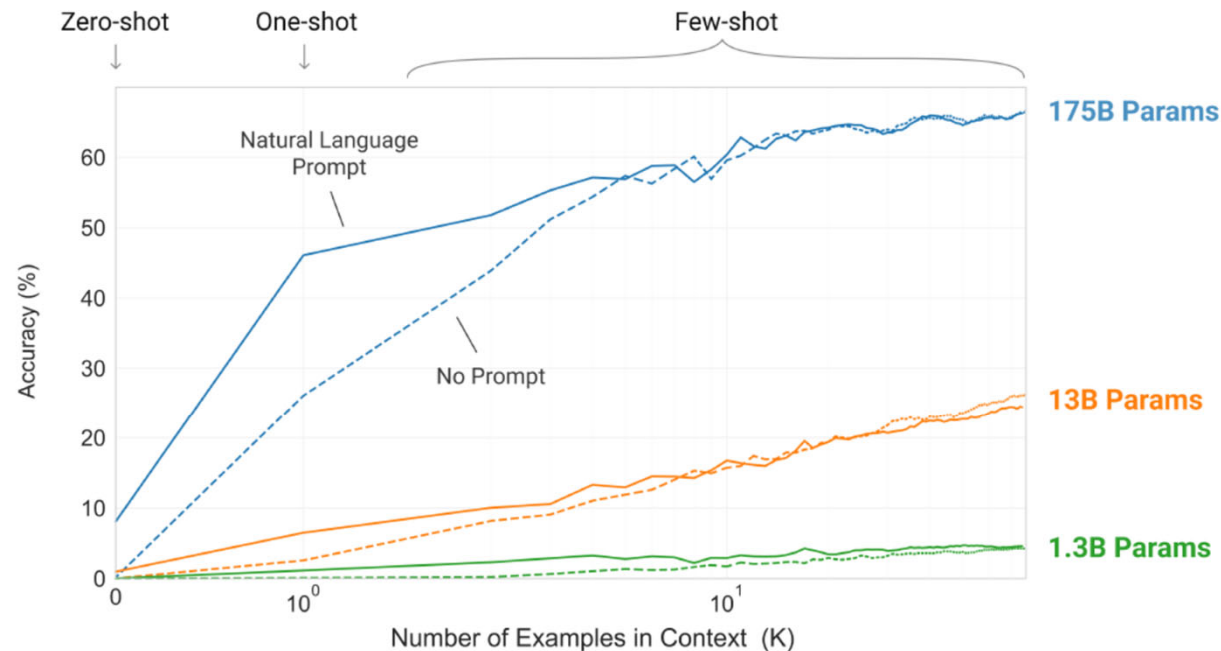


Figure 1.2: Larger models make increasingly efficient use of in-context information. We show in-context learning performance on a simple task requiring the model to remove random symbols from a word, both with and without a natural language task description (see Sec. 3.9.2). The steeper “in-context learning curves” for large models demonstrate improved ability to learn a task from contextual information. We see qualitatively similar behavior across a wide range of tasks.

Large size language models based on transformers

Decoders - GPT family (OpenAI) – GPT3

► Few shot, few shot learning

The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← example
3 cheese => ..... ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée
4 plush girafe => girafe peluche
5 cheese => ..... ← prompt
```

Traditional fine-tuning (not used for GPT-3)

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



Figure 2.1: Zero-shot, one-shot and few-shot, contrasted with traditional fine-tuning. The panels above show four methods for performing a task with a language model – fine-tuning is the traditional method, whereas zero-, one-, and few-shot, which we study in this work, require the model to perform the task with only forward passes at test time. We typically present the model with a few dozen examples in the few shot setting.

Large size language models based on transformers

Decoders - GPT family (OpenAI) – GPT3

► Arithmetic

- To test GPT-3's ability to perform simple arithmetic operations without task-specific training, we developed a small battery of 10 tests that involve asking GPT-3 a simple arithmetic problem in natural language:
- • 2 digit addition (2D+) – The model is asked to add two integers sampled uniformly from $[0; 100)$, phrased in the form of a question, e.g. “Q:What is 48 plus 76? A: 124.”
- • 2 digit subtraction (2D-) – The model is asked to subtract two integers sampled uniformly from $[0; 100)$; the answer may be negative. Example: “Q:What is 34 minus 53? A: -19”.
- • 3 digit addition (3D+) – Same as 2 digit addition, except numbers are uniformly sampled from $[0; 1000)$.

Context →	Q: What is (2 * 4) * 6? A:
Target Completion →	48

Figure G.42: Formatted dataset example for Arithmetic 1DC

Context →	Q: What is 17 minus 14? A:
Target Completion →	3

Figure G.43: Formatted dataset example for Arithmetic 2D-

Large size language models based on transformers

Decoders - GPT family (OpenAI) – GPT3

► task examples

Context →	Analysis of instar distributions of larval I. verticalis collected from a series of ponds also indicated that males were in more advanced instars than females. =
Target Completion →	L'analyse de la distribution de fréquence des stades larvaires d'I. verticalis dans une série d'étangs a également démontré que les larves mâles étaient à des stades plus avancés que les larves femelles.

Figure G.38: Formatted dataset example for En→Fr

Context →	Adevărul este că vă doriți, cu orice preț și împotriva dorinței europenilor, să continuați negocierile de aderare a Turciei la Uniunea Europeană, în ciuda refuzului continuu al Turciei de a recunoaște Ciprul și în ciuda faptului că reformele democratice au ajuns într-un punct mort. =
Target Completion →	The truth is that you want, at any price, and against the wishes of the peoples of Europe, to continue the negotiations for Turkey's accession to the European Union, despite Turkey's continuing refusal to recognise Cyprus and despite the fact that the democratic reforms are at a standstill.

Figure G.41: Formatted dataset example for Ro→En

Large size language models based on transformers

Decoders - GPT family (OpenAI) – GPT3

- ▶ Task examples: choosing an answer
 - ▶ **PIQA**
 - ▶ Common sense questions on the physical world
 - ▶ **COPA**
 - ▶ A task from the superGLUE dataset

Context →	How to apply sealant to wood.
Correct Answer →	Using a brush, brush on sealant onto wood until it is fully saturated with the sealant.
Incorrect Answer →	Using a brush, drip on sealant onto wood until it is fully saturated with the sealant.

Figure G.4: Formatted dataset example for PIQA

Context →	My body cast a shadow over the grass because
Correct Answer →	the sun was rising.
Incorrect Answer →	the grass was cut.

Figure G.5: Formatted dataset example for COPA

Generative models

Variational Auto-Encoders

Flow models

Generative models

► Objective

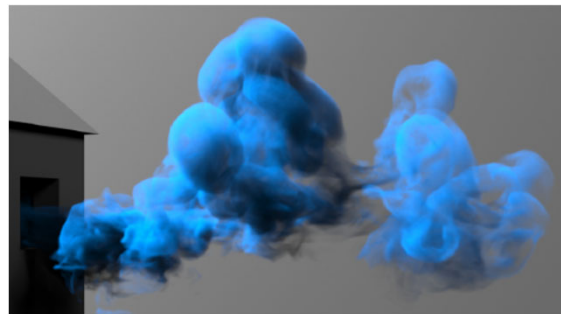
► Learn a probability distribution model from data samples

- Given $x^1, \dots, x^N \in R^n$ learn to approximate their underlying distribution $\mathcal{X}$
- For complex distributions, there is no analytical form, and for large size spaces (R^n) approximate methods (e.g. MCMC) might fail
- Deep generative models recently attacked this problem with the objective of handling large dimensions and complex distributions

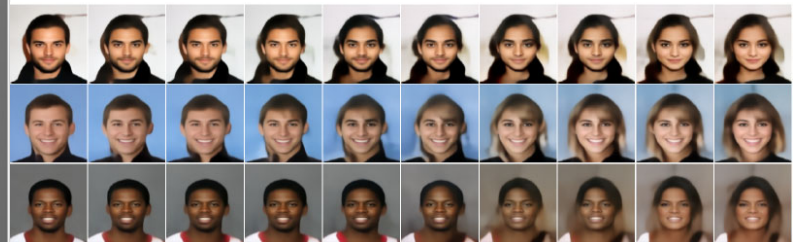


https://en.wikipedia.org/wiki/Edmond_de_Belamy
432 k\$ Christies in 2018

200



Xie et al. 2019
artificial smoke



De Bezenac et al. 2021
Generating female images from
male ones

Advanced Deep learning

Generative models

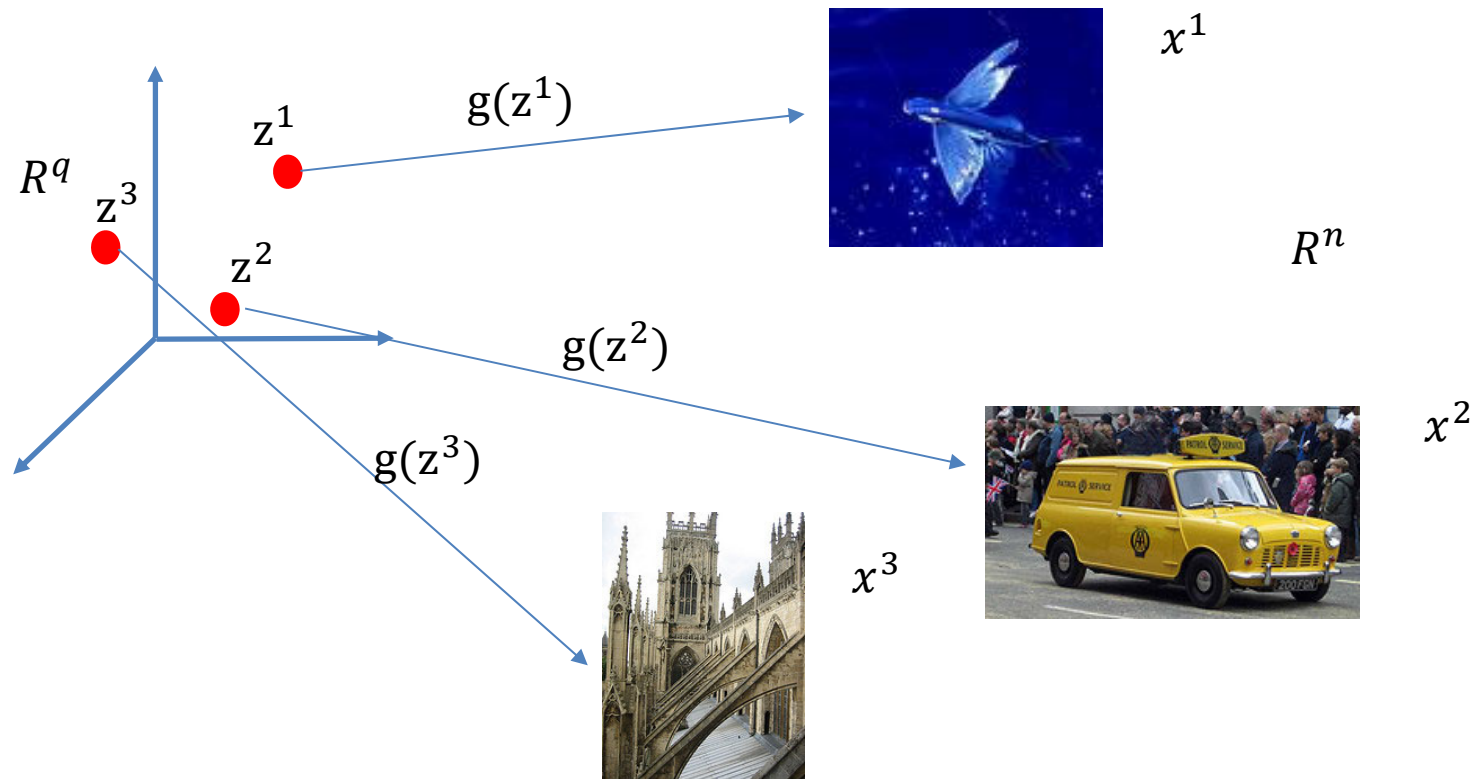
► Objective

► General setup of deep generative models

- Learn a generator network $g_\theta: R^q \rightarrow R^n$ that transforms a latent distribution $\mathcal{Z} \subset R^q$ to match a target distribution $\mathcal{X}$
 - $\mathcal{Z}$ is usually a simple distribution e.g. Gaussian from which it is easy to sample, $q < n$
 - This is unlike traditional statistics where an analytic expression for the distribution is sought
- Once trained the generator can be used for:
 - **Sampling** from the latent space:
 - $z \in R^q \sim \mathcal{Z}$ and then generate synthetic data via $g_\theta(\cdot)$, $g_\theta(z) \in R^n$
 - When possible, **density estimation** $p_\theta(x) = \int p_\theta(x|z)p_{\mathcal{Z}}(z)dz$
 - with $p_\theta(x|z)$ a function of g_θ

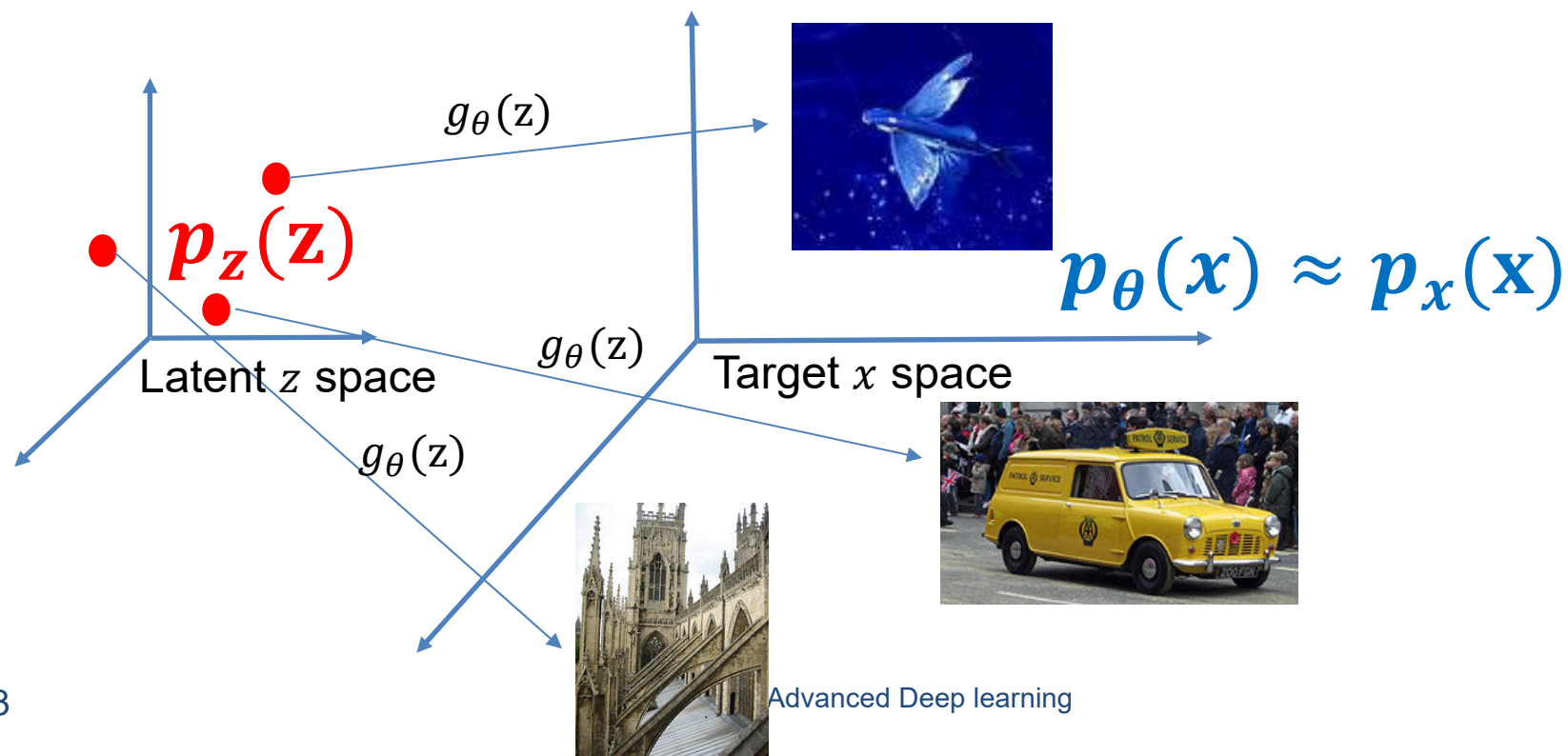
Generative models intuition

- ▶ Let $\{z^1, \dots, z^N\}, z^i \in R^q$ and $\{x^1, \dots, x^N\}, x^i \in R^n$, two sets of points in different spaces
- ▶ Provided a sufficiently powerful model $g(x)$, it should be possible to learn complex deterministic mappings associating the two sets:



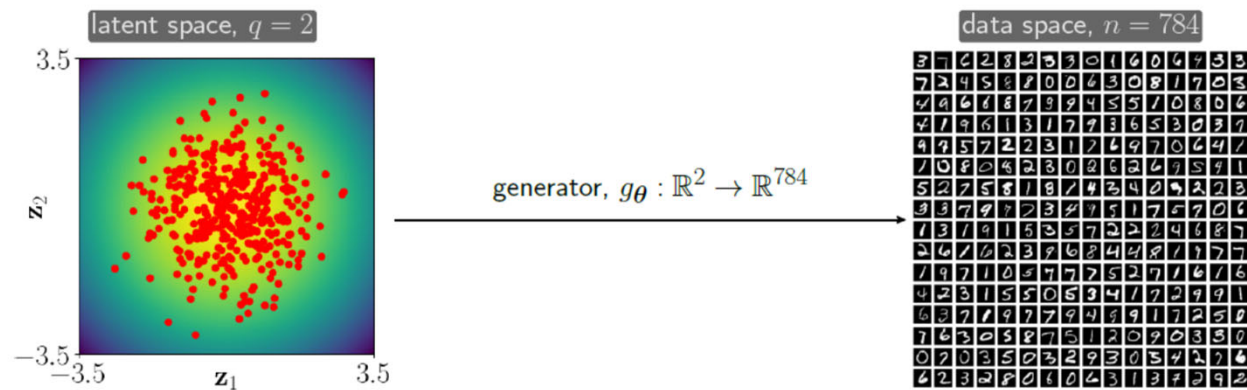
Generative models intuition

- ▶ Given distributions on a latent space $p_z(z)$, and on the data space $p_x(x)$, it is possible to map $p_z(z)$ onto $p_x(x)$?
 - ▶ g_θ defines a distribution on the target space $p_x(g_\theta(z)) = p_\theta(x)$
 - ▶ $p_\theta(x)$ is the generated data distribution, objective: $p_\theta(x) \approx p_x(x)$
 - ▶ Data generation: sample $z \sim Z$, transform with $g_\theta, g_\theta(z)$



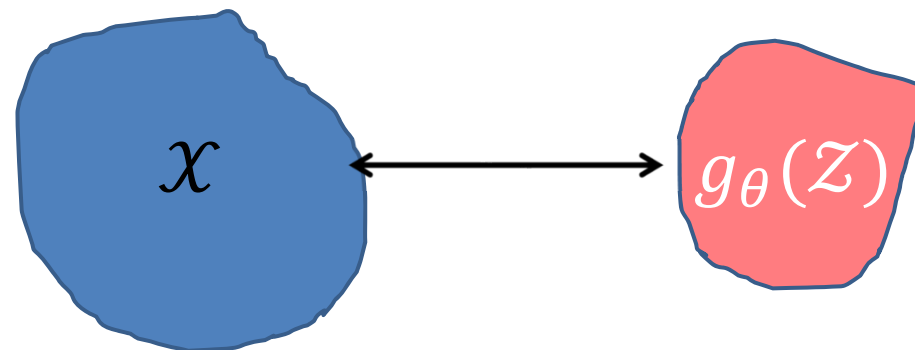
Generative models intuition

- ▶ Data generation: sample $z \sim Z$, transform with $g_\theta, g_\theta(z)$



- ▶ Important issue

- ▶ How to compare predicted distribution $p_\theta(x)$ and target distribution $p_x(x)$?



Generative models key ideas

► Objectives

- Generative modeling aims at learning how to sample from a target distribution $p_{\mathcal{X}}(x)$
- Learn to sample using a training dataset, drawn from $p_{\mathcal{X}}(x): x^1, \dots, x^N$

► Challenges

- Enable fast sampling
- Generate high quality samples
- Cover $p_{\mathcal{X}}(x)$

► How

- Choose from a simple distribution, easy to sample from
- Learn to push this distribution to the target distribution
- The push operation is performed by a learned model (here a neural network)

► Note

- Generation can be performed conditionally on a context
 - e.g. generating image/ video from a text prompt

Course objective

- ▶ Introduce three popular families of generative models
 - ▶ Joint requirements
 - Learn a generator g_θ from samples so that distribution $g_\theta(\mathcal{Z})$ is close to data distribution $\mathcal{X}$, $p_\theta(x) \approx p_x(x)$
 - This requires measuring the similarity between $g_\theta(\mathcal{Z})$ and $\mathcal{X}$
 - Different similarities are used for each family
 - ▶ Three families
 - Variational autoencoders
 - $g_\theta: R^q \rightarrow R^n, q \ll n$
 - Trained to maximize a lower bound of the samples' likelihood
 - Assumption: a density function explains the data
 - Flow matching
 - $g_\theta: R^q \rightarrow R^n, q \ll n$
 - Can approximate any distribution (no density assumption)
 - Maximizes likelihood by learning paths between the two distributions

Variational Auto-Encoders

After Kingma D., Welling M., Auto-Encoding Variational Bayes,
ICLR 2014

Plus some blogs – see the references

Prerequisite KL divergence

▶ Kullback Leibler divergence

- ▶ Measure of the difference between two distributions p and q

- ▶ Continuous variables

- ▶ $D_{KL}(p(y)||q(y)) = \int_y (\log \frac{p(y)}{q(y)})p(y)dy$

- ▶ Discrete variables

- ▶ $D_{KL}(p(y)||q(y)) = \sum_i (\log \frac{p(y_i)}{q(y_i)})p(y_i)$

▶ Property

- ▶ $D_{KL}(p(y)||q(y)) \geq 0$

- ▶ $D_{KL}(p(y)||q(y)) = 0$ iff $p = q$

- ▶ $D_{KL}(p(y)||q(y)) = -E_{p(y)} \left[\log \frac{q(y)}{p(y)} \right] \geq -\log E_{p(y)} \left[\frac{q(y)}{p(y)} \right] = 0$

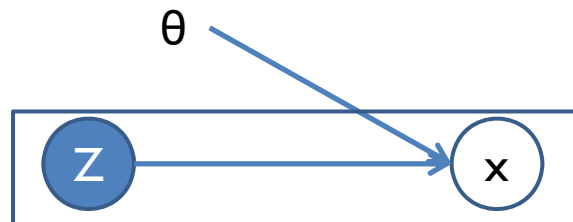
- the first inequality is obtained via Jensen inequality:

- For a convex function f , $f(E[x]) \leq E[f(x)]$, and $-\log x$ is a convex function

- ▶ note: D_{KL} is asymmetric, symmetric versions exist, e.g. Jensen-Shannon divergence

Preliminaries – Variational methods

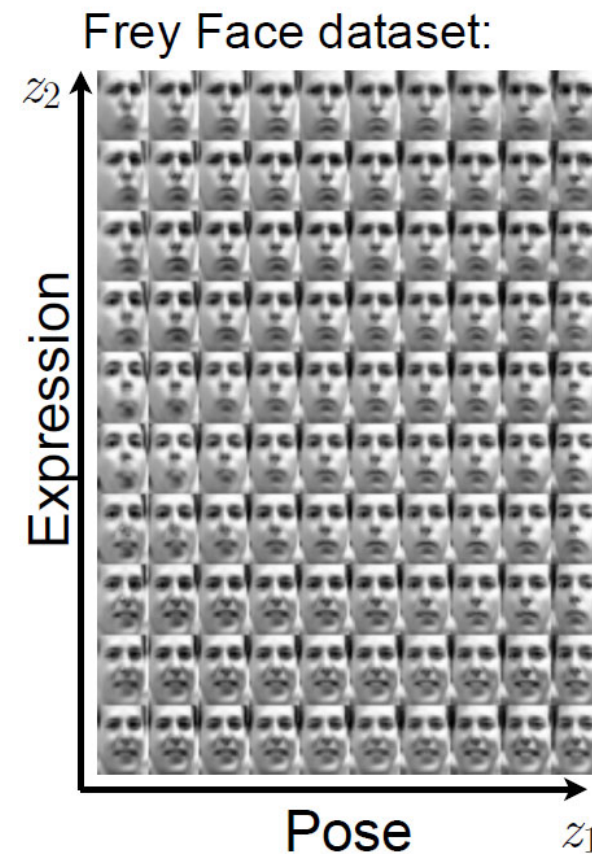
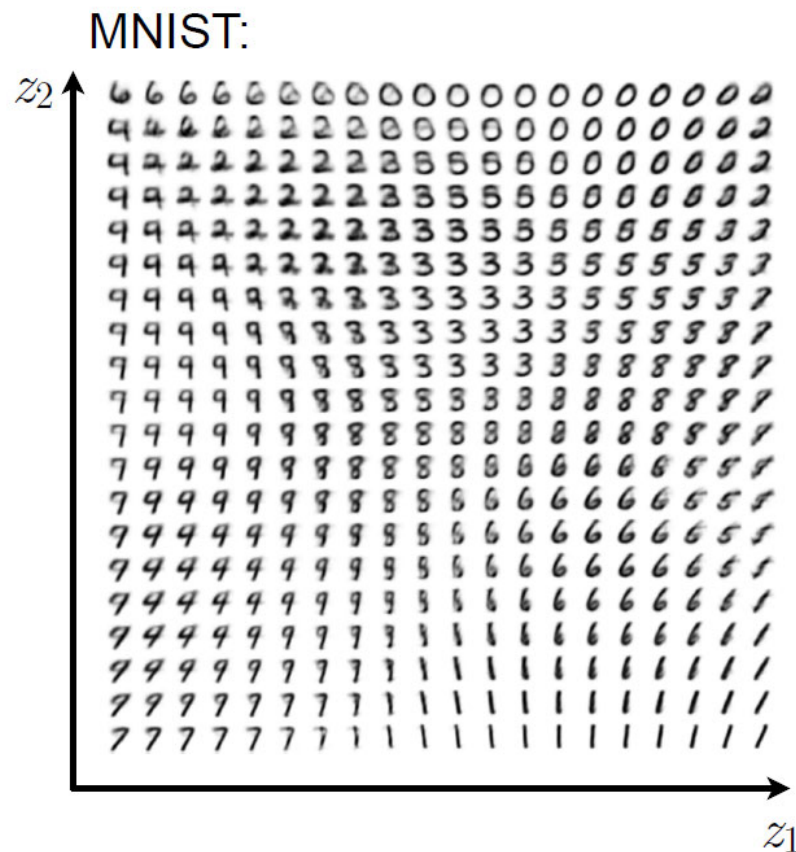
- ▶ Generative latent variable model
- ▶ Let us suppose available a joint model on the observed and latent variables $p_{\theta}(x, z)$



- ▶ The observations x are generated by the following process
 - ▶ Sample from $z \sim p_{\theta}(z)$ - $p_{\theta}(z)$ is the prior
 - ▶ generate $p_{\theta}(x|z)$ - $p_{\theta}(x|z)$ is the likelihood
- ▶ Training objective
 - ▶ We want to optimize the likelihood of the observed data
 - ▶ $p(x) = \int p(x|z)p(z)dz$ - $p(x)$ is called the evidence
 - ▶ Computing the integral requires evaluating over all the configurations of latent variables,
 - ▶ This is often intractable
 - ▶ In order to narrow the sampling space, one may use importance sampling, i.e. sampling important z instead of sampling blindly from the prior
 - ▶ Let us introduce a sampling function $q_{\phi}(z|x)$

VAEs - Intuition

- Intuitively, z might correspond to the factors conditioning the generation of the data



Generative models intuition

- ▶ What we want: organize the latent space according to some characteristics of the observations (images)

An Oversimplified Example of a Cat/Dog Image Latent Space

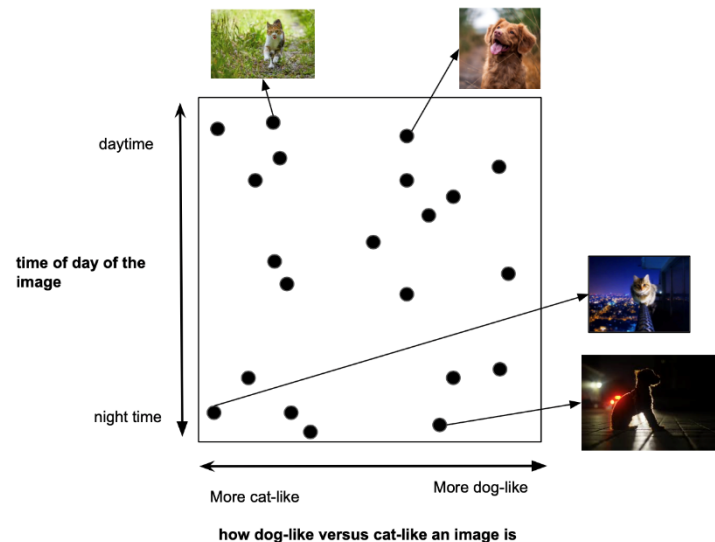


Fig.: <https://ml.berkeley.edu/blog/posts/vq-vae/>

- ▶ See also the demos @
 - ▶ <https://jaan.io/what-is-variational-autoencoder-vae-tutorial/>

VAE

Loss criterion – summary

- ▶ The log likelihood for data point x can be decomposed as
 - ▶ $\log p_{\theta}(x) = D_{KL}(q_{\phi}(z|x)||p_{\theta}(z|x)) + V_L(\theta, \phi; x)$
 - ▶ with
 - ▶ $V_L(\theta, \phi; x) = -D_{KL}(q_{\phi}(z|x)||p(z)) + E_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)]$
- ▶ Why is it useful?
 - ▶ $D_{KL}(\cdot || \cdot) \geq 0$, then $V_L(\theta, \phi; x)$ is a lower bound of $\log p_{\theta}(x)$
 - ▶ in order to maximize $\log p_{\theta}(x)$, we will maximize $V_L(\theta, \phi; x)$
- ▶ $V_L(\theta, \phi; x)$ is called the **ELBO: Evidence Lower Bound**
 - ▶ With an appropriate choice of $q_{\phi}(z|x)$ this is amenable to a computable form
 - ▶ $q_{\phi}(z|x)$ approximates the intractable posterior $p_{\theta}(z|x)$
 - ▶ This method is called variational inference
 - ▶ In general inference denotes the computations of hidden variables given observed ones (e.g. inferring the class of an object)
- ▶ Note
 - ▶ Because each representation z is associated to a unique x , the loss likelihood can be decomposed for each point – this is what we do here
 - ▶ The global log likelihood is then the summation of these individual losses

VAE

Loss criterion – summary

► Variational lower bound:

► $V_L(\theta, \phi; x) = -D_{KL}(q_\phi(z|x)||p(z)) + E_{q_\phi(z|x)}[\log p_\theta(x|z)]$

► Remarks

- $E_{q_\phi(z|x)}[\log p_\theta(x|z)]$ is a **reconstruction** term
 - Measures how well the datum x can be reconstructed from latent representation z
- $D_{KL}(q_\phi(z|x)||p(z))$ is a **regularization** term:
 - Forces the learned distribution $q_\phi(z|x)$ to stay close to the prior $p(z)$
 - Otherwise a trivial solution would be to learn a Dirac distribution for $q_\phi(z|x)$
 - We want the z to be close in the latent space for similar x s
 - $p(z)$ has usually a simple form e.g. $\mathcal{N}(0, I)$, then $q_\phi(z|x)$ is also forced to remain simple

VAE details

Derivation of the loss function

$$\blacktriangleright \log p_{\theta}(x) = D_{KL}(q_{\phi}(z|x)||p_{\theta}(z|x)) + V_L(\theta, \phi; x)$$

Proof

- ▶ $\log p_{\theta}(x) = \int_z (\log p(x))q(z|x) dz \quad (\int_z q(z|x) dz = 1)$
- ▶ $\log p_{\theta}(x) = \int_z (\log \frac{p(x,z)}{p(z|x)})q(z|x) dz$
- ▶ $\log p_{\theta}(x) = \int_z (\log \frac{p(x,z)}{q(z|x)} \frac{q(z|x)}{p(z|x)})q(z|x) dz$
- ▶ $\log p_{\theta}(x) = \int_z (\log \frac{p(x,z)}{q(z|x)})q(z|x) dz + \int_z (\log \frac{q(z|x)}{p(z|x)})q(z|x) dz$
- ▶ $\log p_{\theta}(x) = E_{q(z|x)}[\log p(x, z) - \log q(z|x)] + D_{KL}(q(z|x)||p(z|x))$

$$\log p_{\theta}(x) = V_L(\theta, \phi; x) + D_{KL}(q_{\phi}(z|x)||p_{\theta}(z|x))$$

with

$$V_L(\theta, \phi; x) = E_{q(z|x)}[\log p_{\theta}(x, z) - \log q_{\phi}(z|x)]$$

- ▶ Maximizing $\log p_{\theta}(x)$ is equivalent to maximizing $V_L(\theta, \phi; x)$ (and minimizing $D_{KL}(q_{\phi}(z|x)||p_{\theta}(z|x))$)
- ▶ $V_L(\theta, \phi; x)$ is called an Evidence Lower Bound (ELBO)

VAE details

Derivation of the loss function

$$\blacktriangleright V_L(\boldsymbol{\theta}, \boldsymbol{\phi}; \boldsymbol{x}) = -D_{KL}(\boldsymbol{q}_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})||\boldsymbol{p}(\boldsymbol{z})) + E_{\boldsymbol{q}_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}[\log \boldsymbol{p}_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})]$$

Proof:

- ▶ $V_L(\theta, \phi; x) = E_{q_{\phi}(z|x)}[\log p_{\theta}(x, z) - \log q_{\phi}(z|x)]$
- ▶ $V_L(\theta, \phi; x) = E_{q_{\phi}(z|x)}[\log p_{\theta}(x|z) + \log p_{\theta}(z) - \log q_{\phi}(z|x)]$
- ▶ $V_L(\theta, \phi; x) = -D_{KL}(q_{\phi}(z|x)||p_{\theta}(z)) + E_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)]$

VAE

Loss criterion – summary

- ▶ Variational lower bound:

- ▶ $V_L(\theta, \phi; x) = -D_{KL}(q_\phi(z|x)||p(z)) + E_{q_\phi(z|x)}[\log p_\theta(x|z)]$

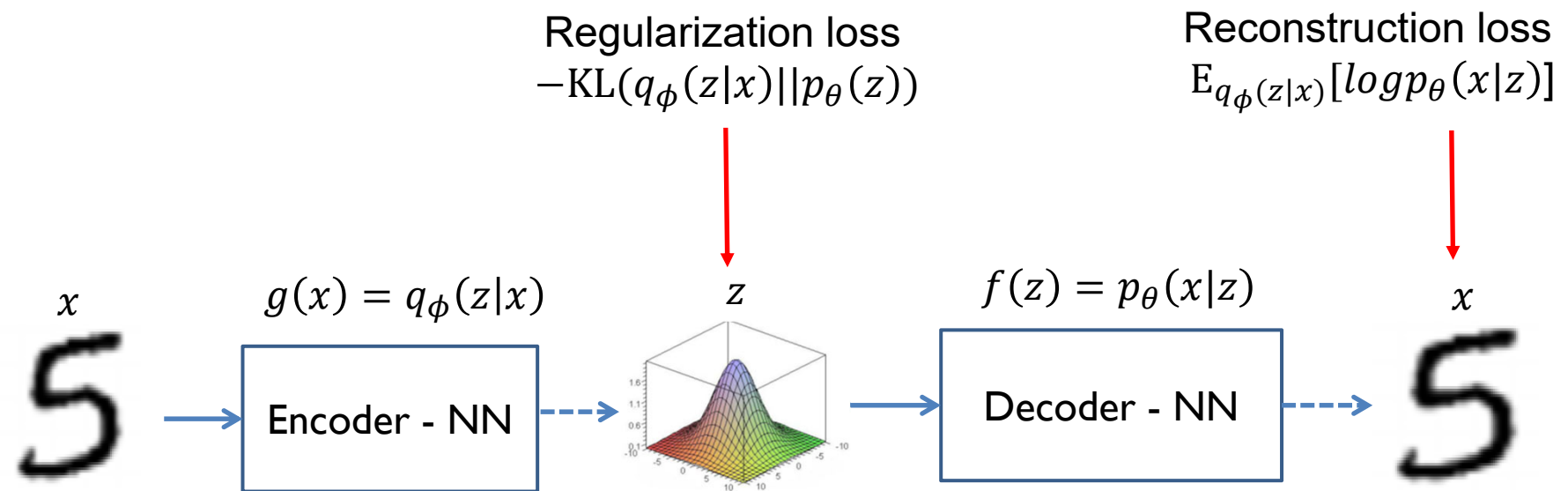
- ▶ This form provides a link with a NN implementation

- ▶ The generative $p_\theta(x|z)$ and inference $q_\phi(z|x)$ modules are implemented by NNs
 - ▶ They will be trained to maximize the reconstruction error for each (z, x) :
 $E_{q_\phi(z|x)}[\log p_\theta(x|z)]$ term
 - ▶ The inference module $q_\phi(z|x)$ will be constrained to remain close to the prior $p(z)$:
 $-D_{KL}(q_\phi(z|x)||p_\theta(z)) \approx 0$

VAE

Loss - summary

- ▶ Loss function in the NN model



- ▶ Training performed via Stochastic gradient
 - ▶ This requires an analytical expression for the loss functions and for gradient computations
 - ▶ $\dashrightarrow$ Sampling
 - ▶ $\rightarrow$ deterministic

VAE- reparametrization trick

- ▶ Training with stochastic units: reparametrization trick
 - ▶ Not possible to propagate the gradient through stochastic units (the z s and x s are generated via sampling)
 - ▶ Solution
 - ▶ Parametrize z as a deterministic transformation of a random variable ϵ : $z = g_\phi(x, \epsilon)$ with $\epsilon \sim p(\epsilon)$ independent of ϕ , e.g. $\epsilon \sim N(0,1)$
 - ▶ Example
 - If $z \sim \mathcal{N}(\mu, \sigma)$, it can be reparameterized by $z = \mu + \sigma \odot \epsilon$, with $\epsilon \sim \mathcal{N}(0,1)$, with $\odot$ pointwise multiplication (μ, σ are vectors here)
 - For the NN implementation we have: $z = \mu_z(x) + \sigma_z(x) \odot \epsilon_z$
 - ▶ This will allow the derivatives to « pass » through the z
 - With this expression, one may compute the gradients of the ELBO with to the NN parameters of $\mu_z(x)$ and $\sigma_z(x)$
 - For the derivative, the sampling operation is regarded as a deterministic operation with an extra input ϵ_z , whose distribution does not involve variables needed in the derivation

VAE - reparametrization trick

- Reparametrization (fig. from D. Kingma)

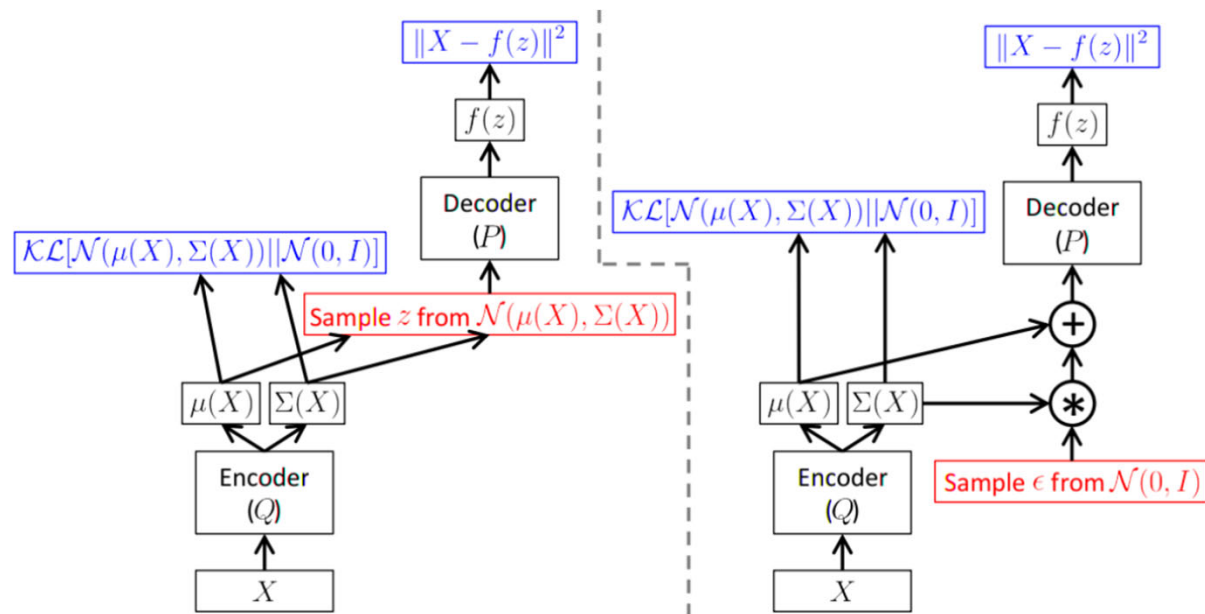


Figure 4: A training-time variational autoencoder implemented as a feed-forward neural network, where $P(X|z)$ is Gaussian. Left is without the “reparameterization trick”, and right is with it. Red shows sampling operations that are non-differentiable. Blue shows loss layers. The feedforward behavior of these networks is identical, but backpropagation can be applied only to the right network.

VAE

Exemple: Gaussian priors and posteriors

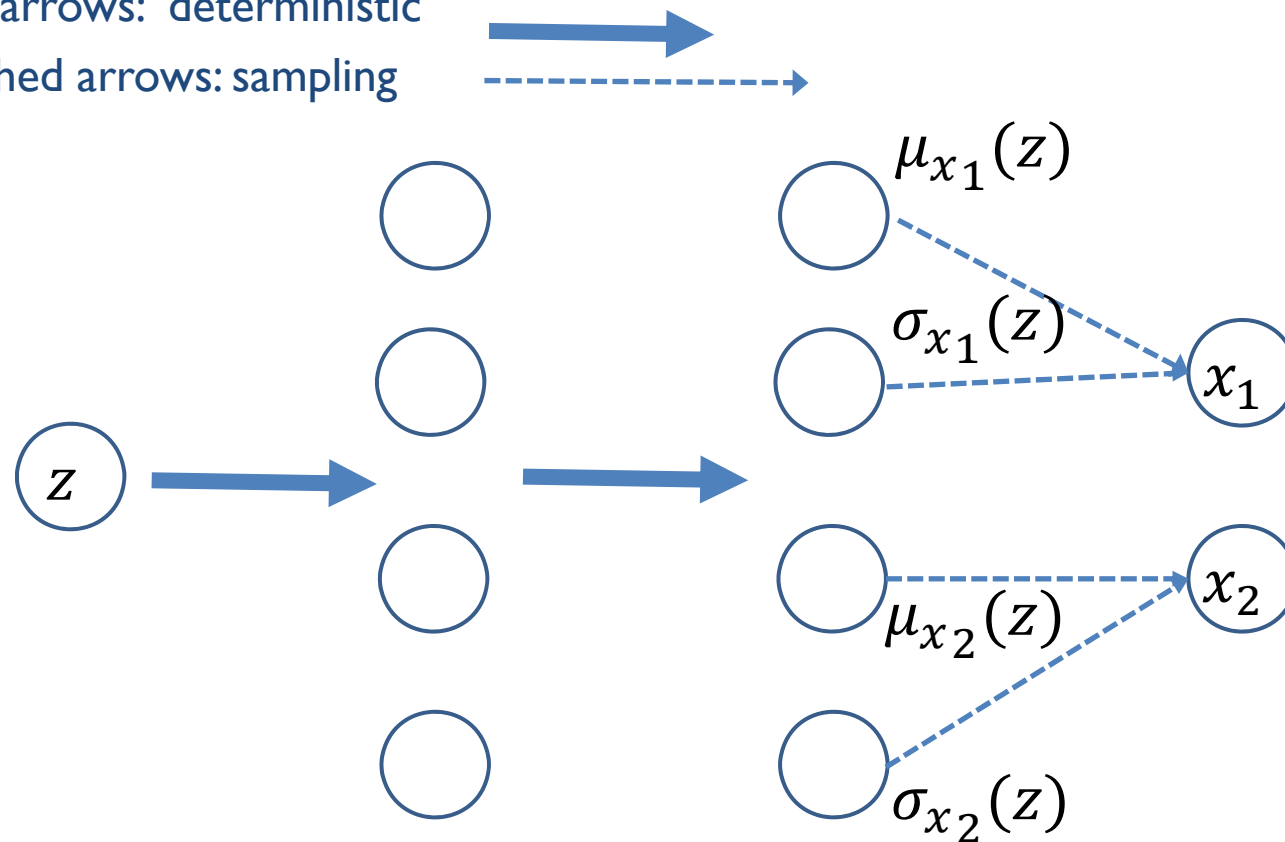
- ▶ Special case: gaussian priors and posteriors
- ▶ Hyp:
 - ▶ $p(z) = \mathcal{N}(0, I)$
 - ▶ $p_{\theta}(x|z) = \mathcal{N}(\mu(z), \sigma(z))$, $\sigma(z)$ diagonal matrix, $x \in R^D$
 - ▶ $q_{\phi}(z|x) = \mathcal{N}(\mu(x), \sigma(x))$, $\sigma(x)$ diagonal matrix, $z \in R^J$

VAE

Exemple: Gaussian priors and posteriors - illustration

► Decoder:

- in the example z is 1 dimensional and x is 2 dimensional, f is a 1 hidden layer MLP with gaussian output units and tanh hidden units
- full arrows: deterministic
- dashed arrows: sampling

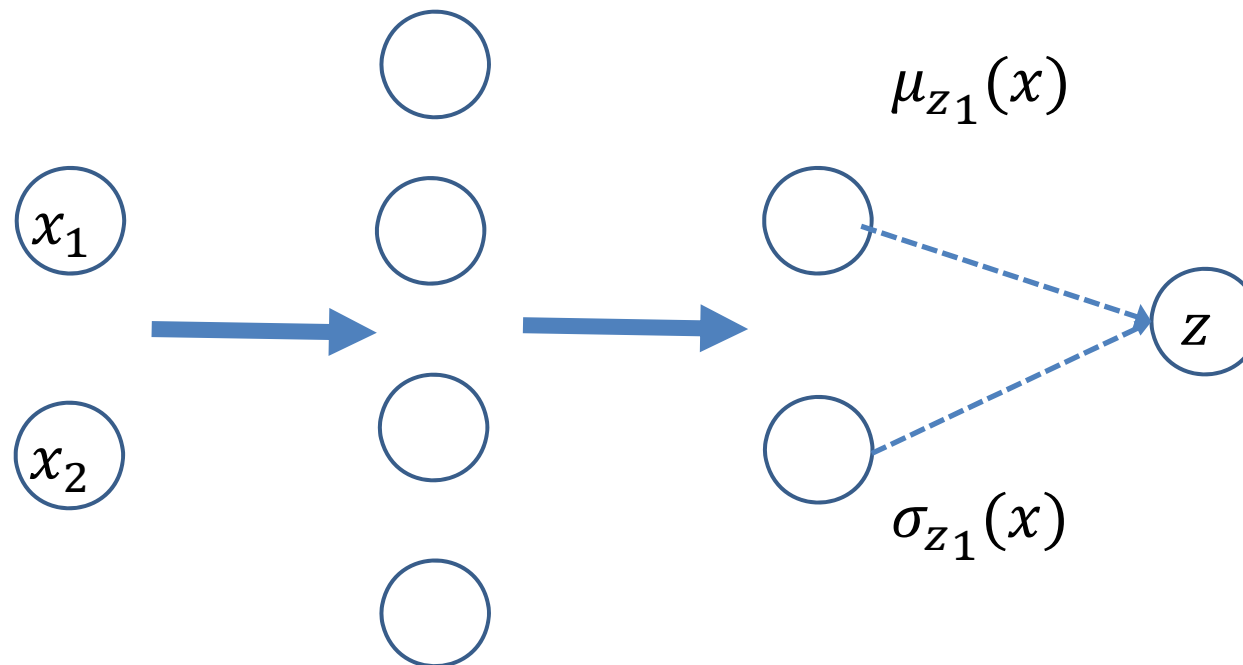


VAE

Gaussian priors and posteriors - illustration

► Encoder

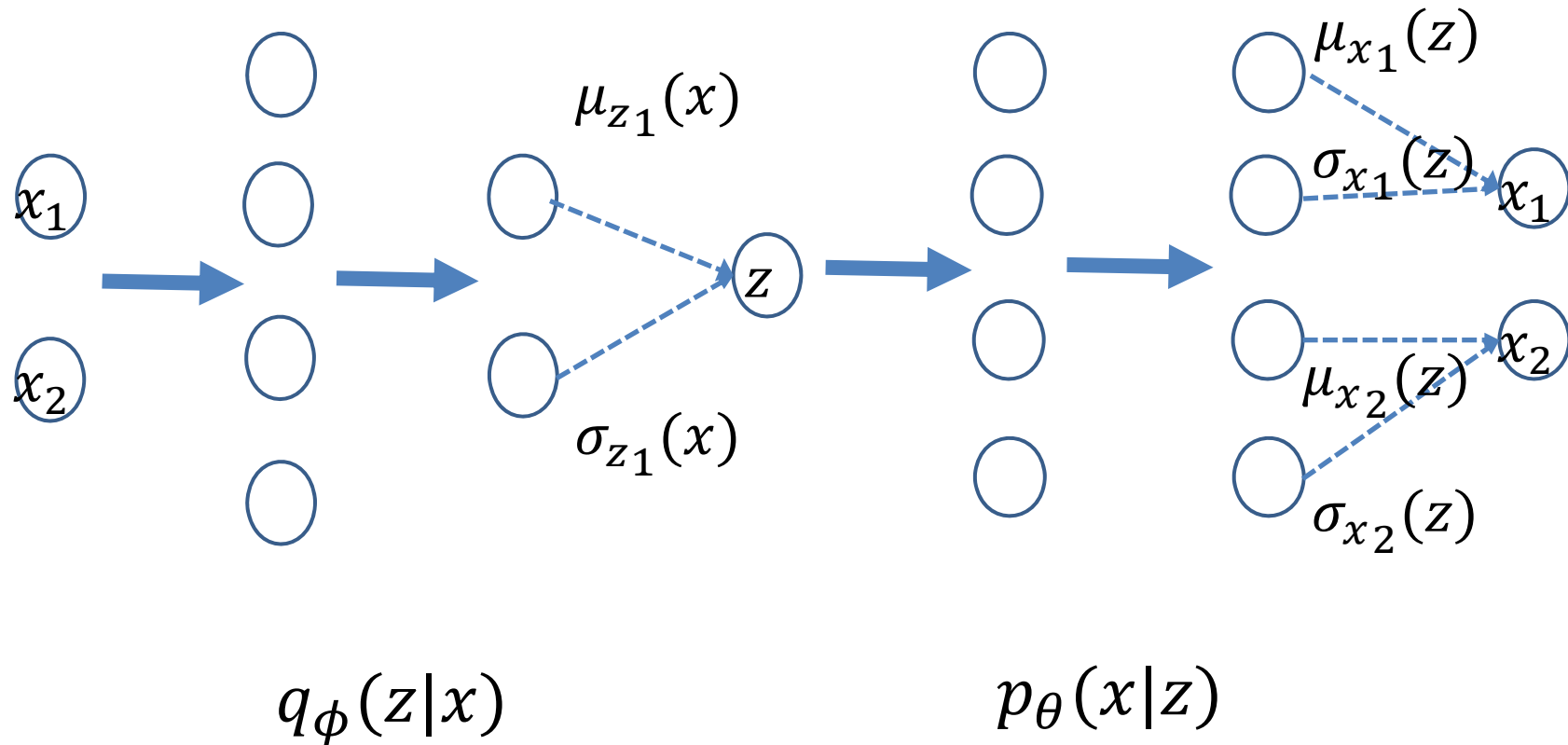
- in the example z is 1 dimensional and x is 2 dimensional, g is a 1 hidden layer MLP with gaussian output units and tanh hidden units
- full arrows: deterministic
- dashed arrows: sampling



VAE

Gaussian priors and posteriors

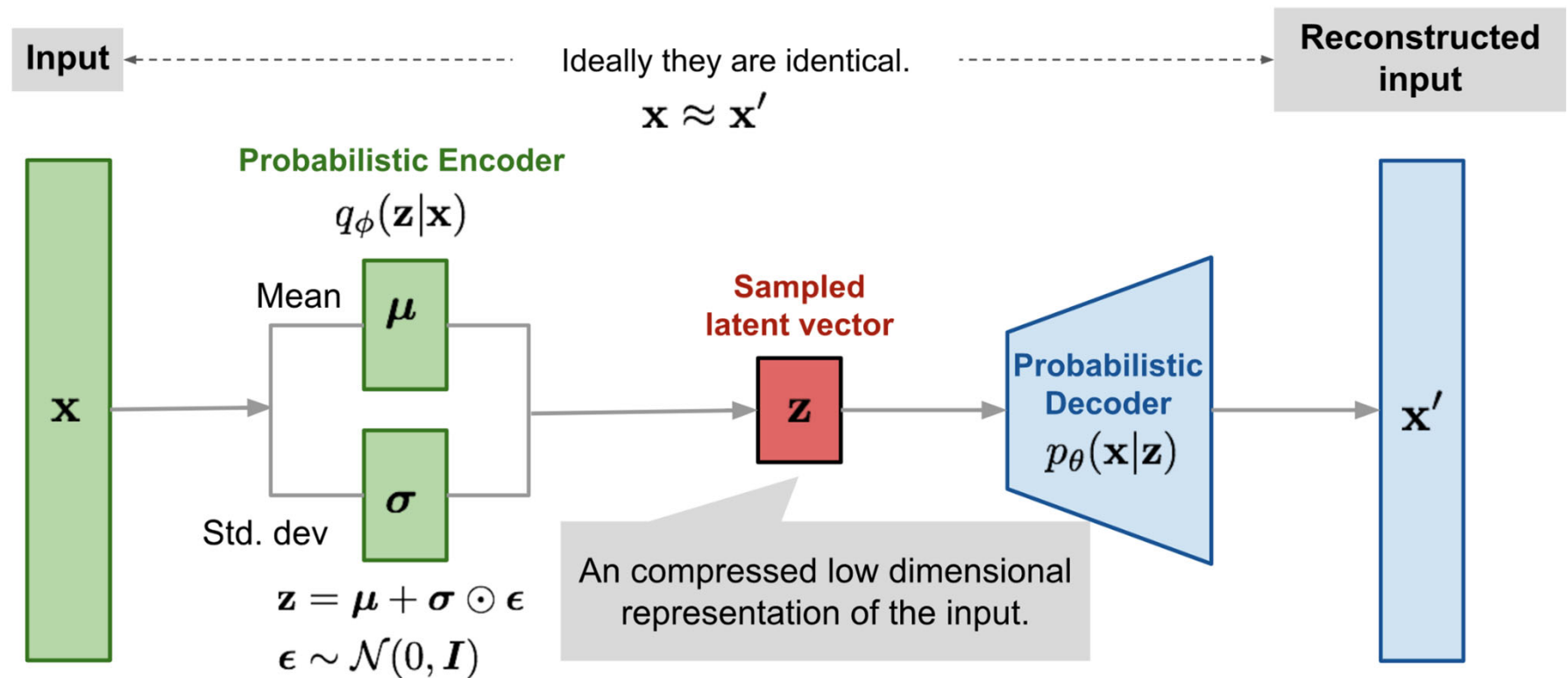
- ▶ Putting it all together



VAE

Gaussian priors and posteriors

► Additional illustration



<https://lilianweng.github.io/posts/2018-08-12-vae/>

VAE details for Gaussian priors and posteriors

VAE – instantiation example

Gaussian priors and posteriors

- ▶ Special case: gaussian priors and posteriors
- ▶ Hyp:
 - ▶ $p(z) = \mathcal{N}(0, I)$
 - ▶ $p_\theta(x|z) = \mathcal{N}(\mu(z), \sigma(z)), \sigma(z)$ diagonal matrix, $x \in R^D$
 - ▶ $q_\phi(z|x) = \mathcal{N}(\mu(x), \sigma(x)), \sigma(x)$ diagonal matrix, $z \in R^J$
- ▶ Variational lower bound
 - ▶ $V_L(\theta, \phi; x) = -D_{KL}(q_\phi(z|x)||p(z)) + E_{q_\phi(z|x)}[\log p_\theta(x|z)]$
 - ▶ In this case, $D_{KL}(q_\phi(z|x)||p(z))$ has an analytic expression (see next slide)
 - ▶ $-D_{KL}(q_\phi(z|x)||p(z)) = \frac{1}{2} \sum_{j=1}^J (1 + \log \left(\left(\sigma_{z_j} \right)^2 \right) - \left(\mu_{z_j} \right)^2 - \left(\sigma_{z_j} \right)^2)$
 - ▶ $E_{q_\phi(z|x)}[\log p_\theta(x|z)]$ is estimated using Monte Carlo sampling
 - ▶ $E_{q_\phi(z|x)}[\log p_\theta(x|z)] \simeq \frac{1}{L} \sum_{l=1}^L \log(p_\theta(x|z^{(l)}))$
 - ▶ $\log(p_\theta(x|z^{(l)})) = -(\sum_{j=1}^D \frac{1}{2} \log \left(\sigma_{x_j}^2(z^{(l)}) \right) + \frac{(x_j - \mu_{x_j}(z^{(l)}))^2}{2\sigma_{x_j}^2(z^{(l)})})$
 - ▶ i.e. L samples with $z^{(l)} = g_\phi(x, \epsilon^{(l)})$

VAE - instantiation example

Gaussian priors and posteriors (demos on next slides)

- ▶ If $z \in R^J : -D_{KL}(q_\phi(z|x)||p(z)) = \frac{1}{2} \sum_{j=1}^J (1 + \log((\sigma_j)^2) - (\mu_j)^2 - (\sigma_j)^2)$
- ▶ proof
 - ▶ $D_{KL}(q_\phi(z)||p(z)) = \int q_\phi(z) \log \frac{q_\phi(z)}{p(z)} dz$
- ▶ Consider the 1 dimensional case
 - ▶ $\int q_\phi(z) \log p(z) dz = \int \mathcal{N}(z; \mu, \sigma) \log \mathcal{N}(z; 0, 1) dz$
 - ▶ $\int q_\phi(z) \log p(z) dz = -\frac{1}{2} \log(2\pi) - \frac{1}{2} (\mu^2 + \sigma^2)$
 - ▶ Property of 2nd order moment of a Gaussian
 - ▶ $\int q_\phi(z) \log q_\phi(z) dz = \int \mathcal{N}(z; \mu, \sigma) \log \mathcal{N}(z; \mu, \sigma) dz$
 - ▶ $\int q_\phi(z) \log q_\phi(z) dz = -\frac{1}{2} \log(2\pi) - \frac{1}{2} (1 + \log \sigma^2)$
 - ▶
- ▶ Since both ddps are diagonal, extension to J dimensions is straightforward, hence the result

VAE - instantiation example

Gaussian priors and posteriors – demos for the 1 dimensional case

- ▶ Remember $q_\phi(z|x) = \mathcal{N}(\mu(x), \sigma(x))$
- ▶ Then $\int q_\phi(z) \log p(z) dz = \int \mathcal{N}(z; \mu, \sigma) \log \mathcal{N}(z; 0, 1) dz$
- ▶
$$\begin{aligned} &= E_{q_\Phi} [\log \mathcal{N}(z; 0, 1)] \\ &= E_{q_\Phi} \left[\log \left(\frac{1}{\sqrt{2\pi}} \exp \left(-\frac{z^2}{2} \right) \right) \right] \\ &= E_{q_\Phi} \left[-\frac{1}{2} \log 2\pi - \frac{z^2}{2} \right] \\ &= -\frac{1}{2} \log 2\pi - \frac{1}{2} E_{q_\Phi} [z^2] \end{aligned}$$
- ▶ What is the value of $E_q[z^2]$?
 - ▶ $E_{q_\Phi} [(z - \mu)^2] = \sigma^2$
 - ▶ $E_{q_\Phi} [z^2] - 2E_{q_\Phi} [z\mu] + \mu^2 = \sigma^2$
 - ▶ $E_{q_\Phi} [z\mu] = \mu^2$
 - ▶ $E_{q_\Phi} [z^2] = \mu^2 + \sigma^2$
- ▶ Then $\int q_\phi(z) \log p(z) dz = -\frac{1}{2} \log 2\pi - \frac{1}{2} (\mu^2 + \sigma^2)$

VAE - instantiation example

Gaussian priors and posteriors – demos for the 1 dimensional case

$$\begin{aligned}\blacktriangleright \int q_{\phi}(z) \log q_{\phi}(z) dz &= \int \mathcal{N}(z; \mu, \sigma) \log \mathcal{N}(z; \mu, \sigma) dz \\ &= E_{q_{\Phi}} \left[\log \left(\frac{1}{\sqrt{2\pi}\sigma} \exp \left(-\frac{(z-\mu)^2}{2\sigma^2} \right) \right) \right] \\ &= -\frac{1}{2} \log 2\pi - \log \sigma - E_{q_{\Phi}} \left[\frac{(z-\mu)^2}{2\sigma^2} \right] \\ &= -\frac{1}{2} \log 2\pi - \frac{1}{2} \log \sigma^2 - \frac{1}{2} \\ &= -\frac{1}{2} \log 2\pi - \frac{1}{2} (\log \sigma^2 + 1)\end{aligned}$$

VAE - instantiation example

Gaussian priors and posteriors

- ▶ Loss

- ▶ Regularization term

- ▶ $-D_{KL}(q_{\phi}(z|x)||p(z)) = \frac{1}{2}\sum_{j=1}^J(1 + \log((\sigma_j)^2) - (\mu_j)^2 - (\sigma_j)^2)$

- ▶ Reproduction term

- ▶ $\log(p(x|z)) = \sum_{j=1}^D \frac{1}{2} \log(\sigma_j^2(z)) + \frac{(x_j - \mu_j(z))^2}{2\sigma_j^2(z)}$

- ▶ Training

- ▶ Mini batch or pure stochastic

- ▶ Repeat

- $x \leftarrow$ random point or minibatch
 - $\epsilon \leftarrow$ sample from $p(\epsilon)$ for each x
 - $\theta \leftarrow \nabla_{\theta} V_L(\theta, \phi; x, g(\epsilon, \phi))$
 - $\phi \leftarrow \nabla_{\phi} V_L(\theta, \phi; x, g(\epsilon, \phi))$

- ▶ Until convergence

Learning discrete distributions: VQ-VAE (highlights)

- ▶ So far we considered continuous latent distributions
- ▶ There are several instances where discrete distributions are more appropriate
 - ▶ Text data, objects in images (color, size, orientation,...), etc
 - ▶ There are several algorithms, e.g. transformers designed to work with discrete data
 - ▶ **Dall-e – makes use of a discrete VAE together with transformers in order to generate diverse images**
 - ▶ <https://openai.com/blog/dall-e/>, <https://openai.com/dall-e-2/>
 - ▶ <https://gpt3demo.com/apps/openai-dall-e>
 - ▶ <https://www.craiyon.com/> (mini version of Dall-e)

Learning discrete distributions: VQ-VAE

- ▶ What is a discrete latent distribution?

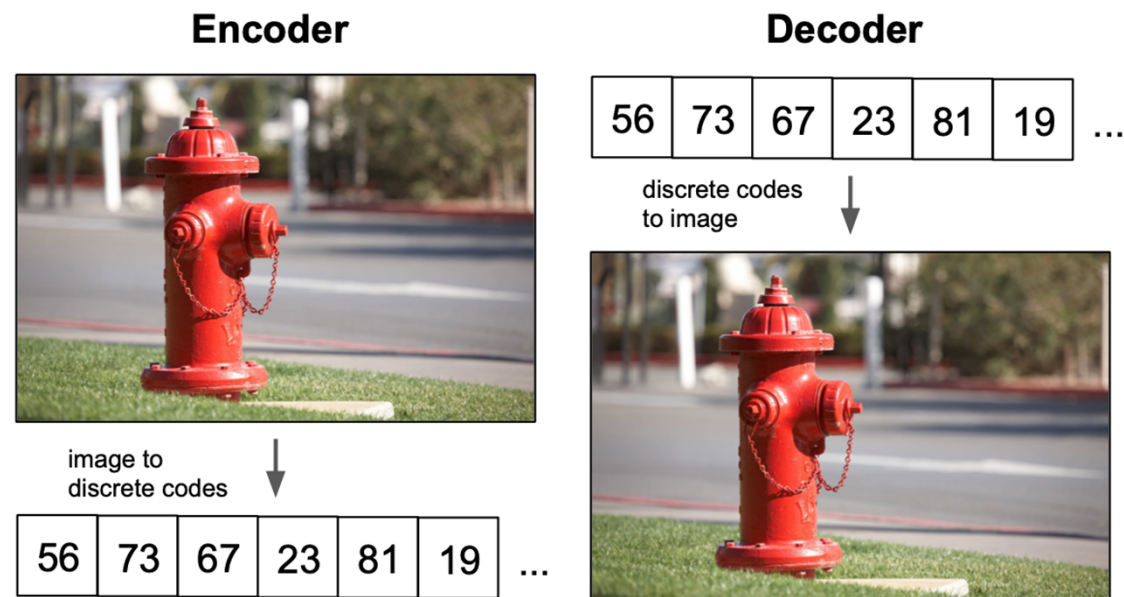
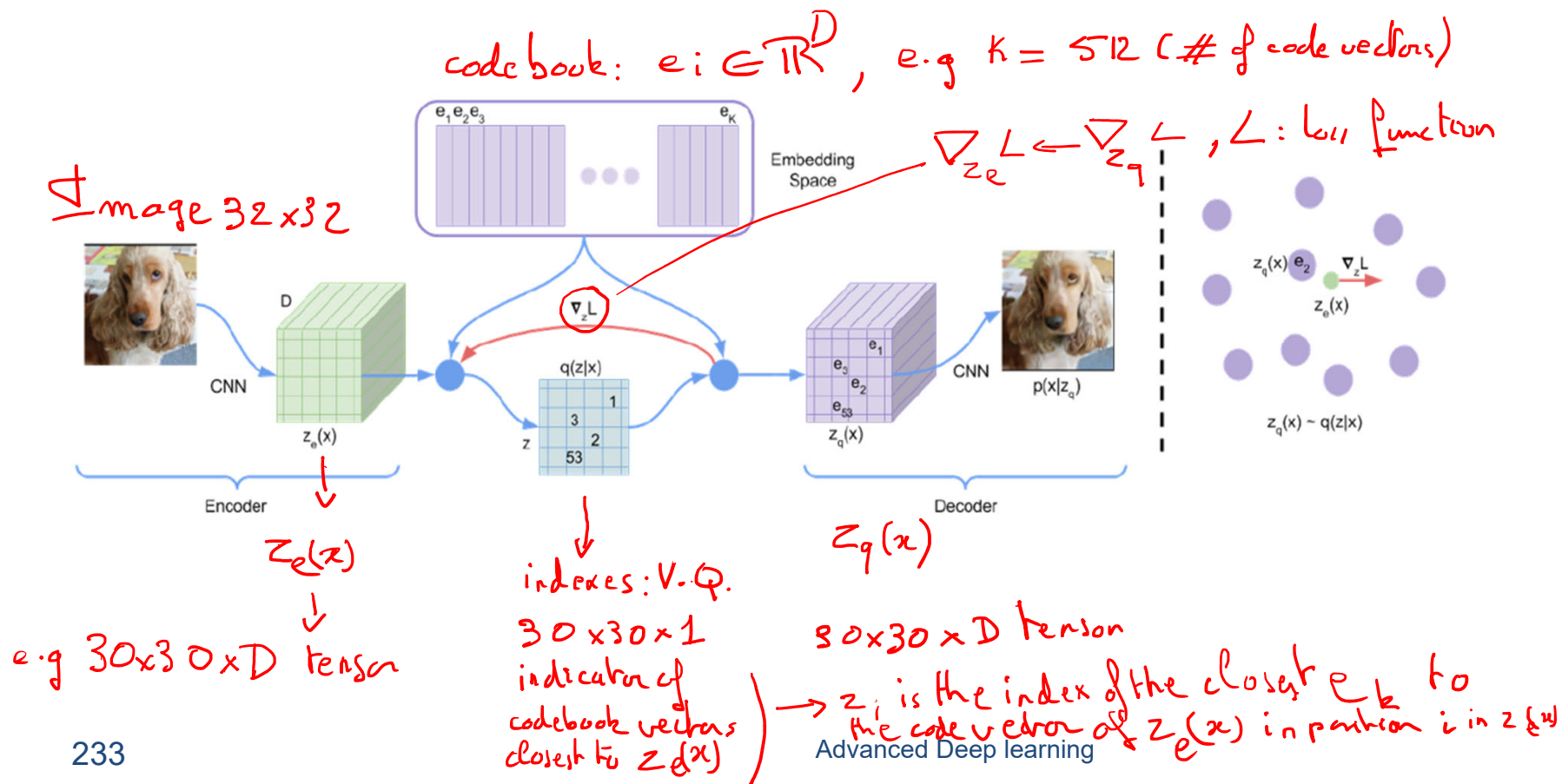


Fig: <https://ml.berkeley.edu/blog/posts/vq-vae/>

Learning discrete distributions: VQ-VAE

- ▶ VQ-VAE modifies the vanilla VAE by adding a discrete codebook of vectors to the VAE - It is used to quantize the VAE bottleneck
 - ▶ General scheme: VQ-VAE paper - <https://arxiv.org/pdf/1711.00937.pdf>



Learning discrete distributions: VQ-VAE

► Loss function

- $L = \|x - \text{Dec}(z_q(x))\|^2 + \|sg(z_e(x)) - z_q(x)\|^2 + \beta \|z_e(x) - sg(z_q(x))\|^2$
- With $sg(z)$ **stop gradient**, i.e. do not back-propagate through z
 - $\|x - \text{Dec}(z_q(x))\|^2$: train **decoder** and **encoder**
 - $\|sg(z_e(x)) - z_q(x)\|^2$: train the **codebook** $e = z_q(x)$
 - $\|z_e(x) - sg(z_q(x))\|^2$: train **encoder**, forces $z_e(x)$ to stay close to $e = z_q(x)$
 - This is because the codebook does not train as fast as the encoder and the decoder
 - Prevents the encoder values to diverge

► Gradients

- Since it is not possible to compute the gradient through the VQ component, it is proposed to simply copy the gradient w.r.t. z_q to z_e
- $\nabla_{z_e(x)} \|x - \text{Dec}(z_q(x))\|^2 = \nabla_{z_q(x)} \|x - \text{Dec}(z_q(x))\|^2$
- This is called **straight-through gradient**

► Note

- This is an incomplete description, the model requires additional steps
- Dall-e makes use of a slightly different discrete VAE (called dVAE)

► References

► Nice blogs explaining VAEs

- <https://lilianweng.github.io/posts/2018-08-12-vae/>
- <https://jaan.io/what-is-variational-autoencoder-vae-tutorial/>
- <https://www.fenghz.xyz/vector-quantization-based-generative-model/>
- Luo, C. (2022). Understanding Diffusion Models: A Unified Perspective.
<http://arxiv.org/abs/2208.11970> - positions hierarchical VAEs w.r.t diffusion models

► Blogs introducing variational inference

- <https://blog.evjang.com/2016/08/variational-bayes.html>
- <https://towardsdatascience.com/bayesian-inference-problem-mcmc-and-variational-inference-25a8aa9bce29>

► Papers

- Kingma, D. P., & Welling, M. (2014). Auto-Encoding Variational Bayes. *ICLR (2014)*, MI, 1–14. <http://arxiv.org/abs/1312.6114>

Flow Matching

Used in different forms in many recent image/ video generators, e.g. Google Banana, OLpenAI Sora, Google Veo, ...

Flow matching - key notions

► Trajectory

- A trajectory is a function that maps time to a location in $\mathbb{R}^d$

- $x(t): [0,1] \rightarrow \mathbb{R}^d$

1D

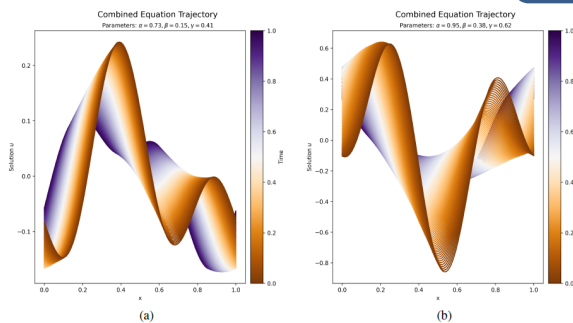


Figure 4: Samples from the Combined Dataset.

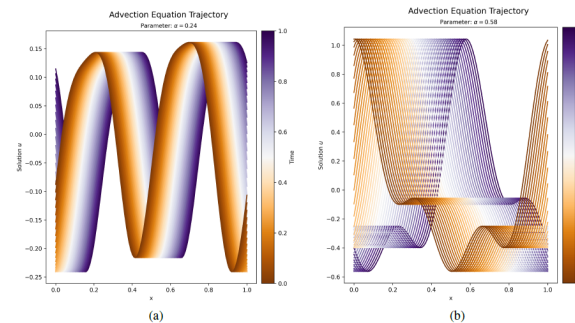
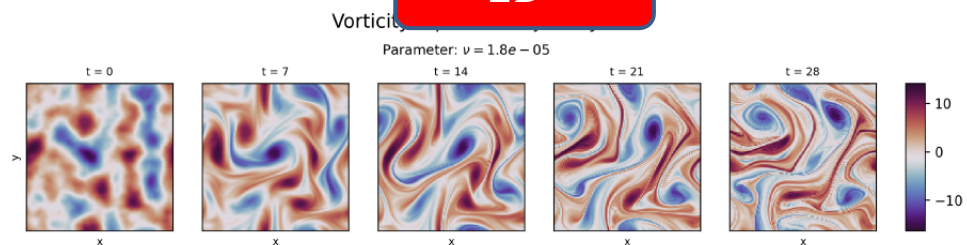


Figure 6: In-domain samples from the Advection Dataset.

2D



Flow matching - key notions

► Velocity field

- A vector field v_t assigns a velocity vector to every point $x \in \mathbb{R}^d$
- $v_t: [0,1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$
- It describes the instantaneous velocity (direction and speed) of a « particle » at each time t and position x
- You can visualize it as a field of arrows, where each arrow shows how a particle located at x would move at that instant.

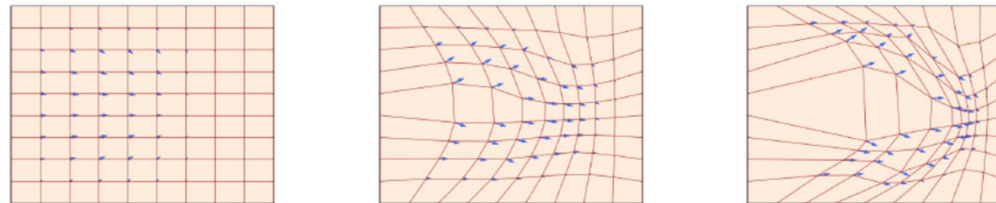


Figure 6 A flow $\psi_t: \mathbb{R}^d \rightarrow \mathbb{R}^d$ (square grid) is defined by a velocity field $u_t: \mathbb{R}^d \rightarrow \mathbb{R}^d$ (visualized with blue arrows) that prescribes its instantaneous movements at all locations. We show three different times t .

Fig. Lipman et al. 2024

Flow matching - key notions

► Ordinary differential equation ODE

- An ODE describes the evolution of a system that depends of a variable, usually called time
- $\frac{d}{dt}x(t) = v_t(x(t))$
- $x(0) = x_0$ - initial condition

► Flow of the ODE

- The **flow** Φ_t is the *map* that takes an initial condition x_0 and a time t , and gives the state of the system at that time: $\Phi_t(x_0) = x(t; x_0)$
- $\Phi_t: [0,1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d, (x_0, t) \rightarrow \Phi_t(x_0)$
- $\frac{d}{dt}\Phi_t(x_0) = v_t(\Phi_t(x_0))$
- $\Phi_t(x_0) = x_0$ - initial condition
- The flow is determined by the vector field

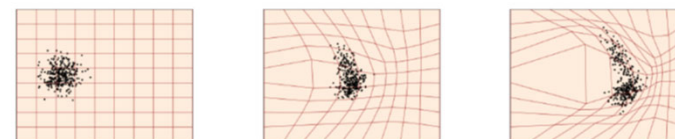


Figure 5 A flow model $X_t = \psi_t(X_0)$ is defined by a diffeomorphism $\psi_t: \mathbb{R}^d \rightarrow \mathbb{R}^d$ (visualized with a brown square grid) pushing samples from a source RV X_0 (left, black points) toward some target distribution q (right). We show three different times t .

Fig. Lipman et al. 2024

Flow matching - key notions

Vector fields define ODEs - ODE solutions are flows

Concept	definition	Relation
Vector field $v_t(x)$	Function assigning a velocity vector to each point	Defines the dynamics
Trajectory $x(t; x_0)$	Solution curve starting from x_0	Integral curve of v_t
Flow $\Phi_t(x_0)$	Map giving position after time t	Generated by integrating v_t

Flow matching - key notions

- ▶ To *simulate* the flow, we need to **numerically integrate** the ODE. Time is discretized between 0 and 1: $0, h, 2h, \dots, 1$
- ▶ The simplest numerical integration method is **Euler's scheme**:

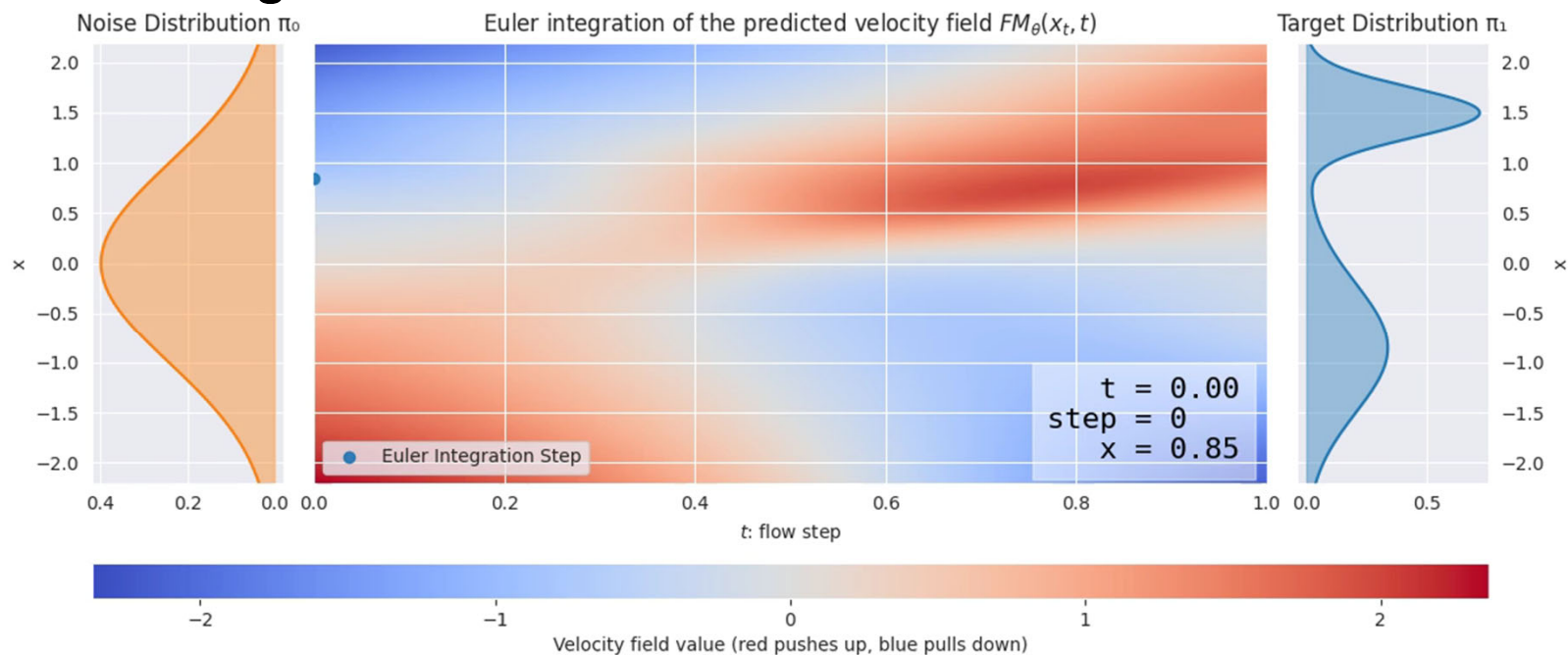
Start at x_0

Iterate $x_{k+h} = x_k + hv_t(x_k), k = 0, h, \dots, 1 - h$

- ▶ This approximates the true trajectory $x_{k+h} \approx x(t_{k+h})$

Flow matching - key notions

► Euler integration



- Velocity field is shown as a heatmap
- The figure shows the integration of a sample from the noise distribution to the target distribution (an approximation of the true trajectory), using the velocity field

Credits: https://peterroelants.github.io/posts/flow_matching_intro/

Flow matching

► Objective

- Learn a mapping from distribution $p_0(x)$ to a target distribution $p_1(x)$

► Probability path

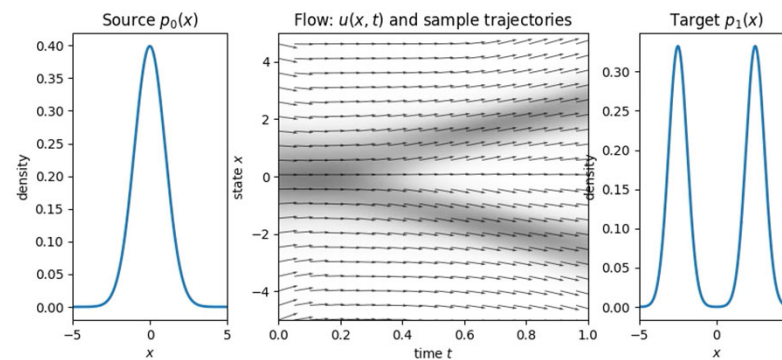
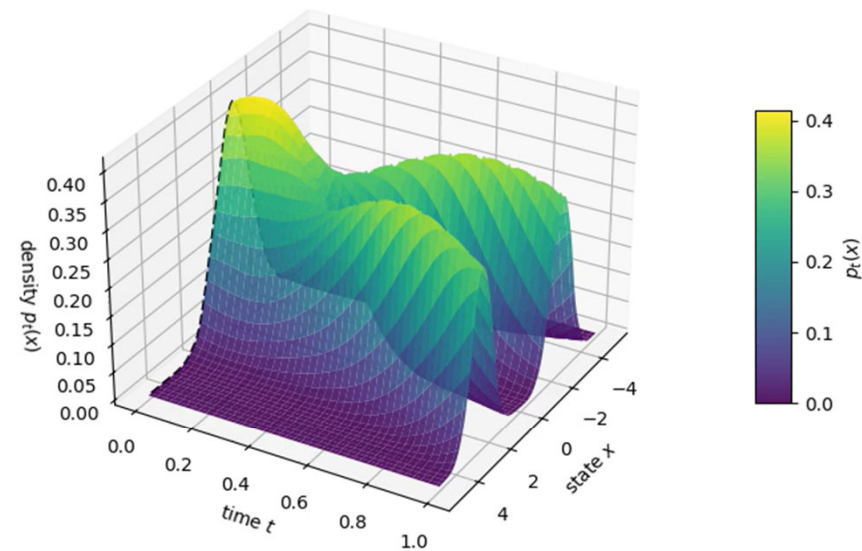
- Flow matching learns a **probability path** $p_t, t \in [0,1]$.
- This is a time dependent probability that interpolates between two distributions p_0 and p_1
 - $p_t: [0,1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$
 - $p_t(t, \cdot)$ is a probability function of variable x
- The flow is defined by an **ODE**

$$\begin{aligned} x_0 &\sim p_0 \\ \frac{dx_t}{dt} &= v_t^\theta(x_t) \\ \text{with } x_t &= \Phi_t(x_0) \end{aligned}$$

- Hence the objective of flow matching is to **learn a parameterized vector field** $v_t^\theta(x_t)$

Illustration: probability path and velocity field

Probability path surface ($t = 0.00$)



Flow matching inference

- ▶ Sampling (inference)
- ▶ Once the parameterized vector field is learned, the target distribution can be sampled by solving the ODE

$$x_0 \sim p_0$$

Integrate the ODE $\frac{dx_t}{dt} = v_t^\theta(x_t)$ from $t = 0$ to $t = 1$

e.g. Euler integration $x_{t+h} = x_t + hv_t^\theta(x_t)$

Output x_1

Training

- ▶ Flow matching loss

- ▶ v_t^θ should approximate the a target field v_t

- ▶ The **flow matching loss** is defined as:

- ▶ $\mathcal{L}_{FM} = E_{t \sim U[0,1], x_t \sim p_t} [\|v_t^\theta(x_t) - v_t(x_t)\|^2]$

- With v_t a target field and v_t^θ its parametric approximation

- ▶ Target v_t is unknown (it is untractable), flow matching proposes to optimize instead a conditional loss

- ▶ $\mathcal{L}_{CFM} = E_{t \sim U[0,1], x_1 \sim p_1(x), x_t \sim p_t(x_t|x_1)} [\|v_t^\theta(x_t) - v_t(x_t|x_1)\|^2]$

- ▶ The conditional field $v_t(x_t|x_1)$ will have a **tractable analytical expression**, enabling the training - We have the following property;

$$\nabla_\theta \mathcal{L}_{FM} = \nabla_\theta \mathcal{L}_{CFM}$$

minimizing $\mathcal{L}_{CFM}$ is equivalent to miminizing $\mathcal{L}_{FM}$

Training

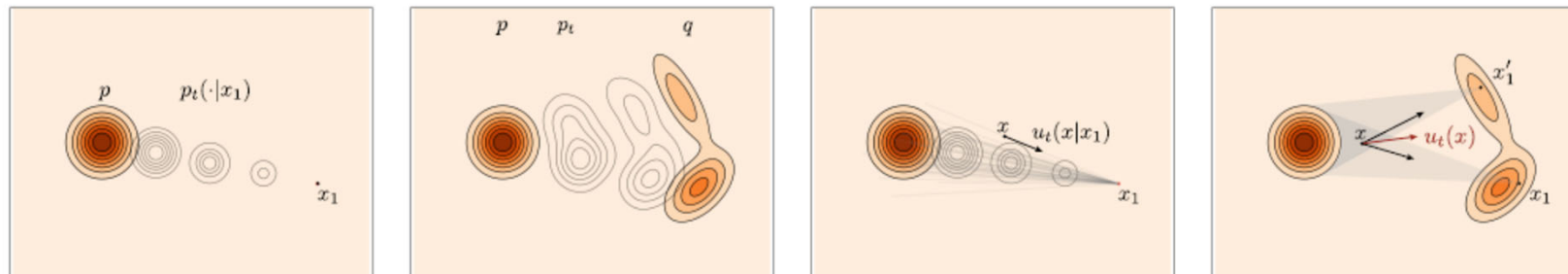
- ▶ CFM introduces an anchor point $x_1 \sim p_1$
 - ▶ Instead of modeling directly the marginal p_t on the whole domain of p_1 , training is performed around this anchor point
 - ▶ Then by sampling several target points x_1 one progressively aggregates the local informations in the target space
 - ▶ This simplifies the problem of modeling the marginal p_t because one can often find a conditional vector field satisfying the following equation analytically by hand.

$$x_0 \sim p_0, \frac{dx_t}{dt} = v_t(x_t|z) \Rightarrow x_t \sim p_t(x|z), t \in [0,1]$$

Conditionals and marginals probability paths

- ▶ The **marginal probability path** p_t results from the aggregation of the **conditional probability paths** $p_{t|1}$ for multiple samplings x_0, x_1

$$p_t(x) = \int p_{t|1}(x|x_1)p_1(x_1)dx_1$$



(a) Conditional probability path $p_t(x|x_1)$. **(b)** (Marginal) Probability path $p_t(x)$. **(c)** Conditional velocity field $u_t(x|x_1)$. **(d)** (Marginal) Velocity field $u_t(x)$.

Figure 3 *Path design in Flow Matching.* Given a fixed target sample $X = x_1$, its conditional velocity field $u_t(x|x_1)$ generates the conditional probability path $p_t(x|x_1)$. The (marginal) velocity field $u_t(x)$ results from the aggregation of all conditional velocity fields—and similarly for the probability path $p_t(x)$.

Fig. Lipman et al. 2024

Training: building the conditional probability path and the conditional vector field

- ▶ Considering $\mathcal{L}_{CFM}$, one needs tractable expressions for $p_t(x_t|x_1)$ and $v_t(x_t|x_1)$
 - ▶ There is an infinite number of (conditional) probability paths mapping a distribution p_0 to a target distribution p_1 (and p_0 to x_1)

- ▶ A popular choice for the conditional path is the linear path:

$$p_{t|1}(x|x_1) = \mathcal{N}(x|tx_1, (1-t)^2I)$$

- ▶ Using this probability path, let us define the following random variable

$$x_t = \Phi_t(x_0) = tx_1 + (1-t)x_0 \text{ with } x_0 \sim p_0 = \mathcal{N}(O, I)$$
$$x_t \sim \mathcal{N}(x|tx_1, (1-t)^2I)$$

- ▶ Given $x_0 \sim \mathcal{N}(O, I)$ and $x_1 \sim p_1$, the flow x_t follows a linear path between x_0 and x_1
- ▶ The flow defines the following vector field

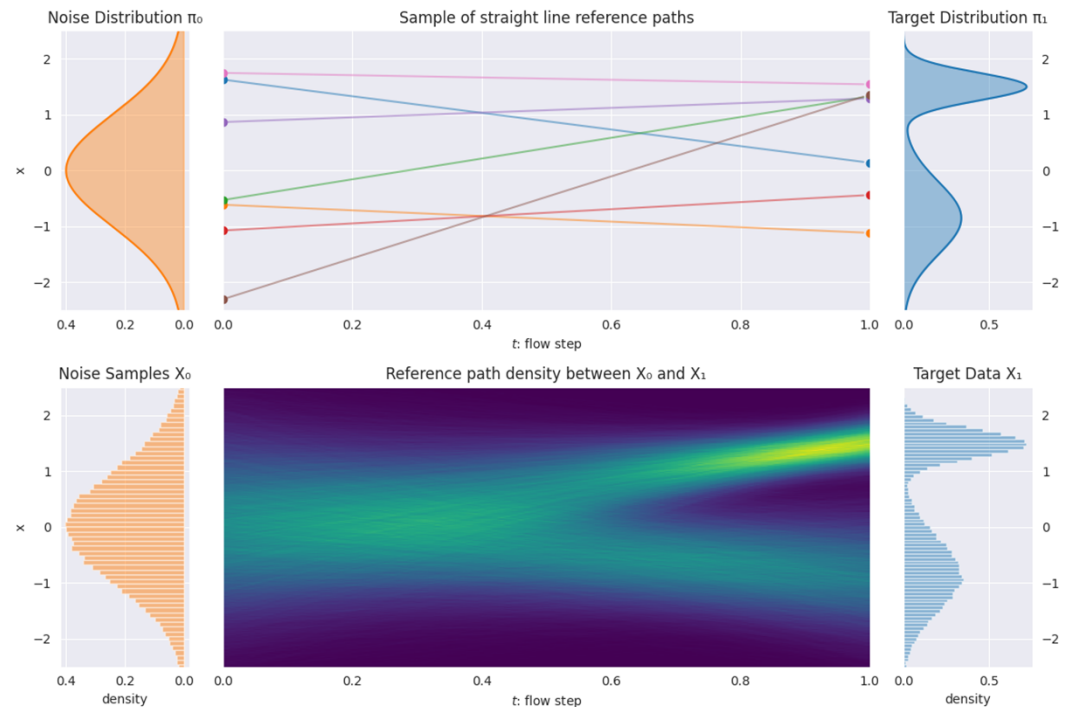
$$v_t(x_t|x_1) = \frac{d}{dt}x_t = -x_0 + x_1$$

this is obtained by simple differentiation of the field $\Phi_t(x_0)$ expression

- ▶ And the loss $\mathcal{L}_{CFM}$ becomes

$$\mathcal{L}_{CFM} = E_{t \sim U[0,1], x_1 \sim p_1(x), x_t \sim p_t(x_t|x_1)} \left[\|v_t^\theta(x_t) - (x_1 - x_0)\|^2 \right]$$

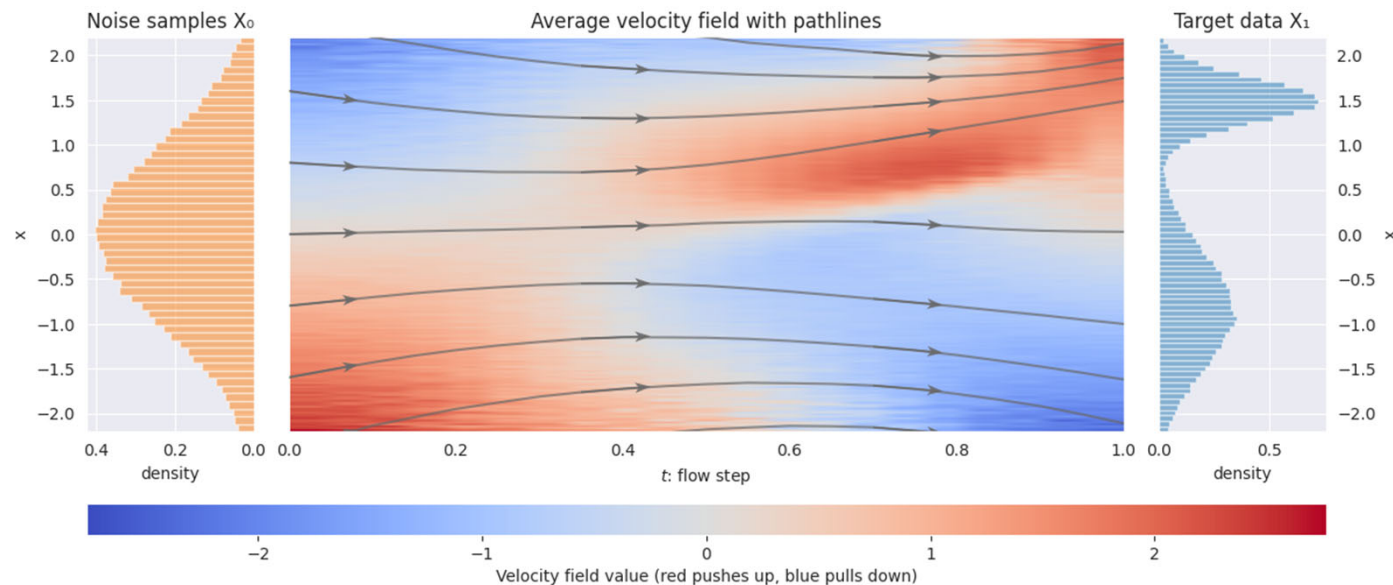
Training: Building the conditional probability path and the conditional vector field



- Top: sample straight line reference paths
- Bottom: reference paths distribution generated by sampling a large nb of straight line reference paths

Credits: https://peterroelants.github.io/posts/flow_matching_intro/

Training: Building the conditional probability path and the conditional vector field



Average flow field computed by sampling a large nb of reference paths and computing the average velocity for fixed bins over the flow field

Credits: https://peterroelants.github.io/posts/flow_matching_intro/

Training algorithm

- ▶ This leads to the following training algorithm

Sample $x_0 \sim p_0$, $x_1 \sim p_1$, $t \sim U[0,1]$

Compute $x_t = \Phi_t(x_0) = (1 - t)x_0 + tx_1$

Compute the loss $\mathcal{L} = \|v_t^\theta(x_t) - (x_1 - x_0)\|^2$

Gradient descent $\theta = \theta - \epsilon \nabla_\theta \mathcal{L}$

Flow matching

A few explanations

Flow existence and uniqueness

Theorem 1

If $v: [0,1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$ is continuously differentiable with a bounded derivative, the ODE, $\frac{d}{dt}\Phi_t(x_0) = v_t(\Phi_t(x_0))$, $\Phi_t(x_0) = x_0$, has a unique solution given by flow Φ_t . Φ_t is a diffeomorphism for all t , i.e. continuously differentiable with a continuously differentiable inverse.

The marginalization trick used in flow matching

- Explains why the conditional vector fields allow us to build the marginal field

Theorem 2

For $z \in \mathbb{R}^d$, let $v_t(\cdot | z)$ denote a conditional vector field, whose corresponding ODE defines a conditional probability path $p_{t|z}(\cdot | z)$:

$$x_0 \sim p_0, \frac{dx_t}{dt} = v_t(x_t | z) \Rightarrow x_t \sim p_t(x | z), t \in [0, 1]$$

Then the marginal vector field $v_t(x)$ defined as:

$$v_t(x) = \int v_t(x | z) \frac{p_t(x | z) p_1(z)}{p_t(x)} dz$$

Follows the marginal probability path:

$$x_0 \sim p_0, \frac{dx_t}{dt} = v_t(x_t) \Rightarrow x_t \sim p_t \quad t \in [0, 1]$$

Continuity equation

Theorem 3

Let a flow model defined by its vector field v_t and initial condition $x_0 \sim p_0$, then $x_t \sim p_t, t \in [0,1]$, if and only if the continuity equation is verified

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot (p_t v_t)(x), \forall x \in \mathbb{R}^d, t \in [0,1]$$

Where $\nabla \cdot$ is the divergence operator:

$$\nabla \cdot v(x) = \sum_{i=1}^d \frac{\partial v_t(x)}{\partial x_i}, \text{ and } x = (x_1, \dots, x_d)^T$$

- ▶ The continuity equation expresses the conservation of quantities like physical mass, energy, probability mass.
 - ▶ Here, $\frac{\partial p_t(x)}{\partial t}$ describes how much p_t changes over time, and the negative divergence $-\nabla \cdot (p_t v_t)$ expresses the total inflow of probability mass.
 - ▶ Probability mass being conserved, the two quantities must be equal

Using the continuity equation to demonstrate the marginalization trick



$$\begin{aligned}\frac{\partial}{\partial t} p_t(x) &\stackrel{(i)}{=} \frac{\partial}{\partial t} \int p_t(x | z) p_1(z) dz \\ &= \int \frac{\partial}{\partial t} p_t(x | z) p_1(z) dz \\ &\stackrel{(ii)}{=} \int -\nabla \cdot (p_t(x | z) v_t(x | z)) p_1(z) dz \\ &\stackrel{(iii)}{=} -\nabla \cdot \left(\int p_t(x | z) v_t(x | z) p_1(z) dz \right) \\ &\stackrel{(iv)}{=} -\nabla \cdot \left(p_t(x) \int v_t(x | z) \frac{p_t(x | z) p_1(z)}{p_t(x)} dz \right) \\ &\stackrel{(v)}{=} -\nabla \cdot (p_t(x) v_t(x)) .\end{aligned}$$

Note: (ii) uses the continuity equation for the conditional probability path $p_t(x|z)$

By theorem 3 this demonstrates theorem 2

Training



Theorem

The marginal flow loss $\mathcal{L}_{FM}$ equals the conditional flow matching loss $\mathcal{L}_{CFM}$ up to a constant, hence their gradients coincide

$$\nabla_{\theta} \mathcal{L}_{FM} = \nabla_{\theta} \mathcal{L}_{CFM}$$

and minimizing $\mathcal{L}_{CFM}$ is equivalent to minimizing $\mathcal{L}_{FM}$

References on flow matching

- ▶ Tutorial and great site at MIT
 - ▶ <https://diffusion.csail.mit.edu/>
 - ▶ Course notes <https://arxiv.org/abs/2506.02070>
- ▶ A simple visual introduction to flow matching
 - ▶ https://peterroelants.github.io/posts/flow_matching_intro/
- ▶ An easy to follow introduction
 - ▶ <https://nilsschaetti.ch/2025/02/02/flow-matching-comprendre-les-derniers-modeles-de-generation-dimages/#:~:text=Qu'est%2Dce%20que%20le,texte%20ou%20m%C3%AAme%20des%20vid%C3%A9os.>
- ▶ Reference papers
 - ▶ Lipman, Y., Havasi, M., Holderrieth, P., Shaul, N., Le, M., Karrer, B., Chen, R. T. Q., Lopez-Paz, D., Ben-Hamu, H., & Gat, I. (2024). Flow Matching Guide and Code. <http://arxiv.org/abs/2412.06264> (lots of details but more tricky than the course note above)
 - ▶ Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., Le, M., & Ai, M. (2023). Flow Matching for Generative Modeling ICLR.